

Universidad Tecnológica de La Habana
“José Antonio Echeverría”



Facultad de Ingeniería Informática

**Nueva versión de la biblioteca de heurísticas de
construcción para problemas de planificación de
rutas de vehículos**

Trabajo para optar por el título de Ingeniero Informático

Autor: Ananda de la Caridad Morales Morales

Tutores:

Dr. C. Isis Torres Pérez

Dr. C. Alejandro Rosete Suárez

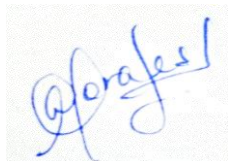
La Habana, Cuba

12 de marzo de 2025

Declaración de autoría

Declaro que soy el único autor de este trabajo y autorizo a la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE) para que hagan el uso que estimen pertinente con este trabajo.

Para que así conste firmo la presente a los 11 días del mes de marzo de 2025.



Ananda de la Caridad Morales Morales

Firma del autor



Dr. C. Isis Torres Pérez

Firma del primer tutor



Dr. C. Alejandro Rosete Suárez

Firma del segundo tutor

OPINIÓN DE LOS TUTORES

El trabajo de diploma “**Nueva versión de la biblioteca de heurísticas de construcción para problemas de planificación de rutas de vehículos**” para optar por el título de Ingeniería en Informática, presentado por la estudiante **Ananda de la Caridad Morales Morales**, se enfoca en un tema de interés en el mundo actual: la optimización de rutas.

Particularmente, la tesis presenta una nueva versión de una biblioteca que brinda numerosas heurísticas para diversos problemas de optimización de rutas de transportación. Como aspectos singulares de esta nueva versión están la incorporación de nuevas heurísticas, un rediseño de la biblioteca y la opción de uso en Python y Java. La tesis incorpora un extenso estudio experimental.

El documento muestra el amplio dominio de la estudiante de los temas mostrados, así como para su capacidad para organizar coherentemente los resultados que obtuvo.

En todo momento, la estudiante mostró alta responsabilidad e independencia.

Adicionalmente, durante los años de su formación como ingeniera, se ha destacado por su trabajo como alumna ayudante, tanto en docencia directa como participando en eventos científicos.

Por todas estas razones, los tutores consideramos que **Ananda de la Caridad Morales Morales** está lista para ejercer como Ingeniera Informática, y proponemos la máxima calificación.

Tutores:

Dr. C. Isis Torres Pérez

Dr. C. Alejandro Rosete Suárez



Cujae, La Habana, Cuba, Marzo 2025

En memoria de Javier Marrero.

Agradecimientos

A mi mamá, por dedicarme cada átomo de su ser. Gracias por ser la mejor, mi salvadora, mi paz y amor, mi “todo va a estar bien”, mi guerrera, por ayudarme a crecer, a levantarme y a nunca rendirme. Te amo y extraño mucho.

A mi papá, por ser mi héroe, mi ídolo y mi competencia. Gracias papito, por ser mi motor impulsor, por ti me he crecido para ser una Ingeniera a tu altura y para hacerte sentir el hombre más orgulloso del mundo. Te adoro con la vida.

A mis abuelitos, Maricela y Gilberto, por siempre recordarme la importancia de ser universitaria, por todo el amor incondicional y por consentirme lo más posible.

A mi mejor amiga, mi hermana, mi alma gemela, Amanda. Gracias por convertirme en mi pedacito de cielo e iluminarme cada día, por confiar en mí cuando ni yo misma podía, por no dejarme caer nunca y porque este título es tuyo también princesa.

A mi perro y mi gatico, gracias por demostrar el significado de la fidelidad en cada noche mientras trabajaba y no permitir sentirme sola estos últimos tiempos.

A la familia Chamorro-Calás y Yami, por haberse convertido en mi segunda familia. Gran parte de la mujer que soy ahora se los debo a uds y los llevo en el corazón.

A mi tutora Isis, por confiar en mi talento cuando me sentía perdida durante la carrera, por transmitirme toda su sabiduría, por motivarme en el mundo de la investigación, por su entrega y dedicación. Gracias a usted tengo el placer de poder decir, soy Ingeniera.

A mi tutor Rosete, por ser el profe que más admiro en el mundo. Gracias a usted me enamoré de la carrera y he podido convertirme en la profesional que soy hoy. Gran parte de la inspiración para seguir adelante, ser profe e investigadora es por seguir su ejemplo.

A mis amigos más antiguos Gaby, Greisy, Deynerd y Felipe, por nunca dudar de mí y siempre apoyarme a conseguir mi sueño. Gracias por cada consejo, cada abrazo y por siempre estar ahí cuando más lo necesité.

A mi amigo y compañero de proyectos, Miguel Angel. Andar juntos ha sido de las mejores decisiones que he tomado en la vida y aunque se termine la carrera seguiremos siendo el mejor dúo programador-redactora.

A los primeros amigos que hice en la universidad, Esperancito y Luisimi. Gracias por haber estado estos “5 años” de carrera en la misma aula y haber compartido tantas risas, chismes y altibajos.

A la profe Raisa, por ser mi mamá versión informática. Gracias por el apoyo en momentos difíciles y por haber tenido el placer de dar clases junto a usted.

A la profe Ariadna, por ser la profe joven que me inspiró a seguir sus pasos. Gracias a sus clases de Matemática Discreta y Computacional, así como sus logros profesionales, he tratado de seguir ese camino 4 años después.

Al profe Eduardo, por sus magníficas clases, sus consejos, por todos esos días de conversaciones *random* en su oficina, por aquel helado en El Palenque, y por ser más que un profe, un amigo.

A la profe Diansy, por ser la mejor compañera de trabajo que pude imaginar. Ha sido un placer para mí dar clases junto a ti y tener esta bonita amistad a pesar de los añitos de diferencia. Gracias por consentirme.

A los profes Tony y Lisandra, por ser testigos de mi trayectoria durante toda la carrera, por ser íconos de espíritu aventurero, por la confianza que han depositado en mí y por haberme hecho sentir importante.

A las profes Mayi y Nayma, por la confianza, amor y dedicación, así como cada consejo milimétrico para ser la mejor ingeniera posible.

A los profes Alfredo y Carbonell, por el apoyo en los últimos tiempos para todo lo relacionado con el campo investigativo.

A mis compañeros de equipo, Dayana, Andy, Omar y Eric. Muchas gracias, me ha encantado la experiencia de ser “los de las guagüitas” y espero que sigamos creciendo como profesionales juntos.

A mis alumnos, por ser fuente de motivación. Gracias a ustedes cada día que creo que voy a caer, me levanto para servirles de inspiración y hacerlos sentir orgullosos al punto de decir “esa es mi profe”. En especial a Luisma, Miguel, Manuel, Arango, Danna, Kevin, Dariel y Alberto del antiguo grupo 14; a Anthony, Amanda, Rafa, Jorge y Ryan del antiguo grupo 13; a Marcos, Kendry, Xavier, Emanuel, Jade, Herbert y Diego del actual grupo 11. Los adoro.

A Sire, por iluminar todos mis días en la facultad, por todo el cariño y por ser una abuelita para mí.

A mis compañeros de aula y de año, por depositar tanta confianza en mí. Para mí ha sido un placer poder ayudarlos a todos con cada mensajito de recordatorio de alguna prueba o entrega, con cada consejo para los informes, con cada crítica a los ppts, en fin.

A los amigos de otros años que he recopilado a través de reuniones, eventos, aclaración de dudas, etc. Gracias Jackie, Adolfo, Patry, Johnny, Rubén, Mariam, Tachiri, Andy, Brenda, entre otros. Es que son muchos, perdón si se me queda alguno :(

A Adrián, porque a pesar de haber coincidido hace poco, eres una de las personas que más estima me tiene y confía en mi potencial. Gracias por recordarme que siempre puedo, por motivarme a escribir cada artículo, participar en cada evento, por apoyarme en mi sueño y por regalarme todos esos abrazos que me calman el alma cuando más ansiosa estoy y solo necesito un lugar seguro.

Al señorito Carrasco, por contagiarme con su juventud y enseñarme a ver la vida de otra manera. Gracias por devolverme el brillo que necesitaba para ser una mejor versión de mí.

Por último, pero no menos importante, a Javier. Esta tesis va dedicada a ti, *little boy*. En vida me enseñaste muchísimas cosas y fuiste un amigo genial. Después de tu partida, me apropié de algunos de tus sueños para hacerlos realidad. Espero que desde allá arriba me mires orgulloso...

De manera general, agradecida con todos los que han formado parte de mi vida y han contribuido a mi formación como persona, estudiante y profesional.

Resumen

Los Problemas de Planificación de Rutas de Vehículos cuentan con una flota de vehículos, buscan satisfacer las demandas de clientes dispersos geográficamente y tienen como objetivo minimizar costos siempre que se cumplan con las restricciones existentes. A diario se producen nuevas situaciones prácticas en el mundo, lo que conlleva al surgimiento de nuevas variantes. Algunos de los métodos de solución para estos problemas son las heurísticas de construcción, que persiguen encontrar de manera rápida y sencilla una primera solución cercana al óptimo. Sin embargo, surge el inconveniente de que la gran mayoría de heurísticas sólo puedan ser empleadas en la variante básica. Además, en la actualidad existen pocas bibliotecas que implementen métodos heurísticos para solucionar estos problemas.

En este contexto surge la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP), con el propósito de dar solución a cuatro tipos de problemas de planificación de rutas de vehículos mediante el empleo de ocho heurísticas de construcción. En este trabajo se presenta una nueva versión de dicha biblioteca implementada en los lenguajes de programación Java y Python, que resuelve las deficiencias detectadas en la versión previa. Además, se incorporan dos nuevas heurísticas de construcción, así como tres nuevas variantes que consideran ventanas de tiempo, autobuses escolares y el retorno a un punto diferente al depósito. Por último, se realizan un conjunto de experimentos, donde se superan los resultados en cuanto a costo en distancia euclidiana obtenidos por la herramienta OR-Tools.

Palabras claves: BHCVRP, diseño de software, heurísticas de construcción, VRP.

Abstract

Vehicle Routing Problems involve a fleet of vehicles, seek to satisfy the demands of geographically dispersed customers and aim to minimize costs as long as existing constraints are met. New practical situations occur daily in the world, which leads to the increase of these problems and the emergence of new variants. Some of the solution methods are construction heuristics, which aim to quickly and easily find a first solution close to the optimum. However, there is the drawback that the vast majority of heuristics can only be used in the basic variant. In addition, there are currently few libraries that implement heuristic methods to solve these problems.

In this context, BHCVRP arises with the purpose of providing solutions to four types of vehicle route planning problems by employing seven construction heuristics. This paper presents a new version of this library implemented in Java and Python programming languages, which solves some of the deficiencies found in the previous version. Two new construction heuristics and three new variants are incorporated. Finally, a set of experiments are carried out, where the results in terms of cost in Euclidean distance obtained by the OR-Tools tool are outperformed.

Keywords: BHCVRP, software design, construction heuristics, VRP.

Índice

Introducción.....	1
Capítulo 1: Biblioteca de heurísticas de construcción para problemas de planificación de rutas de vehículos	8
1.1 Problema de planificación de rutas de vehículos	8
1.2 Algoritmos heurísticos.....	14
1.3 Bibliotecas que implementan métodos heurísticos	18
1.3.1 Bibliotecas especializadas en el cálculo de distancias	23
1.4 Especificaciones de BHCVRP	24
1.4.1 Diseño arquitectónico de BHCVRP	26
1.4.2 Patrones de diseño implementados en BHCVRP	32
1.4.3 Deficiencias en BHCVRP	36
1.5 Conclusiones parciales	37
Capítulo 2: Diseño de la nueva versión de BHCVRP	39
2.1 Modificaciones en la arquitectura de BHCVRP	39
2.1.1 Nuevas variantes de problemas de planificación de rutas de vehículos en BHCVRP.....	40
2.1.2 Nuevas heurísticas de construcción en BHCVRP	42
2.1.3 Otras modificaciones en BHCVRP	45
2.2 Nueva versión de BHCVRP	46
2.2.1 Paquetes y funcionalidades en común	48
2.2.2 Particularidades de BHCVRP en Java y Python.....	59
2.3 Principios de diseño	61
2.4 Patrones de diseño	64
2.4.1 Patrón <i>Factory Method</i>	64

2.4.2 Patrón <i>Template Method</i>	65
2.5 Conclusiones parciales	68
Capítulo 3: Análisis y validación de BHCVRP	70
3.1 Descripción de los experimentos	70
3.1.1 Descripción de las instancias.....	70
3.1.2 Configuración de los experimentos.....	72
3.2 Análisis de los resultados.....	76
3.2.1 Resultados obtenidos con BHCVRP en Python.....	76
3.2.2 Comparación de los resultados obtenidos por las diferentes versiones de BHCVRP	92
3.2.3 Comparación de los resultados con OR-Tools	94
3.3 Conclusiones parciales	97
Conclusiones.....	98
Recomendaciones.....	100
Referencias bibliográficas	101
Anexo 1	113
Anexo 2	117

Índice de tablas

Tabla 1: Comparación de bibliotecas de clases que implementan métodos heurísticos.	21
Tabla 2: Patrones de diseño GRASP implementados en BHCVRP.	35
Tabla 3: Descripción de las clases que integran el paquete <i>solution</i> .	48
Tabla 4: Descripción de las clases que integran el paquete <i>data</i> .	49
Tabla 5: Descripción de las clases que integran el paquete <i>postoptimization</i> .	51
Tabla 6: Descripción de las clases que integran el paquete <i>interfaces</i> .	52
Tabla 7: Descripción de las clases que integran el paquete <i>methods</i> .	53
Tabla 8: Descripción de las clases que integran el paquete <i>heuristic</i> .	54
Tabla 9: Descripción de las clases que integran el paquete <i>exceptions</i> .	56
Tabla 10: Descripción de las clases que integran el paquete <i>tools</i> .	58
Tabla 11: Descripción de las clases que integran el paquete <i>controller</i> .	58
Tabla 12: Particularidades de BHCVRP en Java y Python.	60
Tabla 13: Principios de diseño SOLID que cumple BHCVRP.	62
Tabla 14: Otros principios de diseño aplicados en BHCVRP.	63
Tabla 15: Resumen de los resultados en las ocho instancias CVRP (I).	78
Tabla 16: Resumen de los resultados en las ocho instancias CVRP (II).	78
Tabla 17: Resumen de los resultados en las ocho instancias HFVRP (I).	80
Tabla 18: Resumen de los resultados en las ocho instancias HFVRP (II).	80
Tabla 19: Resumen de los resultados en las ocho instancias MDVRP (I).	82
Tabla 20: Resumen de los resultados en las ocho instancias MDVRP (II).	82
Tabla 21: Resumen de los resultados en las ocho instancias TTRP (I).	84
Tabla 22: Resumen de los resultados en las ocho instancias TTRP (II).	84
Tabla 23: Resumen de los resultados en las ocho instancias OVRP (I).	86
Tabla 24: Resumen de los resultados en las ocho instancias OVRP (II).	86
Tabla 25: Resumen de los resultados en las ocho instancias VRPTW (I).	88
Tabla 26: Resumen de los resultados en las ocho instancias VRPTW (II).	88
Tabla 27: Resumen de los resultados en las ocho instancias SBRP (I).	90
Tabla 28: Resumen de los resultados en las ocho instancias SBRP (II).	90

Tabla 29: Rankings de las heurísticas para las siete variantes VRP en cuanto a calidad de soluciones en distancia euclidiana.	91
Tabla 30: Ranking general de las versiones de BHCVRP.....	93
Tabla 31: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias CVRP.	94
Tabla 32: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias HFVRP.	95
Tabla 33: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias VRPTW.	96
Tabla 34: Descripción de las instancias CVRP y OVRP.	113
Tabla 35: Descripción de las instancias MDVRP.	113
Tabla 36: Descripción de las instancias HFVRP.	114
Tabla 37: Descripción de las instancias TTRP.	115
Tabla 38: Descripción de las instancias VRPTW.	115
Tabla 39: Descripción de las instancias SBRP.	116
Tabla 40: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante CVRP en Python.	117
Tabla 41: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante HFVRP en Python.	117
Tabla 42: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante MDVRP en Python.	117
Tabla 43: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante TTRP en Python.	118
Tabla 44: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante SBRP en Python.	118
Tabla 45: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante VRPTW en Python.	118
Tabla 46: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para la variante OVRP en Python.	118
Tabla 47: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias CVRP (I).	119

Tabla 48: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias CVRP (II).....	119
Tabla 49: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias HFVRP (I).	120
Tabla 50: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias HFVRP (II).	120
Tabla 51: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias MDVRP (I).....	121
Tabla 52: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias MDVRP (II).....	121
Tabla 53: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias TTRP (I).	122
Tabla 54: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias TTRP (II).	122
Tabla 55: Rankings de las heurísticas para la variante CVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.	123
Tabla 56: Rankings de las heurísticas para la variante HFVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.	123
Tabla 57: Rankings de las heurísticas para la variante MDVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.	124
Tabla 58: Rankings de las heurísticas para la variante TTRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.	124
Tabla 59: Resultados de los métodos <i>post-hoc</i> Li, Finner y Holm para el <i>ranking</i> general de Friedman de las diferentes versiones de BHCVRP.....	124

Índice de figuras

Figura 1: Clases de complejidad en problemas de optimización [38].	9
Figura 2: Representación gráfica de un VRP [38].	10
Figura 3: Clasificación de los algoritmos aproximados [23].	15
Figura 4: Diagrama de actividades del funcionamiento de BHCVRP.	25
Figura 5: Patrón n-capas con enfoque basado en reutilización de BHCVRP [38].	26
Figura 6: Diagrama de clases del paquete <i>distances</i> [38].	27
Figura 7: Diagrama de clases del paquete <i>assign</i> [38].	28
Figura 8: Diagrama de clases del paquete <i>factory</i> .	28
Figura 9: Diagrama de clases del paquete <i>postoptimization</i> .	29
Figura 10: Diagrama de clases del paquete <i>heuristic</i> .	29
Figura 11: Diagrama de clases del paquete <i>solution</i> .	30
Figura 12: Diagrama de clases del paquete <i>data</i> .	31
Figura 13: Diagrama de clases del paquete <i>controller</i> .	32
Figura 14: Patrón n-capas con enfoque basado en reutilización de la nueva versión de BHCVRP.	47
Figura 15: Diagrama de clases del paquete <i>data</i> en la nueva versión de BHCVRP.	50
Figura 16: Diagrama de clases del paquete <i>postoptimization</i> en la nueva versión de BHCVRP.	51
Figura 17: Diagrama de clases del paquete <i>factory</i> en la nueva versión de BHCVRP.	52
Figura 18: Diagrama de clases del paquete <i>heuristic</i> en la nueva versión de BHCVRP.	53
Figura 19: Diagrama de clases del paquete <i>exceptions</i> en la nueva versión de BHCVRP.	55
Figura 20: Diagrama de clases del paquete <i>tools</i> en la nueva versión de BHCVRP.	57
Figura 21: Diagrama de secuencia del consumo de BHAVRP.	60
Figura 22: Diagrama de secuencia del patrón <i>Factory Method</i> para las heurísticas de construcción y los operadores de post-optimización.	65
Figura 23: Diagrama genérico común para todas las heurísticas.	66
Figura 24: Diagrama de actividades de la Heurística de Inserción de Kilby.	67

Introducción

Un problema de optimización combinatoria tiene como objetivo encontrar la mejor solución entre un conjunto finito de posibilidades. Cada solución está formada por una combinación de elementos de un conjunto dado. Estos problemas suelen involucrar la selección de un subconjunto de elementos que maximice o minimice una función objetivo, sujeto a ciertas restricciones [1]. En esta categoría se encuentra un problema bastante frecuente en los sistemas logísticos, que es la distribución de bienes y servicios desde ciertos depósitos a usuarios dispersos geográficamente. En la mayoría de las ocasiones se desea mejorar la planificación de las rutas de los vehículos, que determina cómo visitar a los clientes para satisfacer sus demandas [2]. En este contexto, surge el Problema de Planificación de Rutas de Vehículos (VRP, por las siglas en inglés de *Vehicle Routing Problem*) [3].

De manera general, un VRP busca diseñar un conjunto de rutas para que una flota de vehículos preste servicio a un conjunto de clientes, sujeto a una serie de restricciones. Este problema es empleado en la gestión de la cadena de suministro para la entrega física de bienes y servicios. Existen diversas variantes de VRP y se formulan en función de la naturaleza de los bienes transportados, la calidad del servicio requerido, las características de los clientes y los vehículos, entre otros [4]. Según [5], los problemas de planificación de rutas de vehículos se introducen en 1959 y se describe como una variante generalizada del Problema del Viajante de Comercio (TSP, por las siglas de *Traveling Salesman Problem*) [6]. En 1964, se propone el primer algoritmo efectivo para resolver VRP, denominado Algoritmo de Ahorros [7]. A partir de estos sucesos, incrementan las investigaciones y trabajos en el área de la planificación de rutas de vehículos. Por consiguiente, surgen dos líneas: definición de modelos que incorporen cada vez más características de la vida real y búsqueda de algoritmos para resolver eficientemente estos problemas [8].

Para resolver situaciones prácticas se proponen modelos básicos que pueden ser extendidos a diferentes campos de la logística y el transporte [9, 10]. Algunos ejemplos de aplicaciones prácticas de VRP se encuentran en el transporte obrero, las rutas de ómnibus, la recogida de desechos sólidos, la distribución del periódico y correo, entre

otros. Estas situaciones cotidianas pueden ser modeladas con sus características, a través de las diferentes variantes de VRP existentes. Dentro de las variantes más conocidas se encuentran: el Problema de Planificación de Rutas de Vehículos con Capacidades (CVRP) [11], el Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (MDVRP) [12], el Problema de Planificación de Rutas de Vehículos con Flota Heterogénea (HFVRP) [13], el Problema de Planificación de Rutas de Camiones y Remolques (TTRP) [14, 15], el Problema de Planificación de Rutas de Autobuses Escolares (SBRP) [16], el Problema de Planificación de Rutas de Vehículos con Ventanas de Tiempo (VRPTW) [17], entre otros [18-21].

Los problemas de planificación de rutas de vehículos son mayormente complejos de resolver para instancias que contengan un número de clientes elevado. Este fenómeno se debe al esfuerzo computacional requerido y a que los VRP pertenecen a la clase de problemas NP-duro [9]. Los problemas en esta categoría son considerados muy difíciles de resolver [22]. Para desempeñar esta tarea, en ocasiones se necesita comprometer requisitos como la optimalidad y construir una buena solución, a pesar de no garantizar el óptimo. Para eso, se aplican los algoritmos heurísticos, específicamente las heurísticas y metaheurísticas, como alternativa de solución [23].

Particularmente, el propósito de las heurísticas es guiar el proceso de búsqueda en la mejor dirección y sugerir qué camino tomar cuando existen varias opciones disponibles [24, 25]. Esto significa que constituyen métodos más sencillos que brindan soluciones satisfactorias a un problema dado mediante algoritmos específicos. Además, pueden ofrecer la solución final del problema o ser aplicadas dentro del contexto de las metaheurísticas [8, 25].

Por su parte, las heurísticas se clasifican en dos grupos: heurísticas de construcción y heurísticas de mejora [26, 27]. Las heurísticas de construcción se encargan de construir literalmente una solución del problema, paso por paso [26]. Sin embargo, las heurísticas de mejora parten de una solución y reparan las rutas previamente construidas, con el objetivo de conseguir una determinada mejoría [25]. Dentro de las heurísticas de construcción clásicas de la literatura, utilizadas para la resolución de los VRP, se encuentran: el Algoritmo de Ahorros [5], el Algoritmo de Barrido [28], la Heurística de

Inserción en Paralelo de Christofides, Mingozzi y Toth [29], Heurística de Inserción Secuencial de Mole & Jameson [30], entre otras [19, 24, 31, 32].

Debido a la necesidad de resolver problemas de optimización combinatoria utilizando dichos algoritmos y favorecer la reutilización de sus códigos, es apropiado agruparlos dentro de componentes de software. Actualmente, existen pocas bibliotecas que implementan métodos heurísticos para resolver, ya sea VRP u otros problemas de optimización. Algunos de los componentes de software que cumplen con dichas características son:

- BiCIAM: biblioteca de clases en Java, que implementa un modelo unificado de metaheurísticas mono-objetivo y multi-objetivo [33, 34].
- METSlib: biblioteca que implementa metaheurísticas básicas de la literatura [35].
- VRPH: biblioteca de heurísticas de búsqueda local para VRP en C++ [36].
- MOMHLib++: biblioteca diseñada en C++, para la implementación de metaheurísticas multi-objetivos [37].
- BHCVRP: biblioteca de heurísticas de construcción para VRP en Java [38, 39].
- OR-Tools: software de código abierto desarrollado principalmente en Python, para resolver problemas de optimización combinatoria, incluyendo VRP en sus distintas variantes como CVRP, HFVRP y VRPTW [40].
- JSprit: herramienta de código abierto desarrollada en Java, que implementa el principio *Ruin and Recreate* [41] y resuelve determinadas variantes VRP [42].
- VROOM: software de código abierto desarrollado en C++, para resolver VRP utilizando heurísticas para obtener la solución inicial, en dependencia de la variante del problema [43].

De los componentes de software antes mencionados, los que permiten resolver VRP son las bibliotecas BiCIAM [33, 34], VRPH [36] y BHCVRP [38, 39], y las herramientas OR-Tools [40], JSprit [42] y VROOM [43]. De este conjunto, BHCVRP es la única que permite hacerlo a través de heurísticas de construcción exclusivamente. Generalmente, en la

literatura se evidencia la carencia de bibliotecas de heurísticas para la resolución de problemas de optimización combinatoria, debido a que la mayor parte del desarrollo se centra en los algoritmos metaheurísticos. Además, existen pocas adaptaciones de las heurísticas a las distintas variantes VRP, ya que no se consideran las características propias de cada uno de estos problemas. Por último, la mayoría de los componentes de software existentes para solucionar estos problemas están desarrollados para C++ y Java. Debido al crecimiento de la comunidad en Python para trabajos con Inteligencia Artificial y problemas de optimización, es importante también solucionar VRPs con heurísticas desde este lenguaje [44, 45]. Sin embargo, también resulta interesante analizar aspectos como el cálculo de distancias. La mayoría de estas herramientas emplea distancias aproximadas como *Euclidean* [46], *Manhattan* [47], *Haversine* [48] y *Chebyshev* [49], por lo que es recomendable profundizar en la inclusión de herramientas que modelen distancias reales como *Google Distance Matrix API* [50], *GraphHopper* [51] o *Mapbox Directions API* [52], para una mayor precisión. Otra alternativa es utilizar servicios como *Open Street Map* (OSM) [53] y *Open Source Routing Machine* (OSRM) [54] para el trabajo con mapas. Esta carencia puede dificultar el proceso de encontrar la ruta ideal, dado que las métricas de distancia aproximadas no consideran factores como la topografía o las restricciones viales.

En este contexto surge la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP) como propuesta realizada por el proyecto de Optimización y Metaheurísticas de la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE) [38, 39]. Esta biblioteca está desarrollada en Java y contiene la adaptación de ocho heurísticas de construcción de la literatura para las variantes CVRP, MDVRP, HFVRP y TTRP. A pesar de constituir un aporte, presenta deficiencias en cuanto a diseño y adaptabilidad. Su integración está restringida a aplicaciones desarrolladas en Java, limitando su interoperabilidad con otros sistemas. Además, la biblioteca presenta alto acoplamiento entre sus clases, lo que dificulta la extensión y el mantenimiento del código relacionado con las heurísticas implementadas. Este componente carece de un manejo robusto de excepciones, así como de soporte para la carga y exportación de datos en formatos estándares como JSON o CSV.

A partir de la situación descrita anteriormente se plantea como problema a resolver la existencia de deficiencias en la arquitectura de BHCVRP, teniendo en cuenta la manera en que se encuentran implementadas las heurísticas para resolver cada variante VRP.

En este trabajo, el objeto de estudio se enmarca en los problemas de optimización combinatoria, los algoritmos heurísticos y el diseño de software. Específicamente, el campo de acción se enfoca en los Problemas de Planificación de Rutas de Vehículos, las heurísticas de construcción y los patrones de diseño. Para dar solución al problema planteado se identifica como objetivo general: Desarrollar una nueva versión de BHCVRP en Java y Python, que permita incorporar otras heurísticas de construcción de la literatura para diferentes variantes de problemas de planificación de rutas de vehículos.

Para dar cumplimiento al objetivo general, se definen los siguientes objetivos específicos con sus tareas asociadas:

1. Caracterizar el marco teórico investigativo sobre los problemas de planificación de rutas de vehículos, el uso de los algoritmos heurísticos para resolverlos y los componentes de software que agrupan estos algoritmos para su reutilización.
 - 1.1. Realizar búsquedas bibliográficas para identificar las características que presentan los problemas de planificación de rutas de vehículos.
 - 1.2. Realizar búsquedas bibliográficas para realizar un estudio de la literatura referente a los resultados obtenidos por algoritmos heurísticos en distintas variantes VRP.
 - 1.3. Realizar un análisis de los componentes de software existentes y las tecnologías, para la resolución de variantes VRP haciendo uso de algoritmos heurísticos y cálculo de distancias tanto aproximadas como reales.
2. Rediseñar la arquitectura de la biblioteca de clases para la utilización de otras heurísticas de construcción en diferentes variantes VRP.
 - 2.1. Analizar la versión de BHCVRP en Java propuesta anteriormente para determinar los cambios a realizar.

- 2.2. Diseñar las modificaciones necesarias en la arquitectura para permitir la incorporación de otras variantes VRP y heurísticas de construcción.
- 2.3. Incorporar en ambas versiones de la biblioteca, Java y Python, el cálculo de distancias reales mediante el uso de una biblioteca o herramienta especializada.
3. Implementar adaptaciones de otras heurísticas de construcción para los problemas de planificación de rutas de vehículos.
 - 3.1. Analizar nuevas heurísticas de construcción y variantes VRP para su incorporación en la nueva versión de la biblioteca.
 - 3.2. Seleccionar las heurísticas de construcción y variantes VRP a incorporar a la biblioteca.
 - 3.3. Implementar en Python y Java, las clases y métodos necesarios para adaptar las heurísticas seleccionadas teniendo en cuenta las nuevas características que presenten las variantes VRP a incorporar.
4. Validar el funcionamiento de las heurísticas adaptadas a través de un conjunto de experimentos con instancias de la literatura.
 - 4.1. Definir los escenarios para cada uno de los experimentos con las heurísticas adaptadas.
 - 4.2. Analizar los resultados obtenidos con cada experimento utilizando pruebas estadísticas no paramétricas.
 - 4.3. Comparar los resultados obtenidos para las variantes CVRP, MDVRP, HFVRP y TTRP con los resultados obtenidos en la versión anterior de BHCVRP, verificando que funcionen igual.
 - 4.4. Comparar los resultados actuales obtenidos con los resultados de alguna herramienta relevante en Python, identificando cuál es mejor para cada caso.

El cumplimiento del objetivo general tiene como valor práctico un aporte significativo en la resolución de diferentes VRP a través de la nueva versión de la biblioteca desarrollada en Java y Python. Se propone como artefacto de salida un diseño de software, según la

clasificación presentada en [55]. Es importante destacar que los resultados obtenidos ponen a disposición de la comunidad científica un componente de software que brinda nuevos algoritmos para la resolución de distintas variantes de VRP. Además, la biblioteca permite la colaboración con otros componentes que emplean metaheurísticas o sistemas de información geográficos con servicios de planificación de rutas de vehículos. Por último, mencionar que resultados parciales de este trabajo han sido presentados en distintos eventos como la XXI Convención Científica de Ingeniería y Arquitectura [56], y jornadas científicas [57, 58], obteniendo premios.

Para alcanzar los objetivos mencionados el trabajo de diploma se estructura en tres capítulos, con secciones importantes, tales como conclusiones, recomendaciones y referencias bibliográficas. El Capítulo 1: Biblioteca de heurísticas de construcción para problemas de planificación de rutas de vehículos, presenta una revisión detallada del estado del arte sobre VRP, sus características y variantes. Por otra parte, abarca los algoritmos heurísticos, describe algunas de las heurísticas de construcción más utilizadas en estos problemas y presenta los componentes de software relacionados con estas temáticas, así como el cálculo de distancias aproximadas y reales. Además, se muestra el diseño arquitectónico, principios de diseño y deficiencias existentes en BHCVRP. El Capítulo 2: Diseño de la nueva versión de BHCVRP, describe la solución mediante la vista de la arquitectura de la biblioteca de clases y los patrones implementados. Además, incluye elementos como la adaptación realizada de los nuevos algoritmos y variantes VRP incorporadas. El Capítulo 3: Análisis y validación de BHCVRP, se encarga de demostrar la validación de la solución mediante un conjunto de experimentos. Además, se presentan los resultados alcanzados y un análisis del comportamiento de dichas heurísticas aplicando pruebas estadísticas no paramétricas. Por último, se realizan dos comparaciones: una para garantizar igualdad en las implementaciones tanto en Java como en Python, y otra con una herramienta destacada en la resolución de VRPs en Python, para identificar cuál funciona mejor en las diferentes variantes.

Capítulo 1: Biblioteca de heurísticas de construcción para problemas de planificación de rutas de vehículos

En este capítulo se describe el marco teórico que aborda los problemas de planificación de rutas de vehículos, sus variantes y heurísticas de construcción documentadas en la literatura como alternativa de solución. Se comparan componentes de software que implementan métodos heurísticos para la resolución de problemas de optimización y problemas de planificación de rutas de vehículos. Asimismo, se presentan las bibliotecas especializadas en el cálculo de distancias, evaluando su posible aplicación en el contexto de la optimización de rutas.

Por otra parte, se realiza un análisis detallado de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP), componente de software principal de esta investigación. Para ello se tienen en cuenta las especificaciones de la biblioteca, describiendo su propósito, elementos principales y heurísticas empleadas. Además, se tienen secciones para demostrar su diseño arquitectónico y patrones de diseño implementados. Por último, se identifican las deficiencias actuales de BHCVRP y se destacan oportunidades de mejoras en cuanto a diseño y funcionalidad.

1.1 Problema de planificación de rutas de vehículos

En el mundo actual, con el aumento de su complejidad y competitividad, se requiere cada vez más del proceso de toma de decisiones en disímiles actividades. Por ende, la optimización adquiere mayor importancia en las aplicaciones de la vida real donde se necesita seleccionar la mejor opción. La optimización consiste en hallar el valor máximo o mínimo de una o varias funciones objetivo, donde se cumplen determinadas restricciones que condicionan la elección de la solución adecuada y el propósito es alcanzar costos mínimos o máximos beneficios [17].

Dentro del campo de la optimización se encuentran los problemas lineales, considerados los más sencillos de resolver debido a la linealidad de su función objetivo y restricciones. Aunque, la mayor parte de los problemas de optimización están catalogados como difíciles de resolver [26]. A continuación, en la Figura 1, se presenta la relación existente

entre las diferentes clases utilizadas para definir distintos problemas de optimización según su complejidad [23]:

- P: problemas que pueden ser resueltos por algoritmos que demoran un tiempo polinomial en resolverlos.
- NP: problemas que pudieran ser resueltos por un algoritmo no determinístico en un tiempo polinomial.
- NP-completo: problemas que actualmente no cuentan con un algoritmo capaz de obtener el estado óptimo en un tiempo polinomial, pero que tampoco se ha probado que no pueda existir dicho algoritmo, para al menos un problema.
- NP-duro: problemas NP-completo que no han podido ser reducidos a problemas NP. Son considerados intratables o muy difíciles de resolver.

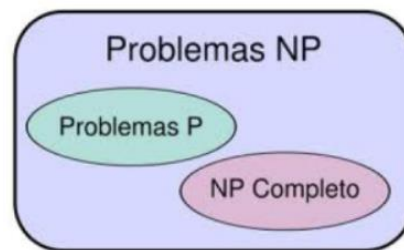


Figura 1: Clases de complejidad en problemas de optimización [38].

Teniendo en cuenta lo mencionado anteriormente, aparece la optimización combinatoria enmarcada dentro de la rama denominada optimización en matemática aplicada y en ciencias de la computación [26]. Un problema de optimización combinatoria puede representarse mediante un modelo matemático con ecuaciones y expresiones que describen el problema en cuestión. Dichos modelos están formados por:

- Restricciones: imponen ciertos límites a los valores posibles, es decir, los coeficientes son entradas del modelo, donde su lado derecho es un valor numérico que representa una cota inferior o superior [59].
- Variables: constituyen las salidas del modelo y generalmente poseen un valor numérico, aunque existen otras maneras de codificar algunos tipos de información según el contexto [59].

- Función objetivo: determina la efectividad para cada solución. Según la cantidad de objetivos a tratar de resolver, los problemas de optimización se clasifican en mono-objetivos o multi-objetivos. El propósito de los mono-objetivos es obtener una solución óptima para un objetivo determinado; puede ser máximo o mínimo. Sin embargo, los multi-objetivos obtienen un conjunto de soluciones conocidas como Frente de Pareto [60].

El Problema del Viajante de Comercio (TSP, por las siglas en inglés de *Travelling Salesman Problem*) [61] constituye el punto de partida para la formulación de otros problemas de optimización combinatoria más complejos que se utilizan en situaciones prácticas, como es el caso de los problemas de planificación de rutas de vehículos. El TSP se basa en un conjunto de n ciudades y el costo C_{ij} que se obtiene al viajar de un punto a otro. Posee como objetivo fundamental encontrar la ruta de costo mínimo para visitar todas las ciudades pasando solo una vez por cada una de ellas, y regresando al punto de partida [62]. Por tanto, el TSP es considerado el problema de planificación de rutas de vehículos más simple, a pesar de pertenecer a la clase de problemas NP-duro y ser uno de los problemas de optimización combinatoria más difundidos [31, 63].

Los Problemas de Planificación de Rutas de Vehículos (VRP, por las siglas en inglés de *Vehicle Routing Problem*) modelan el fenómeno de la distribución de bienes y servicios. De manera general, un VRP consiste en determinar el conjunto de rutas de costo mínimo que cumplan con las restricciones del problema, dado una cantidad de clientes dispersos geográficamente y una flota de vehículos perteneciente a uno o varios depósitos [31]. A continuación, en la Figura 2 se muestra una visualización de este problema:

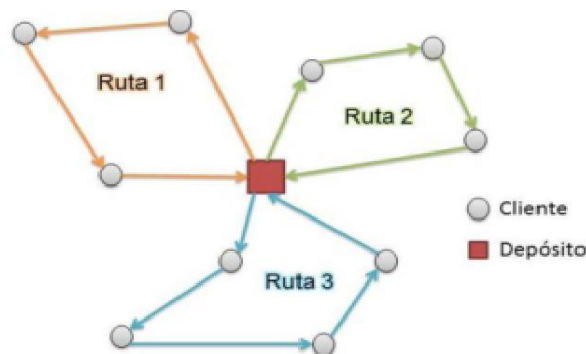


Figura 2: Representación gráfica de un VRP [38].

Los vehículos deben ser capaces de satisfacer cierta demanda por parte de los clientes. En la mayoría de los casos, la demanda constituye un bien que ocupa lugar en los vehículos y lo común es que no se satisfaga la demanda de todos los clientes en una misma ruta. Por otra parte, existen otras situaciones, donde la demanda es un servicio, es decir, el cliente espera ser visitado por el vehículo. En ocasiones, se tienen en cuenta otras características como es el caso de la cantidad de depósitos, el tipo de flota, horarios de servicio, entre otros [9].

En la actualidad se realiza un constante esfuerzo para resolver los problemas de planificación de rutas de vehículos [64, 65]. Según [5], en 1959, se realiza por primera vez una formulación de dicho problema para una aplicación de distribución de combustible. En 1964, [7] propone el primer algoritmo efectivo para su resolución: el conocido Algoritmo de Ahorros. Por tanto, la esfera de la planificación de rutas de vehículos ha crecido en gran medida, tomando dos perspectivas fundamentales. Se trabaja hacia la definición de modelos que incorporen características cada vez más cercanas a la realidad y hacia la búsqueda de algoritmos que permitan resolver estos problemas de manera eficiente [31].

Los modelos básicos creados para los VRP son utilizados para resolver situaciones prácticas en el campo de la logística y el transporte [9, 66]. Dentro de las aplicaciones prácticas del VRP se encuentran: el transporte obrero, las rutas de ómnibus, la recolección de desechos sólidos, la distribución del periódico y correo, entre otros. Dichas situaciones cotidianas, pueden ser modeladas a través de diferentes variantes VRP existentes o la fusión de estas, respetando sus características. A continuación, se muestran algunas de las variantes más reconocidas en la literatura:

- **Problema de Planificación de Rutas de Vehículos con Capacidades** (CVRP, por las siglas en inglés de *Capacitated VRP*): constituye la variante clásica del problema de planificación de rutas de vehículos, incluyendo restricciones relacionadas con la capacidad de los vehículos. Además, se establece que cada cliente posee una demanda específica para ser atendida y la demanda total de los clientes en cada ruta no puede exceder la capacidad del vehículo empleado en dicha ruta. Por tanto, es un problema que presenta un conjunto de vehículos

idénticos, ubicados en un depósito central y que satisface la demanda de los clientes, sujeto a restricciones de capacidad de sus vehículos [24, 67].

- **Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos** (MDVRP, por las siglas en inglés de *Multi-Depot VRP*): es una extensión del clásico CVRP que incorpora varios depósitos con ubicaciones diferentes. En este problema cada vehículo debe comenzar y finalizar su recorrido en el mismo depósito. Además, cada depósito cuenta con una flota limitada de vehículos con capacidad restringida y homogénea, utilizada para satisfacer las demandas de los clientes, cuyos datos son conocidos de antemano. Según [68], si los clientes se agrupan cerca de los depósitos, el problema se puede modelar como CVRP independientes. Por otra parte, si los lugares donde están los clientes y depósitos se encuentran mezclados, el problema debe ser resuelto en dos fases: asignación de clientes a depósitos y determinación de recorridos a realizar por cada depósito para visitar a los clientes asignados [69].
- **Problema de Planificación de Rutas de Vehículos con Flota Heterogénea** (HFVRP, por las siglas en inglés de *Heterogenous Fleet VRP*): esta variante presenta características y/o capacidades diferentes en los vehículos de la flota. La capacidad heterogénea es considerada para determinar las rutas a construir, debido a que un vehículo más grande puede realizar una ruta más larga o de mayor concentración de demanda. La cantidad de vehículos de cada tipo es limitada y existe un costo fijo o costo por unidad de distancia recorrida asociada a cada tipo de vehículo [70].
- **Problema de Planificación de Rutas de Camiones y Remolques** (TTRP, por las siglas en inglés de *Truck and Trailer Routing Problem*): este problema considera el empleo de remolques como parte de la flota de vehículos. Además, presenta ciertas particularidades como la definición de tipos de cliente debido a las restricciones de accesos existentes en las ubicaciones de los mismos. Existen clientes accedidos por camión y remolque, conocidos como clientes de vehículo completo (VC) y otros accedidos solamente por camión sin el remolque denominados clientes de camión puro (TC). Para este tipo de problema surgen

diferentes soluciones a partir del diseño de tres tipos de rutas según los clientes a visitar y el vehículo utilizado (PTR, PVR o CVR) [71].

- **Problema de Planificación de Rutas de Autobuses Escolares** (SBRP, por las siglas en inglés de *School Bus Routing Problem*): se centra en el uso eficiente de una flota de vehículos para transportar a los estudiantes desde sus casas hasta la escuela, utilizando una cantidad determinada de autobuses. De manera general, trata de encontrar el programa más eficiente para una flota de autobuses escolares. Cada vehículo debe recoger a los estudiantes asignados en las paradas, cumpliendo con determinadas restricciones; por ejemplo, tiempo máximo de recorrido, capacidad máxima del autobús, ventanas de tiempo para la llegada a la escuela, entre otros. Este problema se puede clasificar según los siguientes criterios: cantidad de escuelas (una o múltiples), entorno del servicio (urbano o rural) y flotas mixtas (homogénea o heterogénea) [16].
- **Problema de Planificación de Rutas de Vehículos Abiertos** (OVRP, por las siglas en inglés de *Open VRP*): su objetivo es diseñar el conjunto de rutas de coste mínimo con origen en un depósito central para satisfacer la demanda de los clientes, por lo que se trata similar a la variante CVRP. Sin embargo, su característica fundamental es que los vehículos no necesitan volver al depósito tras completar sus servicios de entrega [72, 73].
- **Problema de Planificación de Rutas de Vehículos con Ventanas de Tiempo** (VRPTW, por las siglas en inglés de *VRP with Time Windows*): es considerada una extensión del CVRP, ya que considera las capacidades e incluye como restricción que cada cliente tiene asociada una ventana de tiempo. Esta restricción representa un horario de servicio en que los vehículos pueden arribar a los clientes y se define un tiempo de servicio. El objetivo de este problema radica en minimizar la distancia total de los repartos de manera que se cumpla con la demanda de todos los clientes posibles respetando sus respectivas ventanas de tiempo [66].

A partir del surgimiento de nuevas características en los problemas de la vida real, aparecen nuevas variantes, por ejemplo: el Problema de Planificación de Rutas de Vehículos Dinámico (DVRP, *Dynamic Vehicle Routing Problem*) [74], el Problema de

Planificación de Rutas de Vehículos con Entregas Divididas (SDVRP, *Split Delivery Vehicle Routing Problem*) [75], el Problema de Planificación de Rutas de Vehículos Ecológicos (GVRP, *Green Vehicle Routing Problem*) [18], el Problema de Planificación de Rutas de Vehículos Eléctricos (EVRP, *Electric Vehicle Routing Problem*) [21], entre otros [76-79]. Además, en ocasiones, se requiere la fusión de las variantes mencionadas anteriormente para lograr solucionar una situación del mundo real.

Los problemas de planificación de rutas de vehículos, generalmente, son complejos de resolver para instancias con un número de clientes elevado, debido al costo computacional requerido y a que pertenecen a la categoría de los problemas NP-duro. Con el objetivo de solucionar estos problemas, a veces se comprometen requisitos como la optimalidad y se construye una estructura de control para encontrar una buena solución, aunque no garantice la respuesta óptima. Por consiguiente, es común emplear algoritmos heurísticos en este tipo de problemas [23]. En el siguiente epígrafe se describen algunas de las alternativas más abarcadas en la literatura.

1.2 Algoritmos heurísticos

Para resolver los problemas de optimización combinatoria surgen diversas técnicas, que se clasifican en dos categorías fundamentales: exactas y aproximadas. Los métodos exactos garantizan encontrar la solución óptima, aunque con el aumento del tamaño de las instancias se afecta de manera significativa el tiempo de ejecución [80].

Los métodos aproximados surgen a partir de la inminente necesidad de resolver problemas de mayor complejidad. Estos obtienen buenas soluciones, aunque no necesariamente la óptima. Además, la rapidez del proceso adquiere igual relevancia que la calidad de la solución obtenida [23]. A continuación, en la Figura 3, se muestran sus clasificaciones según [23].

Los métodos heurísticos no siempre poseen una base matemática formal, pues pueden ser desarrollados por intuición. Las heurísticas están diseñadas para resolver un problema y/o instancia específica, mientras que las metaheurísticas se consideran algoritmos de propósito general utilizados para solucionar cualquier problema de optimización [23]. A continuación, se describen los aspectos relacionados con las

heurísticas y específicamente las heurísticas de construcción, ya que constituyen el centro de esta investigación.

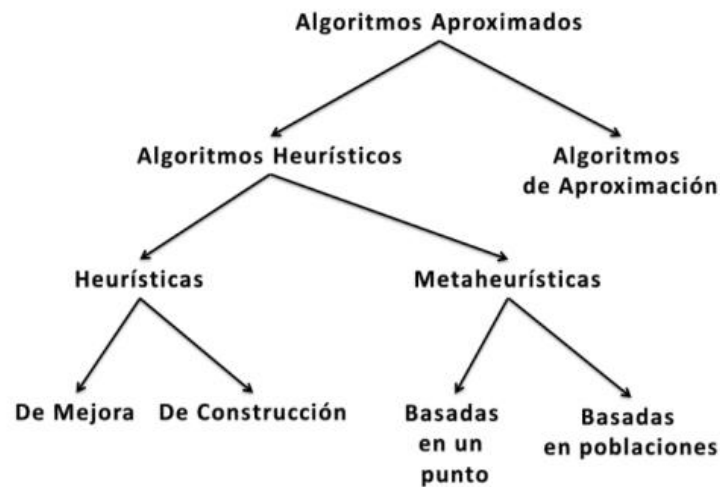


Figura 3: Clasificación de los algoritmos aproximados [23].

La palabra heurística es un término derivado de *heuriskein*, palabra griega que significa encontrar o descubrir. Este vocablo es empleado en el campo de la optimización para describir una clase de algoritmos de resolución de problemas [26]. Específicamente, el término heurística se refiere al procedimiento que busca aportar soluciones a un problema con un buen rendimiento, en cuanto a calidad de soluciones y cantidad de recursos utilizados. Por tanto, se consideran métodos sencillos para obtener soluciones satisfactorias a un problema determinado con algoritmos específicos [8]. De manera general, este término se relaciona con la tarea de resolver con cierta inteligencia problemas reales a partir de conocimiento disponible [81]. Según [82], una heurística es un método que busca soluciones cercanas al óptimo, aunque no garantice encontrarlo, con un costo computacional razonable.

Estos algoritmos, según su naturaleza son enmarcados en dos categorías generales: heurísticas de construcción y heurísticas de mejora. Las heurísticas de construcción consisten en construir paso a paso una solución del problema. En algunos casos son deterministas y suelen basarse en la mejor elección por cada iteración [26]. Sin embargo, las heurísticas de mejora parten de una solución y realizan intercambios de clientes en

las rutas para mejorar las soluciones. Esto implica reparar las rutas previamente construidas para obtener una determinada mejora [74].

Por otra parte, las heurísticas de construcción (HC) para problemas de planificación de rutas de vehículos se dividen en cuatro grupos según [83]:

- Basadas en ahorros (*Saving Based*): mediante ahorros producidos por la sustitución de nodos en la ruta por otros que aún no están, se mantiene un proceso de constante intercambio mientras se cumplan las restricciones. Dentro de los algoritmos más conocidos se encuentran el Algoritmo de Ahorros (*Save Algorithm*) [7] con su versión secuencial y paralela, así como la variante basada en *Matching* [31].
- De inserción o basadas en coste (*Cost Based*): son empleadas para generar un conjunto de rutas iniciales. Se construye una solución mediante constantes inserciones de nodos [30]. Dentro de las heurísticas más empleadas se encuentran Vecino más cercano [31], Inserción Secuencial de Mole & Jameson [30] e Inserción de Kilby [74].
- Ruta primero – Asignar después (*Route first – Cluster second*): se determina el recorrido para visitar a todos los clientes y luego se particiona en varias rutas para que cada una sea factible, garantizando de esta manera el cumplimiento de las restricciones [31]. Dentro de los algoritmos que poseen este comportamiento se encuentran los Algoritmos de Pétalos [32].
- Asignar primero – Ruta después (*Cluster first – Route second*): se busca generar un grupo de clientes que pertenezcan a una misma ruta en la solución final y luego se crea una ruta para cada grupo que visite a todos sus clientes determinando el orden en que serán visitados [31]. Dentro de los algoritmos más conocidos se encuentra el Algoritmo de Barrido [28, 84] y la Heurística de Asignación Generalizada de Fisher y Jaikumar [24].

La mayoría de las heurísticas de construcción están diseñadas para el VRP más sencillo, es decir, el problema de planificación de rutas de vehículos con capacidades (CVRP). Por tanto, deben realizarse adaptaciones a estos métodos constructivos para aplicarlos

en otras variantes. A continuación, se describen algunos de los algoritmos más significativos dentro de esta categoría en la literatura de los problemas de planificación de rutas de vehículos:

- Algoritmo de Ahorros (*Save Algorithm*): constituye el primer algoritmo efectivo para la resolución de un VRP. Este algoritmo plantea que en una solución dos rutas diferentes pueden ser combinadas formando una nueva ruta. Se parte de una solución inicial y se realizan las uniones que den mayores ahorros siempre que se cumplan las restricciones del problema. Existen dos versiones, la paralela que trabaja sobre todas las rutas de manera simultánea y la secuencial que construye las rutas una a una [7].
- Algoritmo de Barrido (*Sweep*): heurística basada en *Cluster first – Route second*, donde primero se forman grupos de clientes y luego se construyen las rutas para cada grupo. En este algoritmo los grupos se forman girando una semirrecta con origen en el depósito e incorporando clientes barridos por dicha semirrecta hasta que se viole la restricción de capacidad. Luego, los clientes se ordenan resolviendo un TSP en cada grupo. Esta heurística generalmente emplea coordenadas polares y distancia euclidiana [28].
- Heurística del Vecino más cercano con lista de candidatos restringidos (*Nearest Neighbor with RLC*): método determinista, pues se basa en la mejor elección en cada iteración. Esta variante incluye una lista de k vecinos más cercanos al cliente en cuestión. Este algoritmo consta de tres procesos: inicialización, inserción y finalización [31, 38].
- Heurística de Inserción Secuencial de Mole & Jameson: heurística de inserción que crea una solución mediante sucesivas inserciones de clientes en las rutas. En cada iteración se obtiene una solución parcial, donde las rutas visitan solamente un subconjunto de clientes y selecciona un cliente no visitado para insertar en dicha solución. Para ello, se siguen los pasos de creación de una ruta, inserción y optimización [30].

Además de las heurísticas de construcción antes mencionadas existen otras relevantes como: Algoritmo de Ahorros basado en *Matching* [85], la Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (CMT) [86], la Heurística de Inserción de Kilby [74], la Heurística de Localización de Bramel y Simchi-Levi [87], la Heurística de Asignación Generalizada de Fisher y Jaikumar [24], entre otras [31]. En muchos casos también se utiliza regularmente un algoritmo de construcción aleatoria. Este algoritmo consiste en seleccionar en cada iteración un elemento al azar dependiendo de las características del problema; por ejemplo, lista de clientes, depósitos, entre otros [31].

1.3 Bibliotecas que implementan métodos heurísticos

Para solucionar problemas de optimización resulta importante el uso de una biblioteca que implemente métodos heurísticos que garanticen la reutilización de los mismos. La mayor parte de las investigaciones se han centrado en el desarrollo de bibliotecas con algoritmos metaheurísticos, ya que son aplicables de manera general a cualquier tipo de problema de optimización. A continuación, se presentan algunas bibliotecas encontradas en el estado del arte que implementan algoritmos metaheurísticos:

- BiCIAM: biblioteca de clases que implementa un modelo unificado de algoritmos metaheurísticos y desarrollada en el lenguaje de programación Java. Inicialmente se enfocaba en los algoritmos metaheurísticos basados en un punto [23] [71] [88]. En la actualidad, se han incorporado nuevos algoritmos poblacionales [23]. Además, se incluye un portafolio de algoritmos con el fin de que los diferentes generadores metaheurísticos implementados colaboren y compitan entre sí. Por último, la versión más reciente, une la versión mono-objetivo y multi-objetivo, las cuales se crearon en paralelo, y se añade el algoritmo multi-objetivo NSGAII [34].
- METSlib: biblioteca que implementa algoritmos metaheurísticos básicos de la literatura [23]. Esta biblioteca presenta como objetivo principal facilitar la implementación y adaptación de modelos, ya que una vez creados son aplicables a cualquier algoritmo implementado [35].
- MOMHLib++: biblioteca de clases orientada a objetos, desarrollada en el lenguaje de programación C++. Su diseño busca la implementación de metaheurísticas multi-objetivos y permite la incorporación de nuevas metaheurísticas [38].

- *Open Metaheuristics*: biblioteca de clases desarrollada en los lenguajes de programación C/C++. Uno de sus propósitos es seguir una búsqueda de aprendizaje adaptada en el diseño de metaheurísticas. Por otra parte, permite realizar pruebas experimentales a los algoritmos metaheurísticos a través de aproximación estadística e incluye una interfaz propia [38].
- *Operation Research Tools* (OR-Tools): software de código abierto, rápido y portable desarrollada por Google para solucionar problemas de optimización combinatoria y con soporte en varios lenguajes de programación como C++, C#, Java y Python. Resuelve diferentes variantes de VRP tales como CVRP, HFVRP, MDVRP, VRPTW y problemas de entregas y recogidas. Presenta algunas heurísticas de inserción dentro de las estrategias de solución, pero incluye mayormente metaheurísticas como Búsqueda Local [13], Recocido Simulado [71], Búsqueda Tabú [88], entre otras [40].
- JSprit: herramienta de código abierto desarrollada en el lenguaje de programación Java, de manera modular y extensible. Su propósito general es resolver una amplia gama de variantes de VPR, incluyendo CVRP, MDVRP, VRPTW, VRPB, VRP con entregas y recogidas, HFVRP, TSP y *Dial a Ride Problem* (DARP). Además, emplea metaheurísticas como *Large Neighborhood Search* (LNS) [23] y variantes del Recocido Simulado [23] [42].
- OptaPlanner: marco de trabajo desarrollado en el lenguaje de programación Java, con el objetivo de configurar diferentes aspectos como las restricciones, condición de parada, heurísticas de construcción y algoritmos de búsqueda local, para obtener la solución. Además, brinda una configuración de *benchmark* para encontrar la mejor estrategia de búsqueda local [89].

Sin embargo, para la resolución de problemas de planificación de rutas de vehículos e incluso otros problemas de optimización, existen escasas bibliotecas que implementen algoritmos heurísticos. A continuación, se describen las bibliotecas encontradas en la literatura para este fin:

- Biblioteca de heurísticas de búsqueda local para el problema de ruteo de vehículos (VRPH): desarrollada en los lenguajes de programación C/C++ con un enfoque orientado a objetos. Dicha biblioteca incorpora heurísticas de búsqueda local que pueden ser utilizadas para mejorar soluciones y aplica específicamente siete para generar soluciones a la variante CVRP [67].
- *Vehicle Routing Open-source Optimization Machine* (VROOM): software de código abierto desarrollado en lenguaje de programación C++. Posee como objetivo fundamental resolver VRPs y otros problemas relacionados con logística y tareas distribuidas geográficamente. Además, emplea varias heurísticas para encontrar las soluciones iniciales en dependencia de la variante del problema, por ejemplo, una versión ajustada de la heurística de inserción de Christofides para TSP, heurísticas de agrupamiento usando árboles de expansión para CVRP, versiones ajustadas de la heurística de inserción de Solomon para CVRP y VRPTW, entre otras. Por último, emplea catorce operadores diferentes para la búsqueda local [43].
- Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP): está desarrollada en el lenguaje de programación Java y presenta la adaptación de ocho heurísticas clásicas de la literatura, como son el Algoritmo de Barrido, el Algoritmo de Ahorros en su versión paralela y secuencial, la Heurística del Vecino más Cercano con Lista de Candidatos Restringidos, la Heurística de Mole & Jameson, las Heurísticas de Inserción de Kilby y de Christofides, Mingozi y Toth (CMT), y un método aleatorio, para cuatro variantes de VRP: CVRP, MDVRP, HFVRP y TTRP [38, 39].
- Capa de servicio para soluciones a Problemas de Planificación de Rutas de Vehículos (CaSVRP): es una capa de servicios web dirigidos a la resolución de diferentes variantes de VRPs mediante algoritmos heurísticos. Esta solución utiliza BiCIAM [33, 34], BHCVRP [38] y LMOVRP [90], este último para generar nuevas soluciones a partir de las obtenidas previamente [91].

A continuación, se realiza una comparación de las bibliotecas mencionadas anteriormente, teniendo en cuenta un conjunto de características. En la Tabla 1, se muestran los criterios establecidos para dicho estudio.

Tabla 1: Comparación de bibliotecas de clases que implementan métodos heurísticos.

Bibliotecas	Características								
	Resuelve VRPs	Implementa HC	Implementa MH	Lenguaje de programación	Interoperable	Extensible	Libre y de código abierto	Calcula distancias aproximadas	Calcula distancias reales
BiCIAM	✓		✓	Java		✓	✓		
METSlib	✓		✓	C++		✓			
MOMHLib++	✓		✓	C++		✓			
Open Metaheuristics	✓		✓	C/C++		✓			
OR-Tools	✓	✓	✓	C++/C#/Java/Python	✓	✓	✓	✓	
JSprit	✓		✓	Java		✓		✓	✓
OptaPlanner	✓		✓	Java		✓			✓
VRPH	✓	✓		C/C++					
VROOM	✓	✓	✓	C++		✓			✓
BHCVRP	✓	✓		Java	✓	✓	✓	✓	
CaSVRP	✓	✓	✓	Java	✓		✓	✓	

A partir del análisis comparativo sobre las bibliotecas presentadas en la Tabla 1, se puede arribar a las siguientes conclusiones:

- La mayoría de las bibliotecas de clases implementan metaheurísticas.
- La única biblioteca interoperable es OR-Tools, aunque existe un servicio web de BHCVRP que se encuentra en desuso [38]. En el caso de CaSVRP posee una interoperabilidad limitada.
- Todas las bibliotecas de metaheurísticas son extensibles, excepto CaSVRP. Sin embargo, la biblioteca de heurísticas extensible solo es BHCVRP.
- De todas las bibliotecas solo BiCIAM, OR-Tools, BHCVRP y CaSVRP son libres y de código abierto.
- Solo los componentes de clases OR-Tools, VRPH, BHCVRP, VROOM y CaSVRP implementan heurísticas de construcción para la resolución de variantes VRP.
- Solo los componentes de software OR-Tools, JSprit, BHCVRP y CaSVRP implementan cálculo de distancias aproximadas.
- Solo las herramientas JSprit, OptaPlanner y VROOM implementan cálculo de distancias reales a partir de servicios OSRM.

Por último, resulta interesante analizar las tecnologías empleadas en el desarrollo de estos componentes de softwares que solucionan problemas de planificación de rutas de vehículos. A partir del estudio plasmado anteriormente, se evidencia que la mayoría de estos componentes se han desarrollado en los lenguajes de programación C++ y Java. Sin embargo, en los últimos tiempos la comunidad de Python ha crecido exponencialmente, sobre todo en trabajos relacionados con Inteligencia Artificial y problemas de optimización [44, 45]. A pesar de la existencia de OR-Tools en este lenguaje, se identifican deficiencias; por ejemplo, se aplican pocas heurísticas de construcción para solucionar VRP y las heurísticas existentes son deterministas, lo cual limita el espacio de búsqueda de nuevas soluciones. Sin embargo, también resulta interesante analizar aspectos como el cálculo de distancias reales para obtener una mayor precisión en los resultados. En el próximo epígrafe se aborda este tema y los

posibles beneficios del trabajo con distancias reales, en vez de con distancias aproximadas.

1.3.1 Bibliotecas especializadas en el cálculo de distancias

El cálculo de distancias es un componente esencial en problemas de optimización, especialmente en los problemas de planificación de rutas de vehículos. La precisión en la generación de matrices de costo depende en gran medida de la estrategia utilizada. Estas estrategias pueden ser a través de distancias aproximadas o reales. Las distancias aproximadas utilizan modelos simplificados para estimar el costo de las rutas de manera más rápida, pero menos precisa. Este enfoque es particularmente valioso en contextos donde el tiempo computacional y los recursos son limitados, permitiendo obtener soluciones de calidad sin necesidad de realizar cálculos exhaustivos. Entre los métodos más comunes se incluyen: la distancia *Euclidean* [46], *Manhattan* [47], *Haversine* [48] y *Chebyshev* [49]. Dentro de las bibliotecas que implementan dichas distancias se encuentran SciPy [92], Geopy [93] y Scikit-learn [94]. Es importante destacar que estas distancias son fáciles de modelar y por ende no siempre es necesario el uso de bibliotecas que la implementen.

Por otra parte, las distancias reales se aplican en los servicios de mapas. Su principal exponente es *Open Street Map* (OSM) [53], que proporciona una plataforma colaborativa con información de mapas gratuitos y abiertos. Los datos incluyen redes viales, información topológica, así como etiquetas con características de carreteras relacionadas con la velocidad máxima o el tipo de vía. Su enfoque promueve la actualización y mejora continua de los datos. Además, se destaca *Open Source Routing Machine* (OSRM) [54], como un motor de código abierto diseñado para calcular rutas óptimas en redes de carreteras. Esta herramienta maneja grandes volúmenes de datos de mapas.

Sin embargo, existen otras herramientas que ofrecen soluciones para calcular distancias reales basadas en datos geospaciales precisos. En este contexto se destacan:

- *Google Distance Matrix API*: calcula distancias y tiempos de viaje reales entre puntos específicos utilizando datos de *Google Maps*. Además, es compatible con

varios modos de transporte, integra información de tráfico en tiempo real y geocodifica a coordenadas de latitud y longitud [50].

- *GraphHopper*: biblioteca de código abierto para la planificación de rutas de vehículos sobre redes viales con datos de OSM. Esta herramienta permite personalizar las rutas generadas mediante perfiles de vehículos y restricciones específicas, así como su ejecución sin necesidad de conexión [51].
- *Mapbox Directions API*: proporciona cálculos de rutas y distancias utilizando datos de *Mapbox*. Resulta apropiada para aplicaciones que requieren distancias reales en múltiples modos de transporte y ofrece opciones para evitar restricciones específicas como carreteras cerradas [52].

Estas herramientas proporcionan un alto nivel de precisión y flexibilidad, siendo una alternativa interesante para aplicar en contextos de optimización de rutas como en BHCVRP. Por tanto, a partir de este pequeño análisis, se propone aplicar alguno de estos componentes de software para obtener mejores soluciones en cuanto a calidad de costo en distancia.

1.4 Especificaciones de BHCVRP

En la actualidad existen escasas bibliotecas que implementen métodos heurísticos para la solución de problemas de optimización, así como problemas de planificación de rutas de vehículos. BHCVRP es una biblioteca de clases que implementa heurísticas de construcción para resolver problemas de planificación de rutas de vehículos. Esta biblioteca fue desarrollada en [38], en el lenguaje de programación Java y contiene la adaptación de ocho heurísticas clásicas para las variantes CVRP, MDVRP, HFVRP y TTRP. Las heurísticas implementadas son: el Algoritmo de Barrido, las dos versiones del Algoritmo de Ahorros (secuencial y paralela), la Heurística del Vecino más Cercano con Lista de Candidatos Restringidos, la Heurística de Inserción de Mole & Jameson, la Heurística de Christofides, Mingozzi y Toth, la Heurística de Inserción de Kilby y un método aleatorio [39].

Para el flujo de trabajo se necesita un usuario experto en el tema o una herramienta como TransO [95], o la propia BHCVRP. En la Figura 4, se describe mediante un diagrama de actividades su funcionamiento.

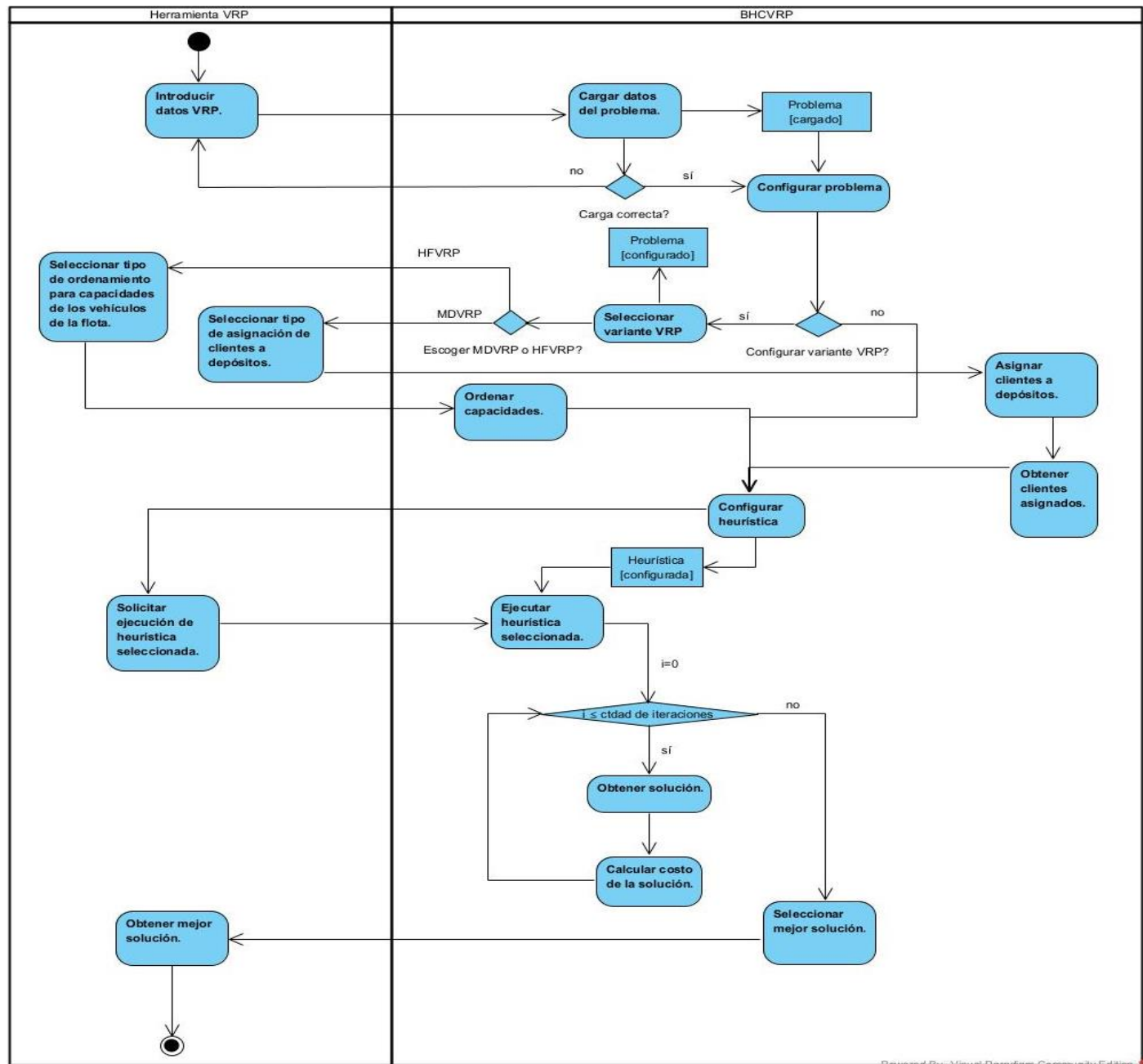


Figura 4: Diagrama de actividades del funcionamiento de BHCVRP.

El flujo se inicia con la introducción de los datos correspondientes a la variante VRP que se desea resolver. Estos datos son procesados por BHCVRP que, a través de sus

heurísticas, devuelve la mejor solución posible, es decir, el conjunto de rutas que minimiza el costo en distancia. Es importante destacar que la biblioteca no cuenta con un servicio diferenciado para diferentes tipos de usuario, ya que no brinda una configuración predeterminada para aquel que no sea experto en el campo de optimización de rutas.

Por último, BHCVRP ofrece un conjunto de funcionalidades que promueven la robustez y adaptabilidad del componente. Entre ellas, la carga de ficheros de entrada en formato estándar *.data* o *.txt*, facilitando la incorporación de datos al sistema. Además, brinda opciones para configurar parámetros específicos de las diferentes heurísticas de construcción según las necesidades del problema que se desea resolver.

1.4.1 Diseño arquitectónico de BHCVRP

El diseño de BHCVRP plantea una arquitectura n-capas con enfoque basado en reutilización, con el objetivo de garantizar su modularidad y adaptabilidad. Para ello se tienen en cuenta cuatro capas: específica, general, intermedia y software del sistema. En la Figura 5 se muestra el diagrama arquitectónico en cuestión.

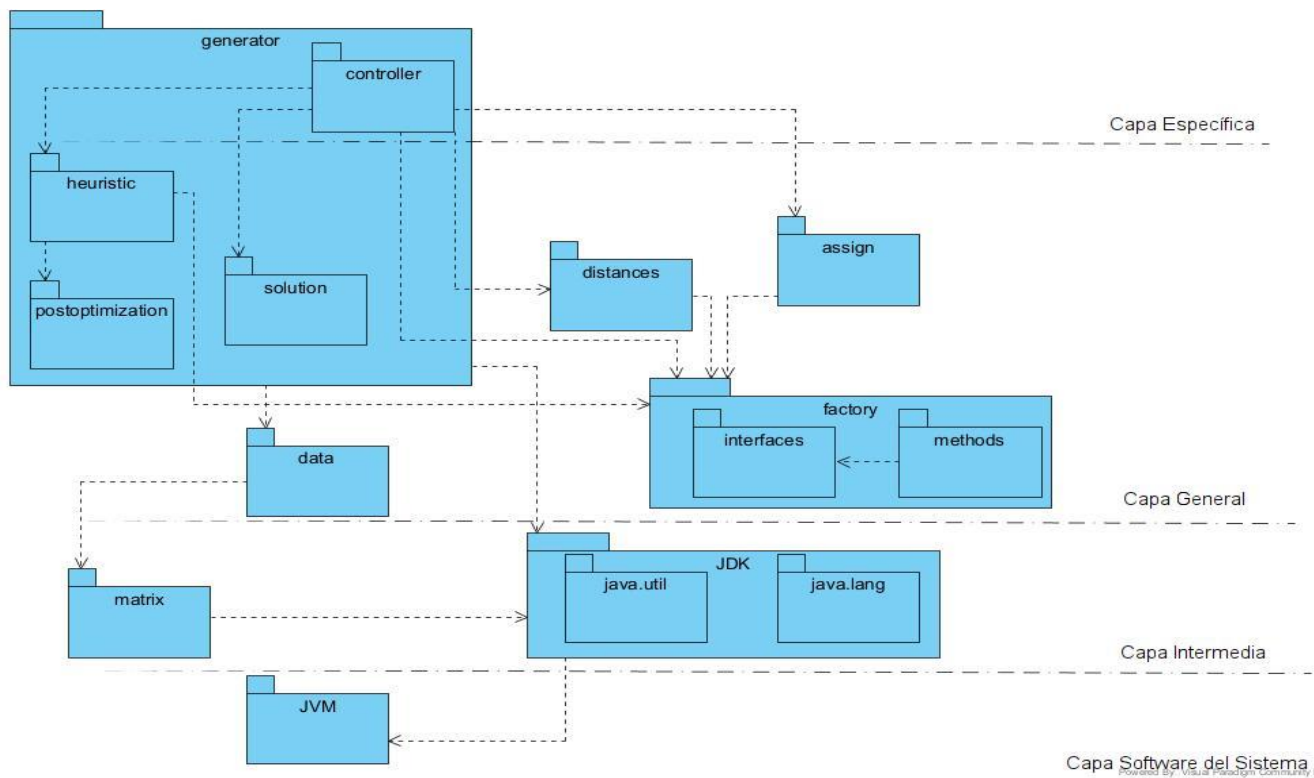


Figura 5: Patrón n-capas con enfoque basado en reutilización de BHCVRP [38].

La capa software del sistema garantiza los protocolos de comunicación. En este caso se aplica a través de la *Java Virtual Machine* (JVM), que constituye la comunicación entre las diferentes librerías utilizadas en la tecnología Java. En BHCVRP se realiza principalmente a través de llamadas de métodos y paso de objetos dentro del mismo proceso JVM.

La capa intermedia es aquella que contiene bibliotecas externas al proyecto. En BHCVRP se evidencia a través del paquete JDK, que contiene clases, implementaciones y excepciones necesarias para el desarrollo del sistema a través de las librerías *java.util* y *java.lang* [78]. Además, en el paquete *matrix*, que contiene los métodos necesarios para el trabajo con matrices de costo.

La capa general está compuesta por paquetes necesarios para el funcionamiento de las heurísticas y la modelación de los datos del problema de planificación de rutas de vehículos que se quiere resolver. Está compuesto por los paquetes *distances*, *assign*, *factory*, *postoptimization*, *heuristic*, *solution* y *data*. A continuación, se describen dichos paquetes y sus funcionalidades.

El paquete *distances* contiene los tipos de distancias *Euclidean*, *Manhattan* y *Chebyshev*, con sus requerimientos. Esta medida es útil para la construcción de la matriz de costos y dar el costo final de la solución. Se presenta en la Figura 6 el diagrama de clases correspondiente:

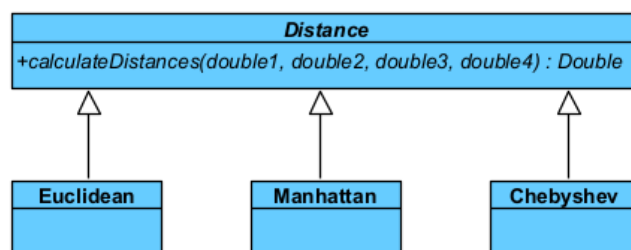


Figura 6: Diagrama de clases del paquete *distances* [38].

El paquete *assign* se encarga de la asignación de clientes a depósitos en la variante MDVRP. En la Figura 7, se muestra su diagrama de clases asociado con los distintos métodos de asignación. Sin embargo, es importante destacar que este proceso sólo es relevante para la variante MDVRP y existen herramientas especializadas para la

realización de esta tarea. En este contexto existe la Biblioteca de Heurísticas de Asignación para el Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) [96] que propone diferentes heurísticas de asignación y algoritmos de agrupamiento para la resolución de esta variante VRP.

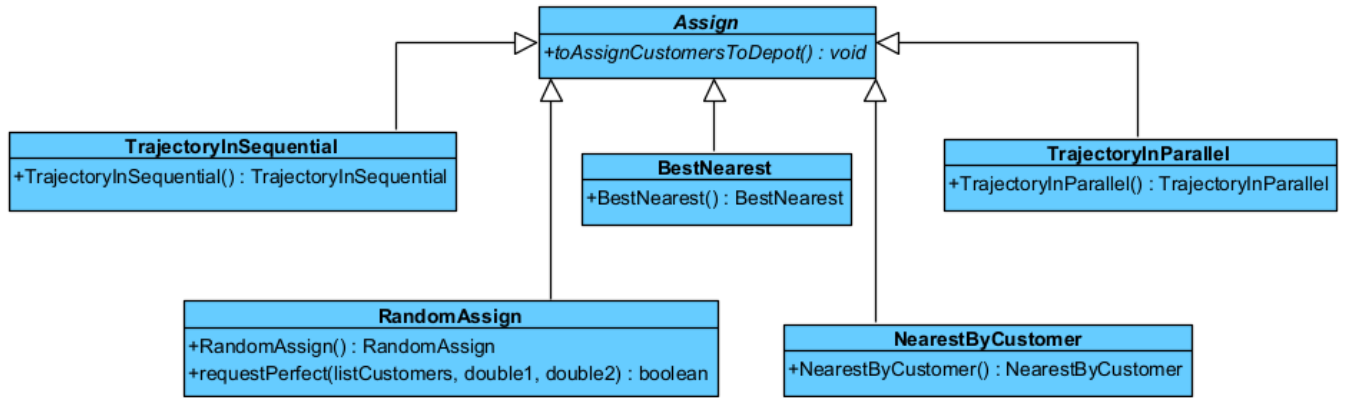


Figura 7: Diagrama de clases del paquete *assign* [38].

El paquete *factory* contiene las interfaces y métodos necesarios para la implementación del patrón de diseño *Factory Method* [97]. En este caso se aplica para la creación de distancias y heurísticas. En la Figura 8, se presenta el diagrama de clases relacionado:

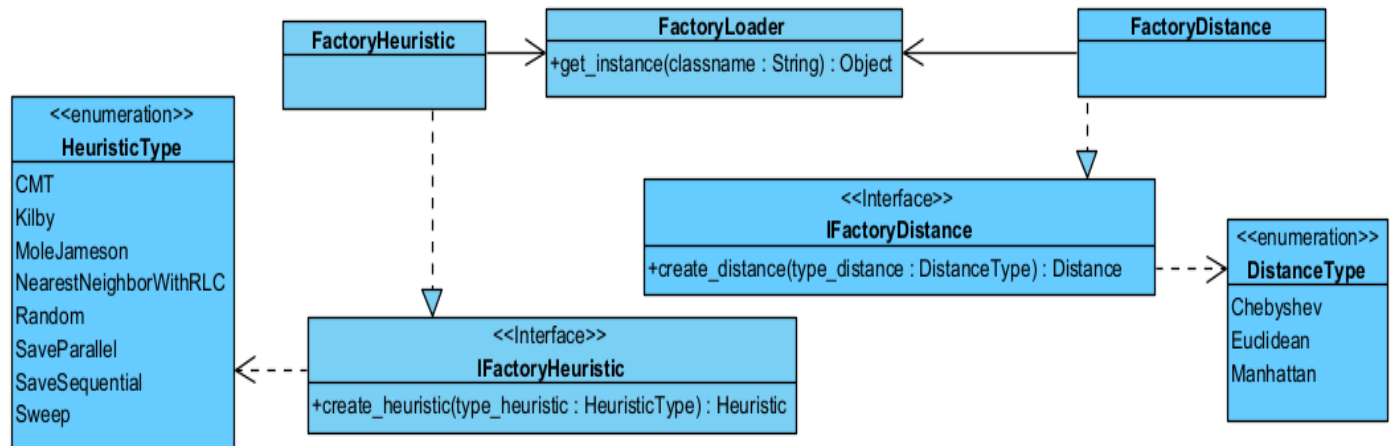


Figura 8: Diagrama de clases del paquete *factory*.

El paquete *postoptimization* contiene métodos de post-optimización, necesarios para la obtención de soluciones en determinadas heurísticas. En la Figura 9 se muestra el respectivo diagrama de clases:

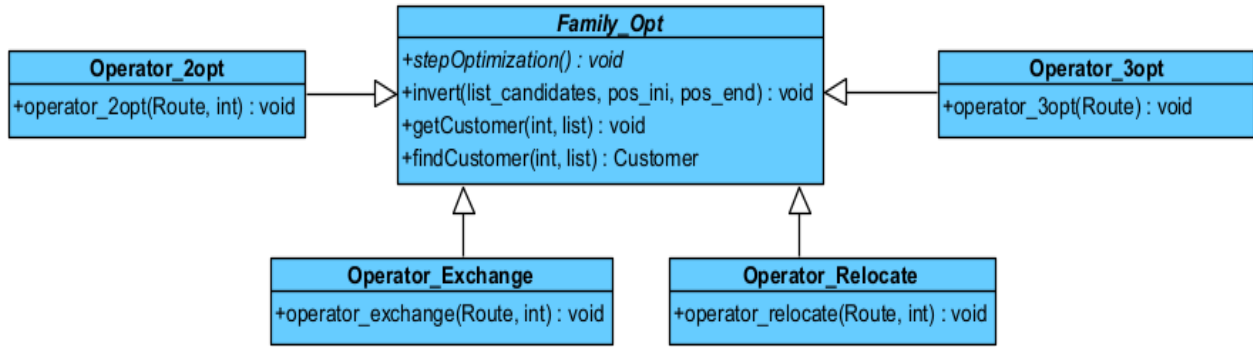


Figura 9: Diagrama de clases del paquete *postoptimization*.

El paquete *heuristic* se encarga de presentar todas las heurísticas disponibles para la construcción de rutas. En la Figura 10, se muestra el diagrama de clases asociado:

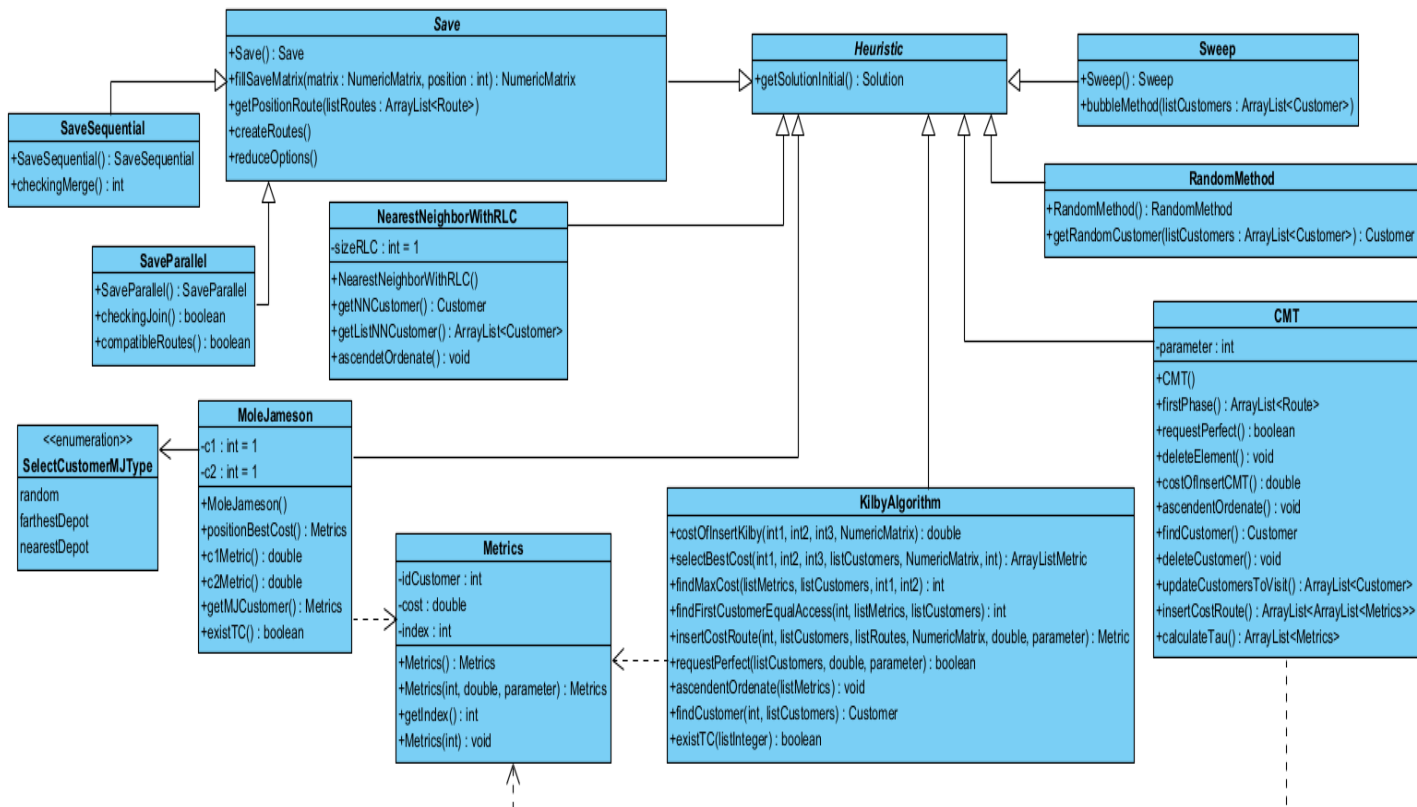


Figura 10: Diagrama de clases del paquete *heuristic*.

El paquete *solution* se encarga de modelar una solución y el conjunto de rutas en dependencia del tipo de VRP. En la Figura 11, se muestra su respectivo diagrama de clases:

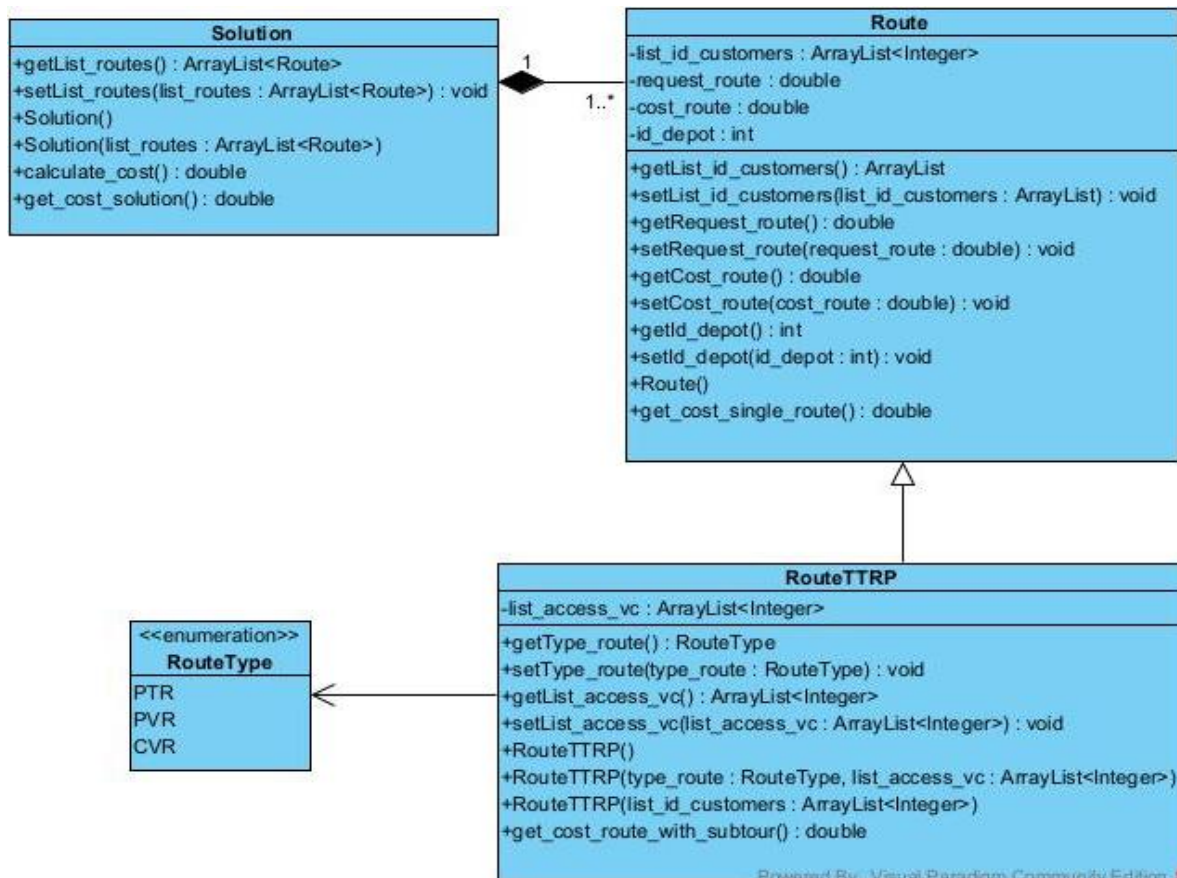
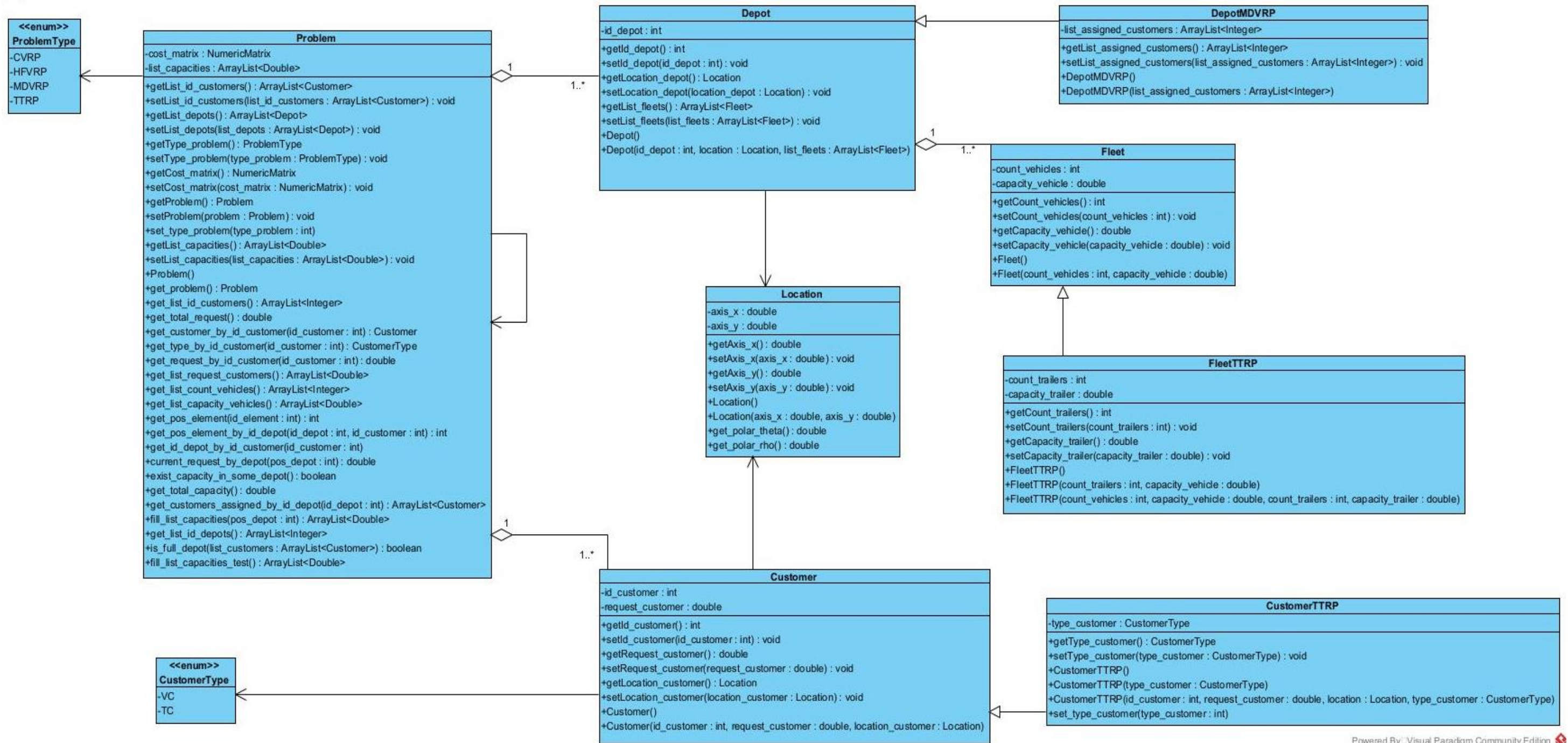


Figura 11: Diagrama de clases del paquete *solution*.

El paquete *data* es el responsable de modelar los datos del VRP en cuestión. Para ello tiene en cuenta la información de los clientes, depósitos, flota de vehículos y ubicación. Además, incluye una clase controladora *Problem*. En la Figura 12, se muestra su diagrama de clases:



Powered By Visual Paradigm Community Edition

Figura 12: Diagrama de clases del paquete *data*.

Por último, la capa específica contiene el paquete *controller*, responsable del control centralizado del sistema, donde se obtienen las posibles soluciones al problema. En la Figura 13, se muestra el diagrama de clases relacionado con la clase controladora:

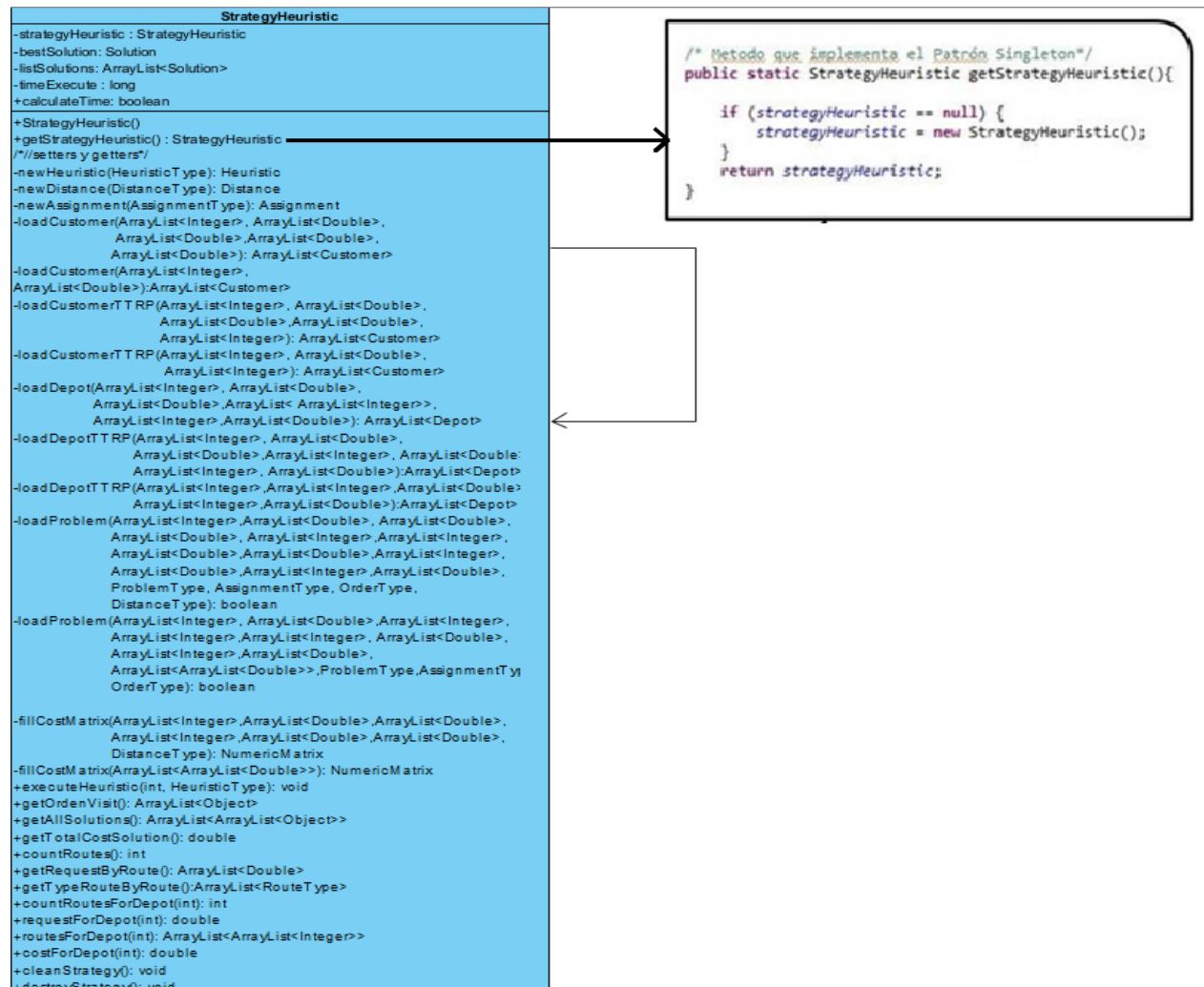


Figura 13: Diagrama de clases del paquete *controller*.

1.4.2 Patrones de diseño implementados en BHCVRP

Durante el proceso de diseño orientado a objetos, es común encontrarse con ciertos tipos de problemas de manera recurrente. Para analizar, compartir y documentar el conocimiento sobre estos problemas, se han desarrollado los patrones de diseño. Estos patrones son modelos formales aplicables a diferentes dominios y ayudan a seguir pautas comunes en la solución de problemas similares en su estructura [97, 98]. En esencia, los patrones de diseño permiten identificar problemas concretos junto con sus

soluciones. Son una forma de documentar la experiencia acumulada en las estrategias empleadas para resolver problemas específicos dentro de un contexto determinado [99].

En el ámbito del diseño de software, los patrones de diseño *Gang of Four* (GoF, por sus siglas en inglés) son esenciales. Estos patrones, descritos en [98], se han convertido en herramientas indispensables para los programadores. Los patrones GoF se dividen en tres categorías según su propósito [98]:

- Patrones estructurales: permiten crear grupos de objetos para abordar tareas complejas.
- Patrones de comportamiento: definen la comunicación entre los objetos de un sistema y el flujo de información entre ellos.
- Patrones creacionales: facilitan la creación de objetos sin necesidad de instanciarlos directamente, lo que brinda a los programas mayor flexibilidad para decidir qué objetos utilizar.

En BHCVRP se garantiza la flexibilidad y reusabilidad del diseño con la utilización de patrones. A continuación, se describe la implementación de los patrones creacionales *Singleton* y *Factory Method* [98-100].

Patrón Singleton

El patrón Singleton está diseñado para restringir la creación de objetos pertenecientes a una clase a un único objeto. Su intención consiste en garantizar que una clase sólo tenga una instancia y proporcionar un punto de acceso global a ella. En otras palabras, provee un mecanismo para limitar el número de instancias de una clase, por lo que el mismo objeto es siempre compartido por distintas partes del código [98-100].

Las situaciones más habituales de aplicación de este patrón son aquellas en las que dicha clase controla el acceso a un recurso físico único o cuando cierto tipo de datos debe estar disponible para todos los demás objetos de la aplicación.

En la Figura 12 y Figura 13, se puede apreciar la implementación de este patrón en BHCVRP. Se aplica en las clases *StrategyHeuristic* y *Problem*, ya que constituyen la clase controladora y la que modela los datos del problema que se desea resolver, respectivamente.

Patrón Factory Method

El patrón *Factory Method* define una interfaz para la creación de un objeto dejando a sus sub-clases la responsabilidad de decidir qué clases instanciar. En concreto retorna una instancia de una o varias posibles clases, en dependencia de los datos que se le provea, sin conocer previamente las clases de objetos a crear [98-100].

Para la implementación del mecanismo de diseño se creó la clase *FactoryLoader*, que contiene el método estático *getInstance()*, que recibe como parámetro el nombre de la clase que se desea instanciar. El comportamiento de este método se basa en verificar que exista una clase con el mismo nombre que el parámetro que se le pasa y devolver una instancia de la misma. En la Figura 8, se puede apreciar el diseño de clases que implementa este patrón para el caso de las heurísticas de construcción y las distancias.

Otros patrones de diseño

Dentro de los patrones más usados además de los GoF también se encuentra el grupo de los patrones GRASP (acrónimo de *General Responsibility Assignment Software Patterns*). Estos patrones son los que describen los principios fundamentales de la asignación de responsabilidades en objetos [99, 100]. Los GRASP no implementan las soluciones, más bien llevan a pensar en el diseño, a nivel de principios generales. A continuación, en la Tabla 2, se describen los que se ponen en práctica en la implementación de BHCVRP [38].

De la Tabla 2 se concluye que en BHCVRP se aplican estos patrones de manera eficaz para optimizar su arquitectura. El patrón Controlador se implementa en la clase *StrategyHeuristic* (ver Figura 13), que centraliza la gestión de eventos del sistema y facilita el desacoplamiento. A través del patrón Experto se pretende resolver el tema de delegar responsabilidades a cada objeto. Por tanto, clases como *Problem*, *Customer* y *Depot* (ver Figura 12), gestionan su propia información, promoviendo cohesión y encapsulación. El patrón Creador asegura que objetos clave como *Customer* y *Depot* sean creados correctamente mediante la clase *Problem* (ver Figura 12). Por otra parte, el patrón Bajo Acoplamiento reduce dependencias mediante componentes como la clase *Save*, que centraliza los comportamientos comunes de *SaveSequential* y *SaveParallel*

(ver Figura 10). Por último, el patrón Alta Cohesión limita las responsabilidades a funciones específicas, por ejemplo, en clases como *Route* y *Solution* (ver Figura 11).

Tabla 2: Patrones de diseño GRASP implementados en BHCVRP.

Patrón	Criterio de evaluación	Grado de cumplimiento
Controlador	Una clase centralizada gestiona los eventos y actúa como intermediaria.	Total
	El patrón facilita pruebas y promueve modularidad.	Amplio
Experto	Las responsabilidades recaen en las clases que poseen la información necesaria.	Medio
	Promueve cohesión y encapsulamiento.	Amplio
Creador	Las clases responsables crean instancias de las clases que necesitan.	Amplio
	La creación está alineada con el ciclo de vida lógico de las instancias.	Amplio
Bajo Acoplamiento	Las dependencias entre clases están minimizadas.	Amplio
	Se garantiza que los cambios en una clase no impacten directamente en otras.	Medio
Alta Cohesión	Las clases tienen responsabilidades relacionadas y específicas.	Bajo

A partir del análisis realizado previamente sobre los patrones de diseño implementados en BHCVRP, se podría considerar la incorporación de otros patrones GoF para propiciar mejoras significativas. Por ejemplo, el patrón *Template Method* podría estandarizar los pasos comunes en la ejecución de las heurísticas, garantizando que las subclases implementen detalles específicos, mientras mantienen una estructura coherente y reutilizable en los algoritmos. Además, los únicos patrones GoF implementados en

BHCVRP son patrones creacionales, lo que indica una oportunidad para enriquecer su diseño.

1.4.3 Deficiencias en BHCVRP

A partir del estudio y el análisis de la arquitectura y del código fuente de la biblioteca de clases BHCVRP, se han identificado un conjunto de deficiencias que no permiten realizar un uso eficiente de la misma. A continuación, se describen estas deficiencias y posibles mejoras:

- No presenta un paquete para tratar excepciones propias relacionadas con parámetros como la distancia, tiempo, capacidad, existencia o no de un depósito, cliente, vehículo, entre otros. Solamente se utilizan las que brinda el paquete *java.lang* propio del lenguaje de programación.
- No posee precisión en los cálculos de distancias. Actualmente, BHCVRP se basa en cálculos de distancias aproximadas en lugar de distancias reales, lo cual puede afectar la exactitud en la construcción de rutas.
- No cuenta con funcionalidades para exportar los resultados generados en formatos estándar como CSV, JSON o XML. Esto limita la capacidad de integrar los resultados con herramientas externas de análisis o visualización, restringiendo su aplicación en algunos entornos.
- A nivel de código se presentan ciertas deficiencias. Se debe mejorar la implementación de las heurísticas para propiciar la flexibilidad y reutilización del código.
- El paquete *assign*, debe actualizarse por el nuevo componente BHAVRP [96] para solucionar la variante MDVRP. Además, la posible incorporación de nuevos métodos de asignación, así como admitir por parte del usuario una asignación predeterminada.
- Con el objetivo de proporcionar mayor flexibilidad a la biblioteca, se debe permitir al usuario el uso de los pasos de post-optimización a su conveniencia.

- Se deben incorporar nuevas variantes de VRP debido a la importancia que tienen en la vida práctica y en la literatura. Por ejemplo: OVRP, VRPTW y SBRP.
- Además, la biblioteca debe ser capaz de adaptar nuevas heurísticas de construcción de la literatura como el Algoritmo de Pétalos, la Heurística de Asignación Generalizada de Fisher y Jaikumar, entre otras.
- Como parte de las buenas prácticas que se siguen en el diseño de software, se necesitan aplicar patrones y principios de diseño para favorecer la flexibilidad y reutilización, ya que solo se aplican *Singleton* [97] y *Factory Method* [97].
- Existe una deficiencia arquitectónica, ya que la biblioteca no brinda servicio diferenciado para dos tipos de usuarios: uno experto con conocimientos de optimización con necesidades de configurar la mayor cantidad de parámetros posibles y otro usuario con conocimientos de su problema VRP real con una configuración predeterminada a partir de los mejores resultados alcanzados en experimentos. Actualmente, no se proporciona una configuración por defecto.
- A la biblioteca le falta interoperabilidad, lo cual implica que no se comunique e integre con otros componentes, plataformas o aplicaciones, intercambie datos, se utilice en diferentes entornos, siga con estándares de codificación, de la manera más efectiva posible. En primera instancia, se probó con la herramienta TransO [95] y el componente CaSVRP [91]. Aunque existe una versión web en desarrollo [38].

1.5 Conclusiones parciales

A partir del estudio realizado sobre el marco teórico investigativo de los problemas de optimización combinatoria, concretamente los Problemas de Planificación de Rutas de Vehículos, sus variantes y los algoritmos heurísticos que existen para resolver dicho problema, se obtienen las siguientes conclusiones:

- Los Problemas de Planificación de Rutas de Vehículos constituyen un problema de optimización combinatoria perteneciente a la categoría de problemas NP-duro, cada vez surgen más variantes y una alternativa de solución apropiada son los algoritmos heurísticos y metaheurísticos.

- Para los Problemas de Planificación de Rutas de Vehículos existen heurísticas de construcción que obtienen buenas soluciones y pueden ser aplicadas como punto de partida en el contexto de las metaheurísticas.
- Los componentes de software han centrado mayormente su desarrollo en las metaheurísticas. De los componentes de software estudiados, solo BiCIAM, VRPH, BHCVRP, JSprit, VROOM, CaSVRP y OR-Tools solucionan VRPs. Sin embargo, solo VROOM, OR-Tools, VRPH, CaSVRP y BHCVRP implementan heurísticas de construcción.
- Debido a la relevancia del lenguaje de programación Python para los trabajos relacionados con Inteligencia Artificial, es importante explorar sus potencialidades. Además, de los componentes estudiados previamente, solo OR-Tools se encuentra desarrollado en Python y resuelve VRPs.
- Resulta interesante analizar la posibilidad de incorporar el trabajo con distancias reales para aumentar la precisión de los resultados en la obtención de rutas en VRPs. Solo los componentes JSprit, OptaPlanner y VROOM implementan estos cálculos.
- La Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos, solo es capaz de resolver las variantes CVRP, MDVRP, HFVRP y TTRP, con ocho heurísticas de construcción.
- La Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos, presenta deficiencias en cuanto a la arquitectura y código fuente. Principalmente sus debilidades se encuentran en la implementación de las heurísticas de construcción y la ausencia de excepciones propias. Por último, las mejoras deben enfocarse en la aplicación de métodos de post-optimización, el cálculo de distancias reales y la incorporación de nuevas variantes VRP y heurísticas de construcción.

Capítulo 2: Diseño de la nueva versión de BHCVRP

La solución que se presenta en este capítulo consiste en una actualización de la biblioteca de clases BHCVRP. La nueva versión parte de un conjunto de cambios introducidos en la arquitectura para garantizar la extensibilidad del componente. Como parte de las transformaciones relevantes es importante mencionar que se propone la primera versión de la biblioteca que cubre dos lenguajes de programación: Python y Java. Además, se incorporan dos nuevas adaptaciones para las heurísticas de construcción: Algoritmo de Ahorros basado en *Matching* y la Heurística de Asignación Generalizada de Fisher & Jaikumar, así como tres nuevas variantes de VRP: Problema de Planificación de Rutas de Autobuses Escolares (SBRP), Problema de Planificación de Rutas de Vehículos Abiertos (OVRP) y Problema de Planificación de Rutas de Vehículos con Ventanas de Tiempo (VRPTW). Para describir la solución se muestra una vista de la arquitectura y los diagramas de clases más significativos. Por último, se presentan los patrones y principios de diseño utilizados.

2.1 Modificaciones en la arquitectura de BHCVRP

La Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP) es una propuesta del proyecto de Optimización y Metaheurísticas, de la Facultad de Ingeniería Informática de la CUJAE en el año 2016 [38, 39]. Esta biblioteca está desarrollada en el lenguaje de programación Java y su objetivo es utilizar heurísticas de construcción para solucionar diferentes variantes VRP. Con este fin la biblioteca cuenta con ocho heurísticas de construcción, para las variantes CVRP, MDVRP, HFVRP y TTRP, la Heurística del Vecino más Cercano con Lista de Candidatos Restringidos, el Algoritmo de Barrido, el Algoritmo de Ahorros en sus versiones secuencial y paralela, la Heurística de Inserción de Mole & Jameson, la Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (CMT), la Heurística de Inserción de Kilby y un método aleatorio.

A partir del análisis realizado en la sección **1.4 Especificaciones de BHCVRP**, no solo se encuentran un conjunto de deficiencias, sino también oportunidades para mejorar su funcionamiento. Estas modificaciones se listan a continuación.

- Incorporar la versión restante del Algoritmo de Ahorros, es decir, su variante basada en *Matching*, así como la Heurística de Asignación Generalizada de Fisher & Jaikumar.
- Incorporar las variantes SBRP, OVRP y VRPTW.
- Ofrecer un paquete de excepciones propias para el tratamiento de restricciones relacionadas con la distancia, capacidad, entre otras.
- Utilizar servicios OSRM para el trabajo con distancias reales.
- La incorporación de BHAVRP para realizar la asignación de clientes a depósitos en la variante MDVRP.
- Utilizar un formato JSON para la carga de datos y emplear formatos como *.txt*, JSON y CSV para la exportación de resultados.
- Adaptar el patrón de diseño *Template Method* para mejorar la reutilización del código y la flexibilidad de las heurísticas implementadas.

2.1.1 Nuevas variantes de problemas de planificación de rutas de vehículos en BHCVRP

En esta nueva versión se decide adaptar las heurísticas de construcción de la biblioteca para las siguientes variantes: SBRP [16], OVRP [72, 73] y VRPTW [66], descritos en la sección **1.1 Problema de planificación de rutas de vehículos**. A continuación, se mencionan las modificaciones realizadas en la biblioteca para cada caso.

Problema de Planificación de Rutas de Autobuses Escolares (SBRP)

Esta variante comprende un conjunto de autobuses para recoger a estudiantes concentrados en paradas. En esta propuesta se soluciona la variante de SBRP para una sola escuela en un entorno urbano con una flota homogénea. El objetivo es minimizar la distancia total del conjunto de rutas. Una solución es factible si todas las rutas satisfacen las restricciones de distancia, capacidad máxima del autobús, máxima distancia a caminar y capacidad de la parada. Además, deben comenzar y terminar en el mismo depósito (escuela) [16]. Para la incorporación de esta variante en la biblioteca se realizan algunas modificaciones que se mencionan a continuación:

- Se implementa una clase para modelar los datos referentes a la parada (ver Figura 15).
- Se adiciona a la clase responsable de modelar los datos del problema un objeto que representa todas las paradas presentes en el problema y un atributo relacionado con la restricción de la máxima distancia a caminar (ver Figura 15).
- Se adiciona la variante de problema de planificación de rutas de autobuses escolares como tipo de problema que pueden ser solucionado con la biblioteca (ver Figura 15).
- Se modifica el método *getCustomerByID(idCustomer, listCustomers)* presente en la clase abstracta *Heuristic*, para que sea capaz de obtener tanto un cliente como una parada. Por tanto, cambia su nombre a *get_element_by_id(id_element, list_elements)* (ver Figura 18).
- Se adiciona a cada heurística el comportamiento para dar solución a esta variante.

Problema de Planificación de Rutas de Vehículos Abiertos (OVRP)

Esta variante se comporta del mismo modo que la variante CVRP, con la diferencia de que el vehículo no regresa al depósito central. Su objetivo es minimizar la distancia total del conjunto de rutas. La solución se considera factible si se cumplen con las restricciones de distancia [72, 73]. Para la incorporación de esta variante en la biblioteca se realizan algunas modificaciones que se mencionan a continuación:

- Se adiciona la variante de problema de planificación de rutas de vehículos abiertos como tipo de problema que pueden ser solucionado con la biblioteca (ver Figura 15).
- Se modifica el método *get_cost_single_route()* perteneciente a la clase encargada de modelar los datos de una ruta, con el objetivo de no calcular el costo de retornar al depósito (ver Figura 11).

Problema de Planificación de Rutas de Vehículos con Ventanas de Tiempo (VRPTW)

Esta variante el comportamiento es similar a la variante CVRP, adicionando las ventanas de tiempo. Su objetivo es minimizar la distancia total del conjunto de rutas cumpliendo

con las demandas de los clientes y respetando sus respectivas ventanas de tiempo. La solución se considera factible si se cumplen con las restricciones de distancia, horarios de servicio y tiempo de servicio [66]. Para la incorporación de esta variante en la biblioteca se realizan algunas modificaciones que se mencionan a continuación:

- Se implementa una clase para modelar los datos referentes a la ventana de tiempo (ver Figura 15).
- Se adiciona a la clase responsable de modelar los datos del cliente un objeto que representa su ventana de tiempo (ver Figura 15).
- Se adiciona la variante de problema de planificación de rutas de vehículos con ventanas de tiempo como tipo de problema que pueden ser solucionado con la biblioteca (ver Figura 15).
- Se adiciona a la clase responsable de modelar los datos del problema los atributos relacionados con la matriz de tiempo y la velocidad del vehículo (ver Figura 15).
- Se adiciona el método *get_time_windows()* en la clase responsable de modelar los datos del problema, para obtener la lista de ventanas de tiempo de todos los clientes en cuestión (ver Figura 15).
- Se adiciona el método *fill_time_matrix(list_distances, vehicle_speed)* en la clase controladora *StrategyHeuristic* para obtener la matriz de tiempo (ver Figura 20).
- Se adiciona a cada heurística el comportamiento para dar solución a esta variante.

2.1.2 Nuevas heurísticas de construcción en BHCVRP

Como parte de la extensión de BHCVRP se incorporan dos nuevas adaptaciones de las heurísticas de construcción: el Algoritmo de Ahorros basado en *Matching* y la Heurística de Asignación Generalizada de Fisher & Jaikumar. A continuación, se describe el funcionamiento de cada una de estas heurísticas constructivas, así como las modificaciones necesarias para su incorporación.

Algoritmo de Ahorros basado en Matching (Matching-based Saving Algorithm)

A partir del Algoritmo de Ahorros [7], surge la variante basada en *Matching* [85]. Elegir siempre el máximo ahorro es una estrategia demasiado voraz e implica, en algunos

casos, que las uniones que permanecen factibles no sean buenas. En el Algoritmo de Ahorros basado en *Matching*, se considera la afectación que provoca la unión a realizar, a las posibles uniones en próximas iteraciones. Para esto, se considera un conjunto de ahorros potenciales entre las rutas, donde cada ahorro representa la reducción en costos al combinar dos rutas en una sola. Luego, se utiliza un *matching* de peso máximo para seleccionar las combinaciones de rutas que maximizan los ahorros totales, garantizando una solución globalmente buena [31].

Hallar un *matching* de peso máximo puede resolverse en tiempo polinomial, en dependencia de la cantidad de nodos. Sin embargo, para problemas grandes se recomienda hallar el *matching* mediante alguna heurística. Según [101] y [102], surgen dos alternativas: combinar solo las rutas que correspondan al arco de mayor peso del *matching* o conectar los nodos donde se calcula el *matching* con nodos ficticios de pesos tales que prioricen la inclusión de algunos de los nodos ficticios en el *matching*. De esta última manera, se garantiza que no todos los arcos del *matching* correspondan con uniones de rutas [31]. Por último, en [103] se propone modificar el ahorro para privilegiar las uniones que están lejos de ser no factibles. Cuando no hay más combinaciones de rutas que sean factibles, algunas rutas son divididas estocásticamente, permitiendo continuar con la búsqueda de soluciones [31].

Según [31], este algoritmo consta de cuatro pasos:

1. Inicialización: para cada cliente i construir la ruta $(0, i, 0)$.
2. Cálculo de ahorros: actualizar S_{pq} para cada par de rutas p y q que pueda ser combinado manteniendo la factibilidad. Si ningún par de rutas puede ser combinado, terminar.
3. *Matching*: resolver un problema de *matching* de peso máximo donde se tienen las rutas de la solución actual como nodos y existe un arco entre las rutas p y q con peso S_{pq} si su combinación es factible.
4. Uniones: dado el *matching* de peso máximo, combinar todo par de rutas p y q tal que (p, q) pertenezca al *matching*. Ir al paso 2.

Para la incorporación de esta heurística se realizan algunas modificaciones que se mencionan a continuación:

- Se implementa una clase que modela el comportamiento de la heurística para las siete variantes de VRP existentes (ver Figura 18).
- Se adiciona el Algoritmo de Ahorros basado en *Matching* como tipo de heurística que puede ser empleada para resolver VRP en la biblioteca (ver Figura 17).
- Se incorporan métodos auxiliares que implementa el problema de *matching* de peso máximo en el proceso de construcción de rutas del Algoritmo de Ahorros basado en *Matching* (ver Figura 18).

Heurística de Asignación Generalizada de Fisher & Jaikumar

Esta heurística desarrollada por [24] posee dos etapas. La primera etapa consiste en crear clústeres de clientes a visitar por cada vehículo solucionando un problema de asignación generalizada (GAP, por las siglas en inglés de *Generalized Assignment Problem*) [24]. Las rutas son creadas en función de cada vehículo. En la segunda etapa el problema es tratado como un TSP. A continuación, se presentan los pasos de este algoritmo según [104]:

1. Calcular las distancias entre todos los puntos (depósito y clientes).
2. Dividir angularmente el plano donde los clientes están juntos en conos iguales al número de vehículos.
3. Escoger un cliente semilla para guiar el proceso de asignación de clientes a vehículos.
4. Calcular el costo de inserción entre clientes semillas y otros clientes.
5. Asignar clientes a vehículos teniendo en cuenta restricciones como capacidad del vehículo y demanda de los clientes.
6. Utilizar cualquier método de solución para TSP con el objetivo de obtener las rutas.

Es importante destacar que, en el proceso de resolución del GAP, la función objetivo es minimizar el costo de inserción para cada vehículo. Además, cada cliente debe ser asignado una sola vez a un solo vehículo y la demanda total para cada clúster es limitada por la capacidad del vehículo. Finalmente, se tendrán en cuenta las siguientes consideraciones: los clientes semillas serán aquellos con mayor demanda y la heurística para solucionar la segunda etapa como TSP será el Vecino más cercano por su naturaleza determinista [104].

Para la incorporación de esta heurística se realizan algunas modificaciones que se mencionan a continuación:

- Se implementa una clase que modela el comportamiento de la heurística para las siete variantes de VRP existentes (ver Figura 18).
- Se adiciona la Heurística de Asignación Generalizada de Fisher & Jaikumar como tipo de heurística que puede ser empleada para resolver VRP en la biblioteca (ver Figura 17).
- Se incorporan métodos auxiliares que resuelven el GAP en la primera etapa de la Heurística de Asignación Generalizada de Fisher & Jaikumar (ver Figura 18).

2.1.3 Otras modificaciones en BHCVRP

Luego del análisis detallado de las deficiencias de la versión anterior de la biblioteca se realizaron otras modificaciones en función de lograr mejorar su funcionamiento:

- Se convierte la clase abstracta *Family_Opt* en la interfaz *StepOptimization*, ya que no existen atributos y métodos que heredar. Por tanto, cada operador implementa su método de optimización (ver Figura 16).
- A la clase controladora del proceso se le adiciona la instancia correspondiente a la carga dinámica de las clases para los operadores de post-optimización usadas por el *Factory Method*. Se mantiene para las heurísticas, pero se elimina para las distancias (ver Figura 17).
- Se modifica la implementación de la clase *Heuristic* y sus subclases como parte de la incorporación del patrón de diseño *Template Method* (ver Figura 18).

- Se adiciona un paquete de excepciones propias para manejar las restricciones relacionadas con capacidad, distancia, velocidad, tiempo, entre otros elementos, atendiendo a las características de las diferentes heurísticas y variantes VRPs (ver Figura 19).
- Se adicionan las clases *LoadFile* y *ExportResult*, para la carga de instancias y la exportación de resultados respectivamente. Estas clases manejan diferentes formatos como *.data*, *.txt*, CSV y JSON.
- Se adiciona la clase *Tools* para manejar el ordenamiento de la lista de capacidades para la variante HFVRP, así como la incorporación de funcionalidades extras (ver Figura 20).
- Se adiciona una clase *OSRMService* para el cálculo de distancias reales.
- Se elimina el paquete *distances* (ver Figura 6), ya que se decide utilizar bibliotecas especializadas en el cálculo de distancias aproximadas (ver Tabla 12).
- Se elimina el paquete *assign* (ver Figura 7), ya que se decide utilizar la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) [96] con el objetivo de resolver la asignación de clientes a depósitos en la variante MDVRP eficientemente (ver Figura 21).

2.2 Nueva versión de BHCVRP

Como resultado de las modificaciones realizadas a la biblioteca, se muestran en la Figura 14, el diseño arquitectónico de la nueva versión de BHCVRP que es la base para su implementación en los lenguajes de programación Java y Python. Se utilizan colores diferentes para reflejar los cambios introducidos. En el caso de los paquetes que son nuevos en la arquitectura se emplea el color verde, el color morado para aquellos paquetes que sufrieron algún tipo de cambio y el color azul para los que se mantienen.

En primera instancia, la arquitectura de BHCVRP se basa en el patrón n- capas con un enfoque basado en reutilización [105]. A continuación, en la Figura 14, se muestra el diagrama para la nueva versión.

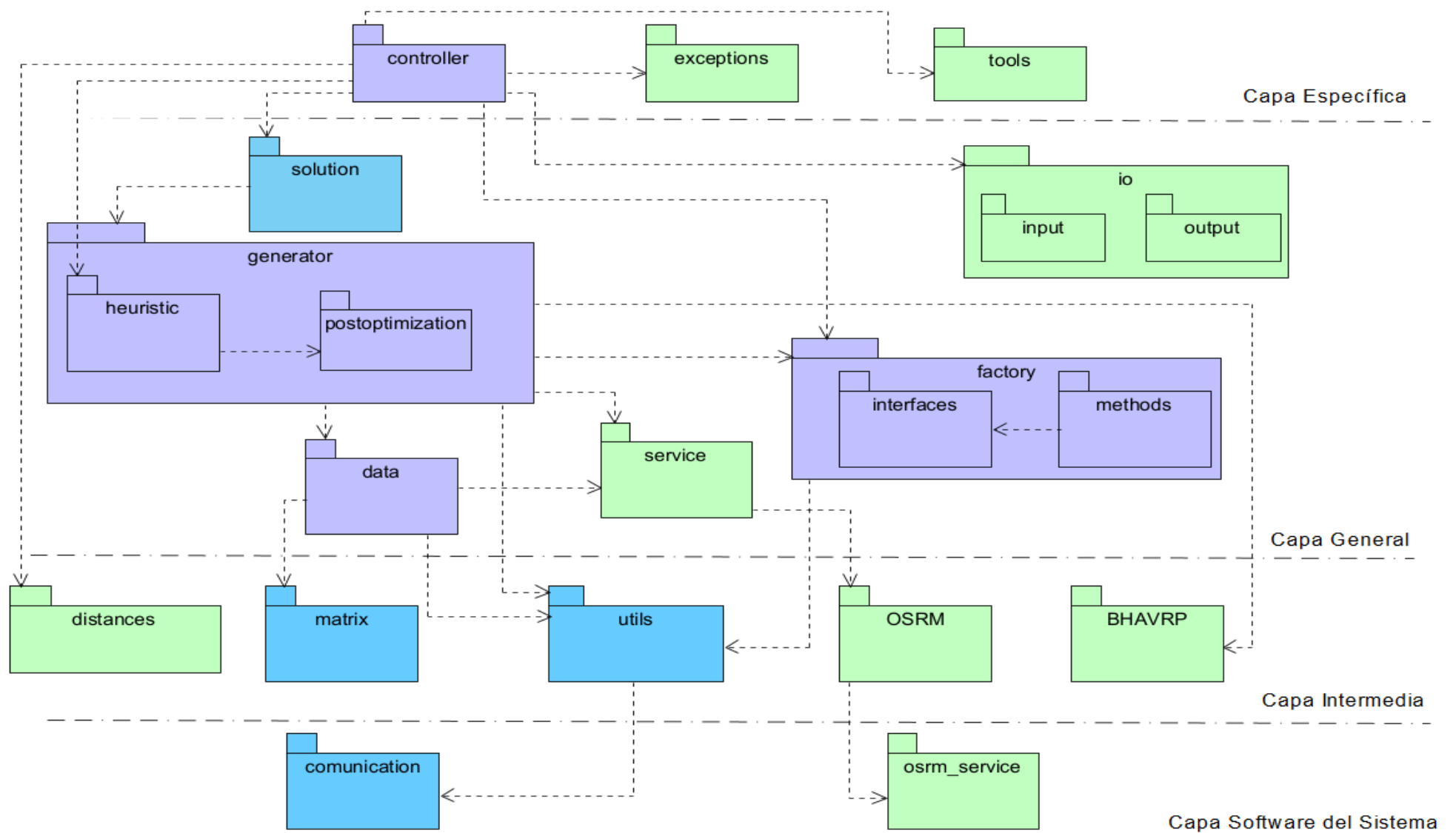


Figura 14: Patrón n-capas con enfoque basado en reutilización de la nueva versión de BHCVRP.

A continuación, se describen los paquetes y las funcionalidades generales que forman parte de la arquitectura, destacando su propósito y rol dentro del sistema.

2.2.1 Paquetes y funcionalidades en común

En la arquitectura de la biblioteca, los aspectos comunes están centralizados en la capa general, que actúa como un puente entre los componentes específicos y las funcionalidades de soporte. Esta organización permite reutilizar elementos clave, estandarizar procesos y facilitar la extensión de la biblioteca sin modificar el núcleo. La capa general está compuesta por paquetes necesarios para el funcionamiento de las heurísticas y la modelación de los datos del problema de planificación de rutas de vehículos que se quiere resolver. A continuación, para cada uno de los paquetes contenidos en esta capa se presenta su diagrama de clases correspondiente y una breve descripción de los elementos participantes.

El diagrama de clases del paquete *solution* se mantiene sin modificaciones (ver Figura 11). Su objetivo es contener las clases que modelan una solución del problema de planificación de rutas de vehículos.

En la Tabla 3 se describen las clases que contienen el paquete *solution*.

Tabla 3: Descripción de las clases que integran el paquete *solution*.

Clases	Descripción
<i>Route</i>	Clase que modela los datos de una ruta.
<i>RouteType</i>	Enumerado que indica el tipo de ruta para el caso de la variante TTRP.
<i>RouteTTRP</i>	Clase que modela los datos de una ruta para la variante TTRP.
<i>Solution</i>	Clase que modela la solución que se obtiene con una heurística de construcción, es decir, el orden en que serán visitados los clientes y el costo en distancia de los recorridos.

El diagrama de clases que se muestra en la Figura 15 refleja las clases del paquete *data*. Su objetivo es contener las clases que modelan un problema de planificación de rutas de vehículos. En la Tabla 4 se describen las clases que contienen el paquete *data*.

Tabla 4: Descripción de las clases que integran el paquete *data*.

Clases	Descripción
<i>Location</i>	Clase que modela la ubicación geográfica de un cliente, una parada o un depósito.
<i>Customer</i>	Clase que modela los datos de un cliente.
<i>CustomerType</i>	Enumerado que indica los tipos de clientes para la variante TTRP.
<i>CustomerTTRP</i>	Clase que modela los datos de un cliente para la variante TTRP.
<i>Depot</i>	Clase que modela los datos de un depósito.
<i>DepotMDVRP</i>	Clase que modela los datos de un depósito para la variante MDVRP.
<i>Fleet</i>	Clase que modela los datos de una flota de vehículos.
<i>FleetTTRP</i>	Clase que modela los datos de una flota de vehículos para la variante TTRP.
<i>BusStop</i>	Clase que modela los datos de una parada para la variante SBRP.
<i>TimeWindow</i>	Clase que modela las ventanas de tiempo para la variante VRPTW.
<i>ProblemType</i>	Contiene los datos según la variante, la matriz de costo y el tipo de problema.
<i>Problem</i>	Clase que controla el problema a solucionar. Contiene los datos según la variante, la matriz de costo y el tipo de problema.

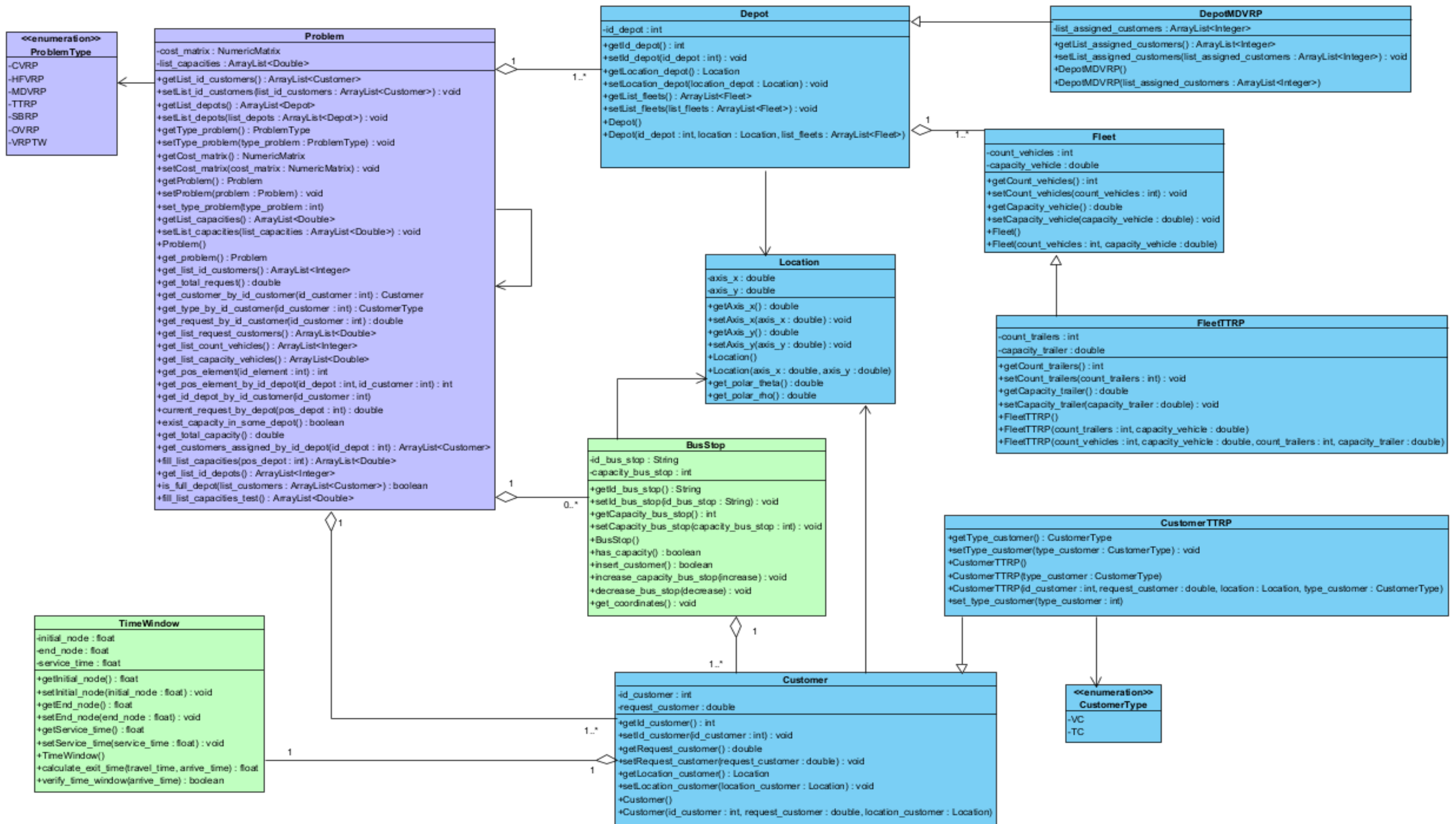


Figura 15: Diagrama de clases del paquete *data* en la nueva versión de BHCVRP.

El diagrama de clases que se muestra en la Figura 16 refleja las clases del paquete *postoptimization*. Su objetivo es contener los diferentes métodos para el uso de post-optimización. De color azul se encuentran las clases existentes, de color morado se encuentra *StepOptimization* que se convierte en interfaz y *Operator_3opt* que modifica su implementación.

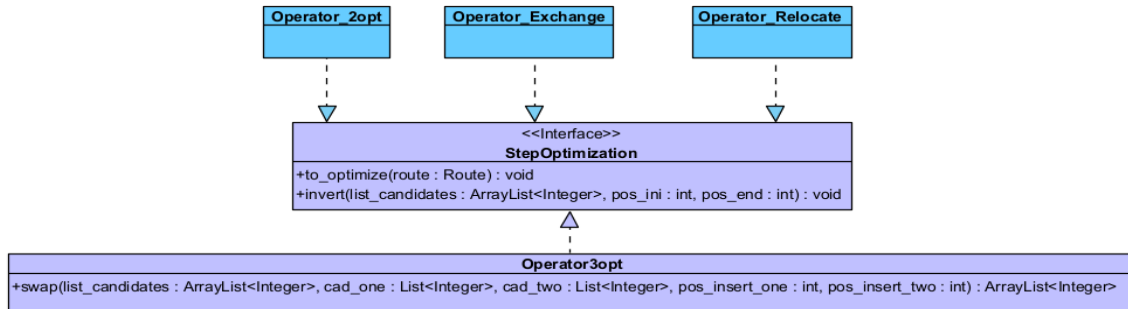


Figura 16: Diagrama de clases del paquete *postoptimization* en la nueva versión de BHCVRP.

En la Tabla 5 se describen las clases que contienen el paquete *postoptimization*.

Tabla 5: Descripción de las clases que integran el paquete *postoptimization*.

Clases	Descripción
<i>StepOptimization</i>	Interfaz que contiene los métodos de uso común para los métodos de postoptimización.
<i>Operator_3opt</i>	Clase que modela el operador para intercambiar tres nodos no adyacentes de una ruta [31].
<i>Operator_2opt</i>	Clase que modela el operador para intercambiar dos nodos no adyacentes de una ruta [31].
<i>Operator_Exchange</i>	Clase que modela el operador para intercambiar dos nodos de diferentes rutas [31].
<i>Operator_Relocate</i>	Clase que modela el operador para mover un nodo a una nueva posición en la ruta [31].

Por otra parte, el paquete *factory* está compuesto por los siguientes paquetes:

- *interfaces*: paquete que contiene las interfaces y los enumerados para la creación de heurísticas de construcción y métodos de post-optimización.

- *methods*: paquete que contiene las clases que implementan los métodos declarados en las interfaces del paquete *interfaces*.

La Figura 17 muestra el diagrama de clases con la relación que se establece entre los elementos de ambos paquetes. En las Tabla 6 y

Tabla 7 se describen los elementos participantes en el artefacto antes mencionado.

Tabla 6: Descripción de las clases que integran el paquete *interfaces*.

Clases	Descripción
<i>HeuristicType</i>	Enumerado que indican los tipos de heurísticas de construcción para resolver problemas de planificación de rutas de vehículos en la biblioteca.
<i>IFactoryHeuristic</i>	Interfaz que declara los métodos para la creación de heurísticas de construcción.
<i>OperatorType</i>	Enumerado que indican los tipos de métodos de post-optimización a utilizar en determinadas heurísticas de construcción en la biblioteca.
<i>IFactoryOperator</i>	Interfaz que declara los métodos para la creación de métodos de post-optimización.

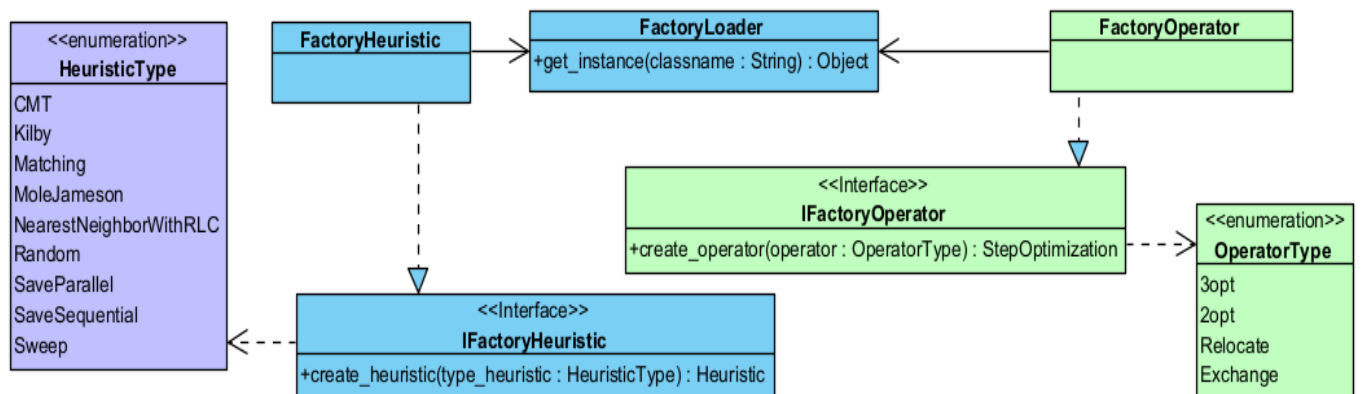


Figura 17: Diagrama de clases del paquete *factory* en la nueva versión de BHCVRP.

Tabla 7: Descripción de las clases que integran el paquete *methods*.

Clases	Descripción
<i>FactoryHeuristic</i>	Clase que implementa la interfaz <i>IFactoryHeuristic</i> y construye una heurística de construcción para resolver problemas planificación de rutas de vehículos.
<i>FactoryOperator</i>	Clase que implementa la interfaz <i>IFactoryOperator</i> y construye un método para usar pasos de post-optimización en problemas planificación de rutas de vehículos.
<i>FactoryLoader</i>	Clase que implementa un método genérico para la carga dinámica de cualquier objeto.

El paquete *heuristic* se encarga de presentar todas las heurísticas disponibles para la construcción de rutas. A continuación, en la Figura 18, se muestra su diagrama de clases correspondiente.

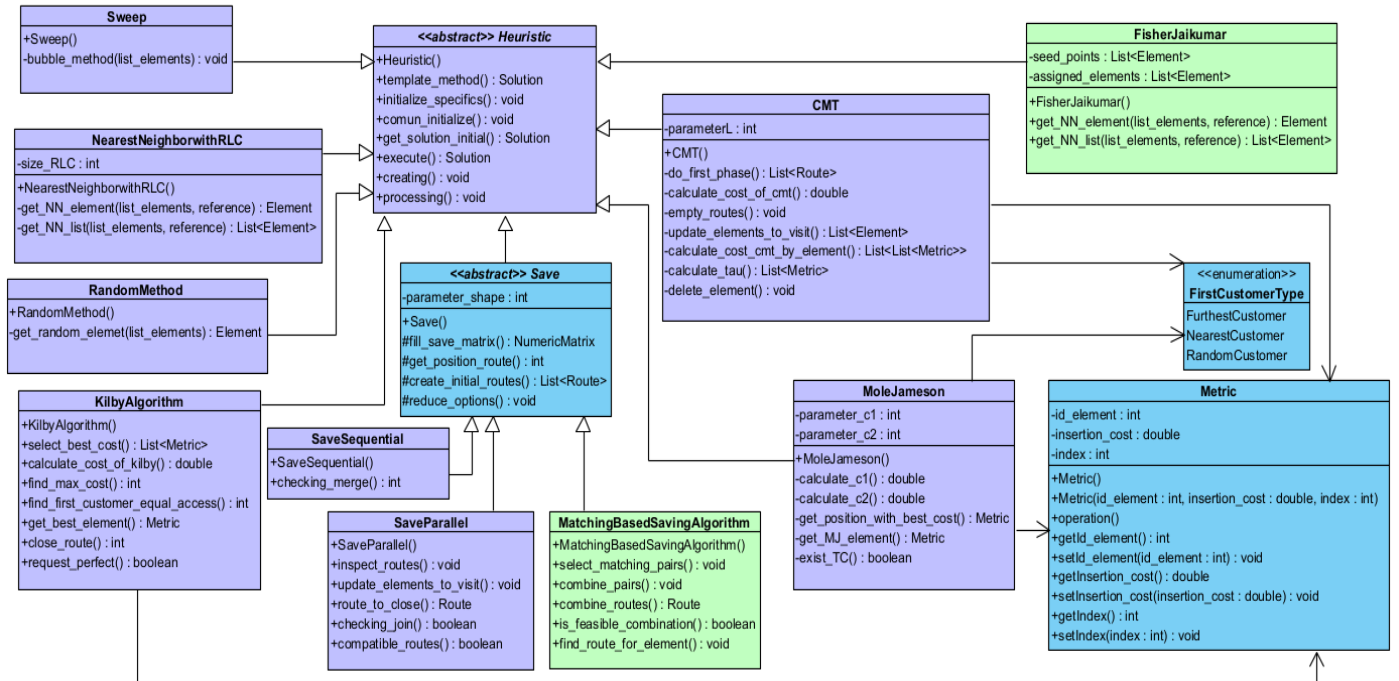


Figura 18: Diagrama de clases del paquete *heuristic* en la nueva versión de BHCVRP.

En la Tabla 8 se describen las clases que contienen el paquete *heuristic*.

Tabla 8: Descripción de las clases que integran el paquete *heuristic*.

Clases	Descripción
<i>Heuristic</i>	Clase abstracta que define el comportamiento de una heurística de construcción. De aquí heredan seis de las heurísticas implementadas: <i>CMT</i> , <i>KilbyAlgorithm</i> , <i>MoleJameson</i> , <i>FisherJaikumar</i> , <i>NearestNeighborWithRLC</i> , <i>RandomMethod</i> y <i>Sweep</i> .
<i>Save</i>	Clase abstracta que hereda de <i>Heuristic</i> , que modela el Algoritmo de Ahorros. De aquí herendan las heurísticas <i>SaveParallel</i> , <i>SaveSequential</i> y <i>MatchingBasedSavingAlgorithm</i> .
<i>FirstCustomerType</i>	Enumerado que indica la forma de seleccionar el primer cliente en las heurísticas <i>MoleJameson</i> y <i>CMT</i> .
<i>Metric</i>	Clase auxiliar utilizada para almacenar la información de los costos de inserción en las heurísticas de inserción.

El paquete *io* está compuesto por los siguientes paquetes:

- *input*: paquete encargado de cargar las instancias del problema que se desea resolver. Estos datos pueden variar según la variante VRP y comprende formatos como *.data*, *.txt* y JSON.
- *output*: paquete encargado de exportar los resultados obtenidos, es decir, el conjunto de rutas con mínimo costo en distancia para la heurística escogida. Este proceso puede ser realizado en los formatos *.txt*, CSV, XML y JSON.

Este paquete es completamente nuevo y posee únicamente las clases *LoadFile* y *ExportResult*, según su responsabilidad, *input* y *output* respectivamente. Estas clases contienen los métodos necesarios para procesar la información en diferentes formatos. Además, *LoadFile* contiene los elementos acordes para transferir los datos a las clases encargadas de modelar la variante VRP que se desea resolver, es decir, las pertenecientes al paquete *data*.

Por otra parte, *ExportResult* se relaciona con el paquete *controller* que recibe las rutas resultantes tras ejecutar la heurística seleccionada. Dicho fichero exportado contiene el nombre de la heurística ejecutada, la cantidad de rutas, el costo en distancia, el tiempo de ejecución en segundos y las respectivas rutas con su identificador y los clientes en el orden a visitar.

Por otra parte, en la capa específica se encuentran los siguientes paquetes:

- *exceptions*: paquete encargado de manejar excepciones propias del contexto VRP, tales como capacidad, distancia, tiempo, costo, entre otros.
- *tools*: paquete que contiene diferentes clases auxiliares, por ejemplo, para el ordenamiento de las capacidades de la flota de vehículo de los depósitos y las demandas de los clientes.
- *controller*: paquete responsable del control centralizado del sistema, donde se obtienen las posibles soluciones al problema y se aplica el patrón *Singleton* (ver Figura 13).

El diagrama de clases que se muestra en la Figura 19 refleja las clases del paquete *exceptions* y se utiliza el color verde para resaltar que es nuevo en su totalidad.

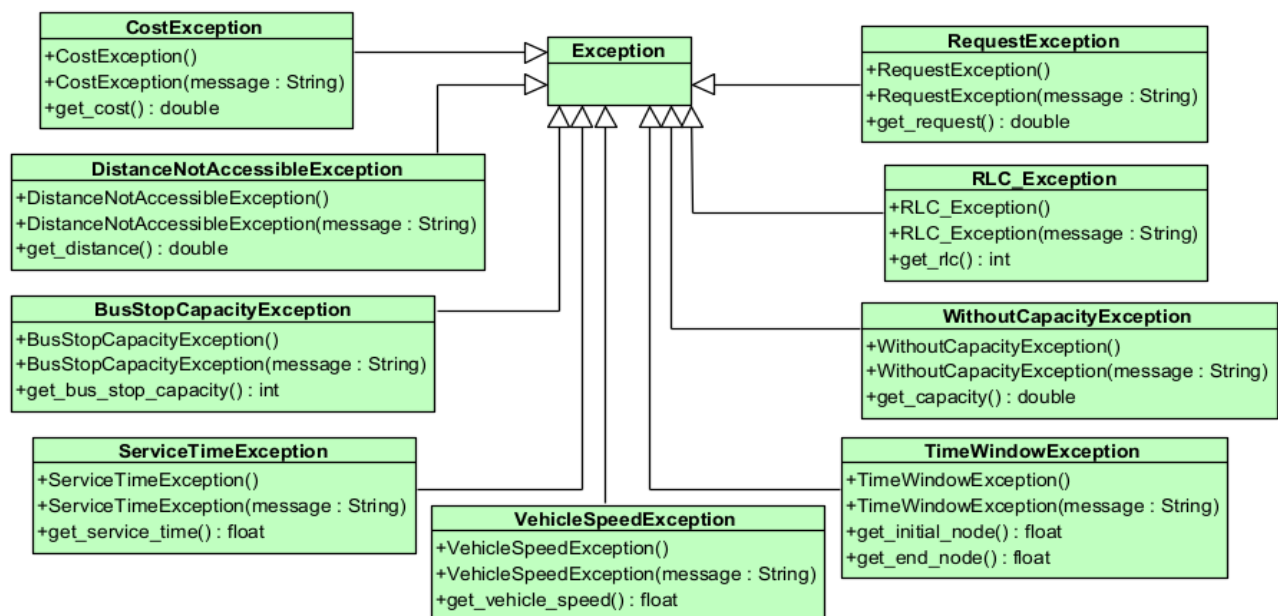


Figura 19: Diagrama de clases del paquete *exceptions* en la nueva versión de BHCVRP.

En la Tabla 9 se describen las clases que contienen el paquete *exceptions*.

Tabla 9: Descripción de las clases que integran el paquete *exceptions*.

Clases	Descripción
<i>BusStopCapacityException</i>	Clase que maneja la excepción relacionada con la capacidad de la parada en la variante SBRP, que debe ser mayor que cero.
<i>CostException</i>	Clase que maneja la excepción relacionada con el costo, que debe ser mayor que cero.
<i>DistanceNotAccessibleException</i>	Clase que maneja la excepción relacionada con un límite de distancia a recorrer por determinado vehículo.
<i>RequestException</i>	Clase que maneja la excepción relacionada con la demanda, que debe ser mayor que cero.
<i>RLC_Exception</i>	Clase que maneja la excepción relacionada con la heurística del Vecino más Cercano con Lista de Candidatos Restringidos, donde la lista no puede ser mayor que la mitad de la cantidad de clientes.
<i>ServiceTimeException</i>	Clase que maneja la excepción relacionada con el tiempo de servicio en la variante VRPTW, que debe ser mayor que cero.
<i>TimeWindowException</i>	Clase que maneja la excepción relacionada con las ventanas de tiempo (nodo inicial y nodo final) en la variante VRPTW, que debe ser mayor que cero y el nodo final no debe ser menor que el inicial.
<i>VehicleSpeedException</i>	Clase que maneja la excepción relacionada con la velocidad del vehículo en la variante VRPTW, que deber ser mayor que cero.
<i>WithoutCapacityException</i>	Clase que maneja la excepción relacionada con la capacidad de un vehículo.

El diagrama de clases que se muestra en la Figura 20 refleja las clases del paquete *tools*. Su objetivo es contener diferentes clases auxiliares, por ejemplo, para el ordenamiento, la carga de ficheros, entre otros.

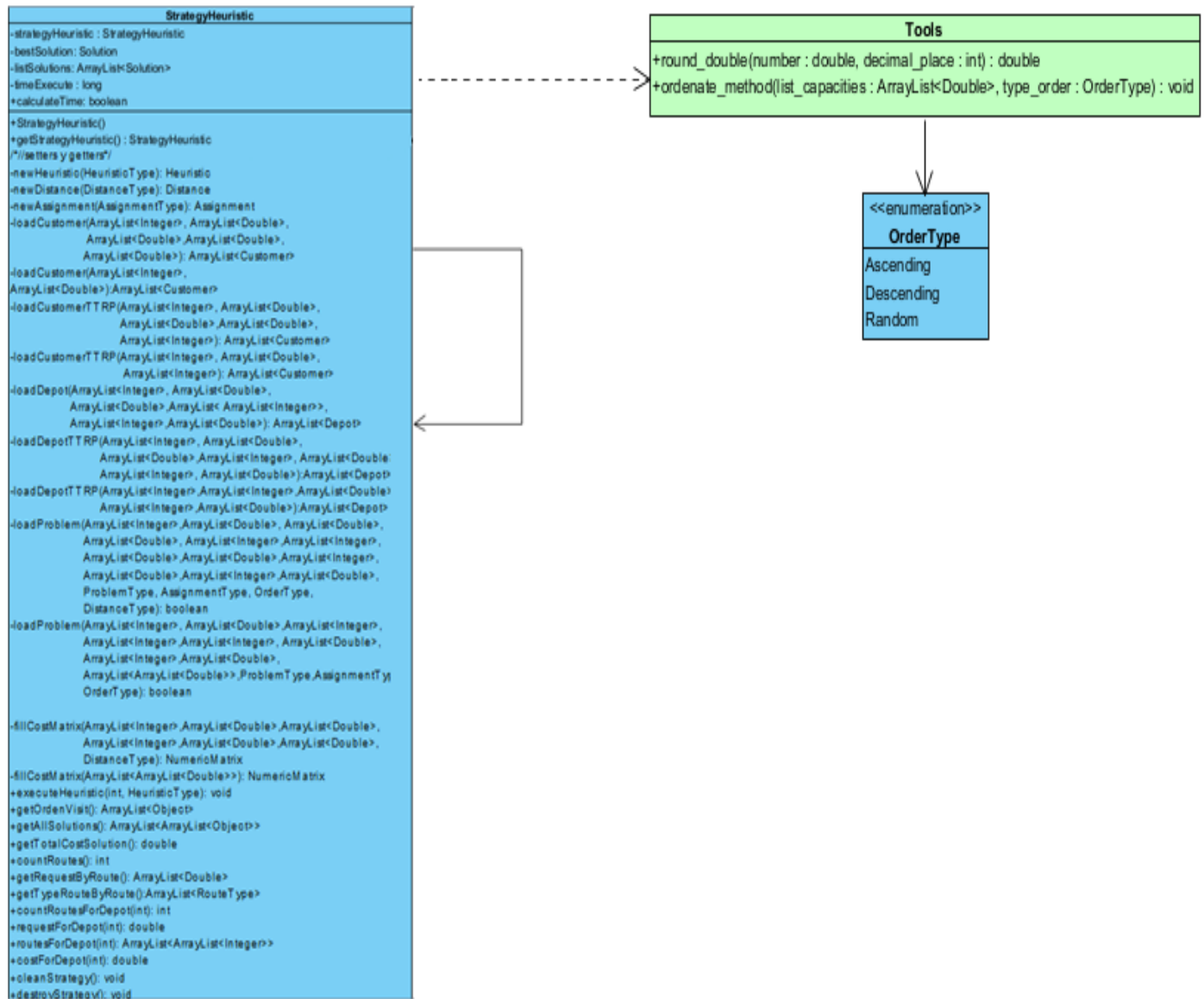


Figura 20: Diagrama de clases del paquete *tools* en la nueva versión de BHCVRP.

En la Tabla 10 se describen las clases que contienen el paquete *tools*.

Tabla 10: Descripción de las clases que integran el paquete *tools*.

Clases	Descripción
<i>OrderType</i>	Enumerado que contiene los distintos tipos de ordenamiento de la lista de capacidades para la variante HFVRP.
<i>Tools</i>	Clase auxiliar para realizar operaciones como redondeo y ordenamiento.

En la Tabla 11: Descripción de las clases que integran el paquete *controller*. se describe la clase que contiene el paquete *controller*.

Tabla 11: Descripción de las clases que integran el paquete *controller*.

Clases	Descripción
<i>StrategyHeuristic</i>	Clase que controla el proceso de construcción de soluciones a los problemas de planificación de rutas de vehículos en la biblioteca.

Finalmente, en la capa general, se incorpora un paquete llamado *service*. En este paquete se encuentra únicamente la clase *OSRMService*, que abstrae los detalles técnicos del cálculo de distancias reales mediante OSRM. Para ello, ofrece dos opciones para obtener las distancias: a través de un servicio local OSRM o mediante el consumo de una API en línea.

- La API en línea de OSRM proporciona acceso a servicios OSRM basados en la web para los entornos donde no es posible instalar el servicio local. Sin embargo, se identifican problemas asociados al abuso del consumo del servidor en línea, como límites en la cantidad de solicitudes permitidas y bloqueos temporales. Esto lleva a priorizar la configuración de un servicio local para garantizar disponibilidad constante.
- Para el servicio local de OSRM se utiliza una instancia local de *Open Source Routing Machine* instalada y configurada en un servidor local. Este enfoque reduce la dependencia de servicios externos y elimina problemas relacionados

con la latencia y los límites de uso impuestos por servidores remotos. Sin embargo, una desventaja es que se requiere la instalación y el inicio previo del servidor local antes de su uso, lo que puede aumentar la complejidad en la configuración y el mantenimiento del entorno.

La clase *OSRMService* permite alternar entre el uso del servicio local o la API en línea mediante configuraciones. Adicionalmente, la clase implementa el uso de la caché interna para almacenar las distancias calculadas entre pares de puntos, evitando cálculos redundantes en iteraciones posteriores. Con esta implementación se ahorran peticiones al servidor, reduciendo significativamente el tiempo total de ejecución del algoritmo. Al final de la construcción de la matriz de costos, la caché es limpiada mediante el método *clearDistanceCache*, lo que garantiza un uso eficiente de la memoria.

Por último, el paquete *osrm_service*, ubicado en la capa de software del sistema, es común para ambas implementaciones, cumpliendo la misma función de gestionar el servicio OSRM para los cálculos de distancias reales, ya sea a través de una instancia local o una API remota.

2.2.2 Particularidades de BHCVRP en Java y Python

En esta sección se detallan los paquetes de la capa intermedia de BHCVRP en sus versiones Java y Python, explicando su función y la manera en que contribuyen al funcionamiento general del sistema. A continuación, en la Tabla 12 se comparan los paquetes de la capa intermedia y la capa software del sistema.

Por otra parte, debido a la relevancia que posee la incorporación de BHAVRP [96] para la asignación de clientes a depósitos en la variante MDVRP, se presenta el siguiente diagrama de secuencia. En la Figura 21, se muestran las colaboraciones de los elementos internos de BHCVRP, respondiendo a las llamadas de los clientes cuando *typeProblem* es MDVRP, así como su interacción con BHAVRP.

Tabla 12: Particularidades de BHCVRP en Java y Python.

Capa	Paquete	Java	Python
Intermedia	<i>distances</i>	<i>DISTANCE_MEASURE</i> : proporciona el cálculo de distancias aproximadas como <i>Euclidean</i> , <i>Manhattan</i> , <i>Chebyshev</i> y <i>Haversine</i> .	<i>Scipy</i> : presenta herramientas matemáticas optimizadas para distancias aproximadas como <i>Euclidean</i> , <i>Manhattan</i> , <i>Chebyshev</i> y <i>Haversine</i> .
	<i>matrix</i>	<i>MATRIX_CLASS</i> : proporciona cálculos matriciales para la confección de la matriz de costos y matriz de tiempos.	<i>NumPy</i> : realiza cálculos matriciales y operaciones con arreglos. En este caso, se aplicaría para la construcción de la matriz de costo y de tiempos.
	<i>utils</i>	Bibliotecas Java (<i>java.util</i> , <i>java.lang</i>): presenta las clases, implementaciones y excepciones propias del lenguaje.	Bibliotecas Python (<i>time</i> , <i>typing</i> , <i>tqdm</i> , <i>abc</i>): manejo de tiempos, tipos de listas y clases base abstractas.
	BHAVRP	Se corresponde con la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) [96]. Su objetivo es manejar los métodos de asignación de clientes a depósitos.	Se utiliza BHAVRP [96] desarrollada en el lenguaje Java, a través de la librería <i>jpype</i> existente en Python.
	<i>OSRM</i>	<i>OSRMLib</i> : gestiona las solicitudes de distancias reales enviadas a OSRM mediante su API.	<i>osrm_py</i> : gestiona las solicitudes de distancias reales enviadas a OSRM mediante su API.
Software del sistema	<i>comunication</i>	<i>Java Virtual Machine</i> constituye la comunicación entre las diferentes librerías utilizadas en la tecnología Java. Esta se realiza principalmente a través de llamadas de métodos y paso de objetos dentro del mismo proceso JVM.	Protocolo de comunicación <i>buffer</i> . Este consiste en tener capacidad para manejar grandes volúmenes de datos y su flexibilidad de uso.

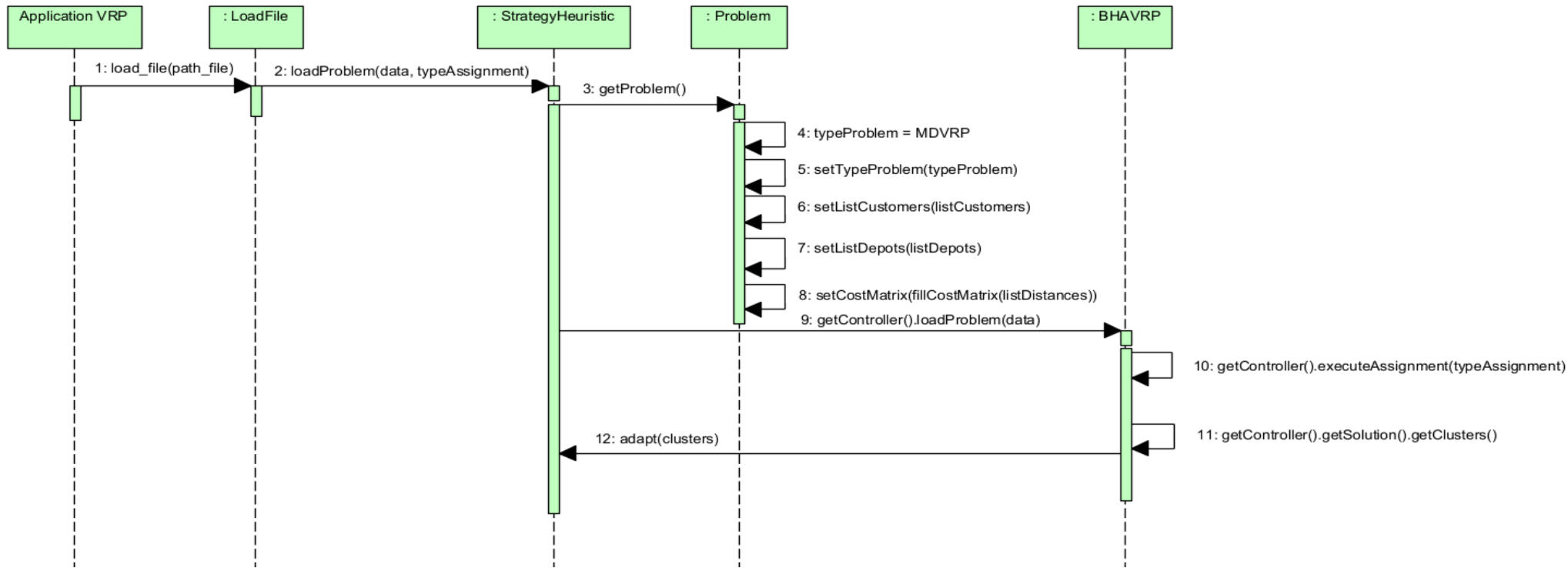


Figura 21: Diagrama de secuencia del consumo de BHAVRP.

2.3 Principios de diseño

Los principios de diseño de software son fundamentales para construir sistemas que sean robustos, escalables y fáciles de mantener. Estos principios actúan como cimientos en la arquitectura de software, guiando a los desarrolladores en la toma de decisiones de diseño que resultan en un software eficiente y confiable [106]. Para lograr este propósito, BHCVRP ha adoptado diversos principios de diseño que no sólo mejoran su estructura, sino que también optimizan su mantenimiento y extensión.

Los principios SOLID son un conjunto de directrices de diseño de software que buscan mejorar la calidad, flexibilidad y mantenibilidad del código en la programación orientada a objetos [106]. En la Tabla 13, se resume cómo BHCVRP cumple con la implementación de estos principios.

A partir de la información presentada en la Tabla 13, se puede observar que la biblioteca BHCVRP cumple con la mayoría de los principios SOLID, lo cual refleja una estructura de diseño robusta y bien organizada. En particular, los principios de Responsabilidad Única, *Open/Close*, Sustitución de Liskov y Segregación de Interfaces, se implementan casi en su totalidad, garantizando cohesión, extensibilidad y modularidad en el sistema.

Por otra parte, se identifican áreas de mejora en los principios Sustitución de Liskov e Inversión de Dependencias. En el primer caso, se pueden incorporar nuevas excepciones que puedan surgir según la variante VRP o algún criterio específico de alguna heurística. Por último, sería apropiado inyectar las dependencias en lugar de crearlas dentro de las clases.

Además de los principios SOLID, existen otros principios de diseño que son esenciales para lograr robustez y mantenibilidad en el software. Estos proporcionan un marco más amplio para la creación de código limpio, modular y adaptable [98, 107]. En la Tabla 14, se presenta cómo están implementados en BHCVRP otros principios de diseño que complementan los principios SOLID. Se explica brevemente cada principio y se ejemplifica su aplicación en la biblioteca, destacando cómo estos enfoques mejoran la estructura y el funcionamiento del componente.

Tabla 13: Principios de diseño SOLID que cumple BHCVRP.

Principio	Criterio de evaluación	Grado de cumplimiento
Responsabilidad Única	Cada clase tiene una única responsabilidad bien definida.	Total
	Los métodos de una clase están estrechamente relacionados con su propósito principal.	Total
<i>Open/Close</i>	Es posible extender el comportamiento de las clases sin modificar su código existente.	Total
	Se usan interfaces y clases abstractas para permitir la extensión.	Total
Sustitución de Liskov	Las subclases pueden ser utilizadas en lugar de sus clases base sin causar errores.	Total
	Las subclases no lanzan excepciones que no están en la firma de los métodos de la clase base.	Amplio
Segregación de Interfaces	Las interfaces son específicas y cohesivas, en lugar de ser grandes y generales.	Total
	Las clases que implementan interfaces no tienen métodos vacíos o sin implementar.	Total
Inversión de Dependencias	Se usan abstracciones (interfaces o clases abstractas) en lugar de implementaciones concretas.	Total
	Las dependencias se inyectan en lugar de ser creadas dentro de las clases.	Nulo

Tabla 14: Otros principios de diseño aplicados en BHCVRP.

Principio	Descripción	Ejemplo en BHCVRP
Ley de Deméter [98]	Limita el conocimiento de un objeto sobre otros, promoviendo un bajo acoplamiento.	Las clases operan de manera autónoma, llamando sólo a los métodos de su propia instancia, objetos pasados como argumentos o creados dentro de la clase, minimizando las interacciones innecesarias entre clases.
<i>Hollywood</i> [108]	Las clases de alto nivel deben llamar a las clases de bajo nivel, no al revés.	Las clases de alto nivel como <i>StrategyHeuristic</i> (ver Figura 13) y <i>Problem</i> (ver Figura 15), orquestan todo el procesamiento relacionado con las clases de bajo nivel con que interactúan directamente, favoreciendo un diseño limpio y desacoplado.
Encapsulación de la variabilidad [109]	Separa las partes del sistema que pueden cambiar de las que deben permanecer inalteradas.	En las heurísticas y pasos de post-optimización, donde el código que maneja estos paquetes está separado en clases específicas (ver Figura 17).
<i>Don't Repeat Yourself</i> (DRY) [110]	Evita la duplicación innecesaria de información o código. Toda pieza de conocimiento debe tener una única, precisa y completa representación dentro de un sistema.	En la clase <i>Heuristic</i> que brinda una implementación reutilizable para las clases hijas <i>RandomMethod</i> , <i>Sweep</i> y <i>NearestNeighborWithRLC</i> (ver Figura 18).

Los principios implementados en BHCVRP trabajan en sinergia para abordar la complejidad del diseño modular y la escalabilidad. La ley de Deméter y el principio de Bajo Acoplamiento reducen dependencias entre clases facilitando la mantenibilidad. El principio de *Hollywood* complementa la Segregación de Dependencias al promover la delegación y abstracción, asegurando que los módulos de bajo nivel no interfieran con la lógica central. Además, ejemplos como *IFactoryHeuristic* e *IFactoryOperator* (ver Figura 17), ilustran cómo las interfaces refuerzan la flexibilidad, logrando una biblioteca adaptable, cohesiva y alineada con buenas prácticas de diseño.

2.4 Patrones de diseño

En la versión anterior de la biblioteca de clases se garantizó la flexibilidad y reusabilidad del diseño con la utilización de patrones. Por lo que en esta nueva versión se decidió reutilizar la implementación de los patrones creacionales *Factory Method* y *Singleton* [98-100]. Es importante destacar que los patrones de diseño que se proponen emplear en los siguientes sub-epígrafes son comunes para BHCVRP en ambos lenguajes de programación, Java y Python. El patrón *Singleton* permanece sin modificaciones, sólo su adaptación en Python, mientras que en el patrón *Factory Method* se incorporan y eliminan elementos.

2.4.1 Patrón *Factory Method*

La nueva versión del componente está diseñada de forma robusta para permitir la incorporación de nuevas heurísticas de construcción y métodos para los pasos de post-optimización. Se implementa este patrón que permite realizar la carga dinámica de cualquier clase. En el caso de las distancias se eliminan del patrón porque se obtienen a partir de la capa intermedia propuesta en la arquitectura previamente mostrada.

En la Figura 17, se puede apreciar el diseño de clases que implementa este patrón para el caso de las heurísticas de construcción y los operadores. Además, en la Figura 22, se presenta el diagrama de secuencia asociado al funcionamiento del mecanismo de diseño para la carga dinámica utilizando el patrón *Factory Method*.

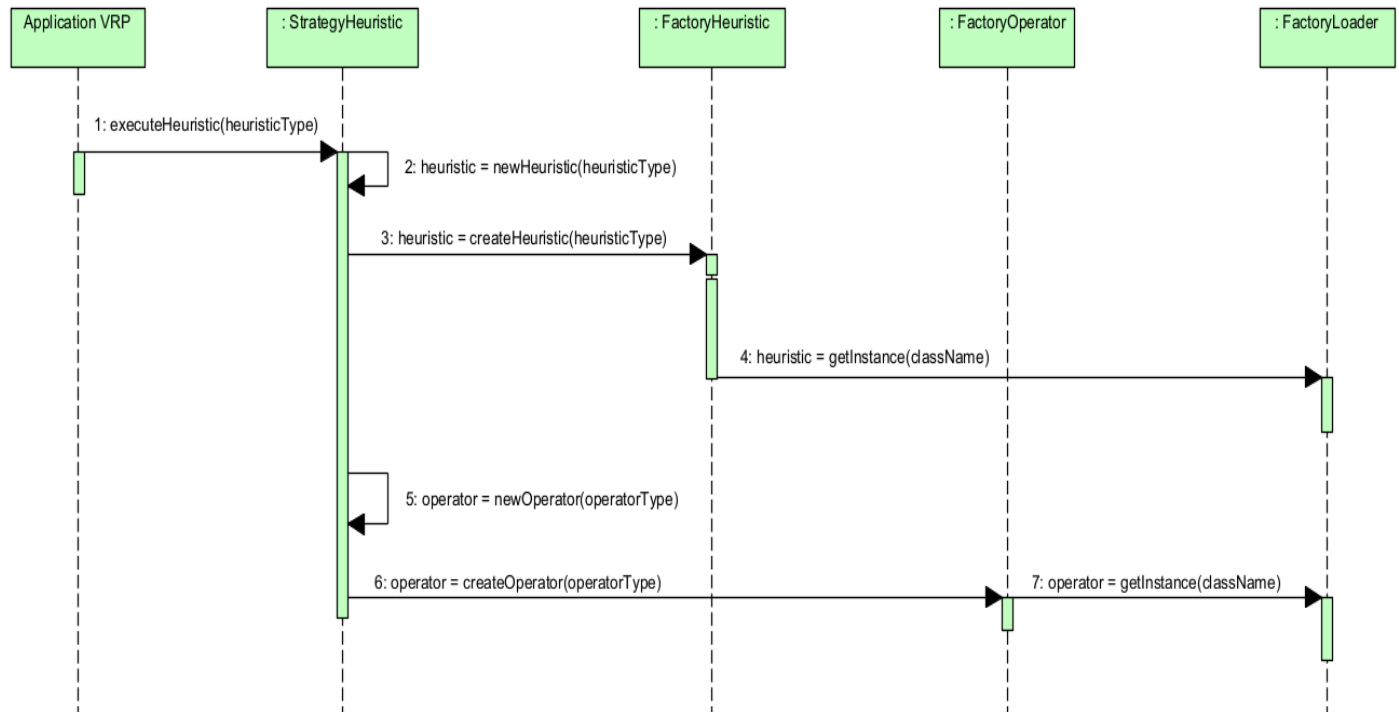


Figura 22: Diagrama de secuencia del patrón *Factory Method* para las heurísticas de construcción y los operadores de post-optimización.

2.4.2 Patrón *Template Method*

El patrón de diseño *Template Method* es un patrón estructural que proporciona una plantilla para la operación en lugar de implementarla completamente. Este patrón define el esqueleto de un algoritmo en una operación, delegando algunos pasos a las subclases. Una de las principales ventajas del patrón *Template Method* es que permite cambiar la secuencia de pasos de un algoritmo sin modificar su estructura. Esto es especialmente útil cuando diferentes variantes de un algoritmo comparten la misma estructura, pero difieren en sus pasos. Al definir estos pasos en métodos individuales, las subclases pueden sobrescribir esos métodos para personalizar el comportamiento del algoritmo según sea necesario. Esto mejora la cohesión y reduce la duplicación de código, ya que la lógica común se encapsula en la clase base [98].

En BHCVRP, las heurísticas de construcción siguen un conjunto de pasos estructurados. El proceso comienza con la preparación de los elementos necesarios para la ejecución de la heurística en cuestión y las particularidades según la variante VRP que se desea resolver. Luego, se procede a crear las rutas, teniendo en cuenta los criterios a partir del

tipo de heurística. Posteriormente, se realiza un procesamiento de dichas rutas teniendo en cuenta el comportamiento de la heurística y verificando que se cumplan las restricciones del problema (por ejemplo, demandas de los clientes, capacidades máximas de los vehículos, paradas y depósitos, entre otras). Por último, se ejecuta dicha heurística y se construye la solución que comprende un conjunto de rutas. En la mayoría de los casos se contemplan varias ejecuciones, debido a la naturaleza aleatoria de las heurísticas.

En la Figura 24, se presenta el diagrama de actividades de la Heurística de Inserción de Kilby, donde se pueden observar los pasos fundamentales identificados en el proceso de obtención del conjunto de rutas. Este diagrama sirve como base para establecer la estructura general del proceso en BHCVRP.

Para generalizar el comportamiento común a todas las heurísticas de construcción, se propone en la Figura 23, un diagrama genérico, que encapsula los cuatro pasos previamente establecidos. Este diagrama define una estructura estándar que facilita la comprensión y organización del flujo de trabajo en BHCVRP.

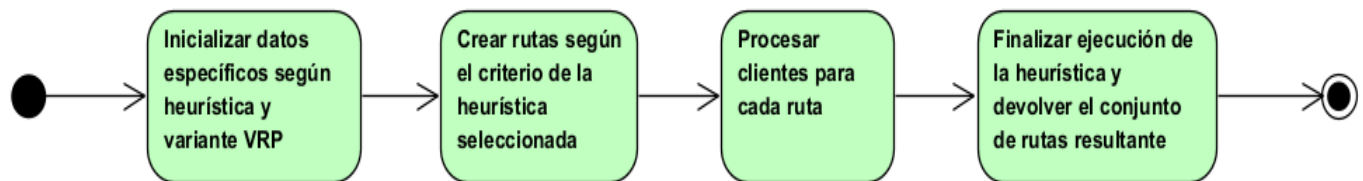


Figura 23: Diagrama genérico común para todas las heurísticas.

Por tanto, este patrón se aplica para manejar los comportamientos de las heurísticas y solucionar la deficiencia relacionada con flexibilidad y reutilización. Se tiene una clase abstracta *Heuristic*, que contiene la configuración de los métodos necesarios para sus respectivas clases hijas. La función *comun_initialize()* agrupa todos los parámetros que se repiten, mientras que *initialize_specifics()* permite a cada clase definir el resto de los parámetros que más necesita. Se decide hacer énfasis en esta fase, ya que existen heurísticas como Algoritmo de Barrido, Vecino más Cercano con Lista de Candidatos Restringidos y el método aleatorio, que funcionan del mismo modo, sólo escogen el cliente de manera distinta. Sin embargo, las heurísticas de ahorros necesitan una matriz asociada y las heurísticas de inserción necesitan determinados elementos.

Los métodos *creating()*, *processing()* y *execute()* permiten crear las rutas, procesarlas de manera distinta según los intereses de cada heurística en particular y ejecutarlas para obtener una solución respectivamente. Por último, el método *get_solution_initial()* obtiene la solución esperada y *template_method()* define el esquema general de trabajo de todas las heurísticas. En la Figura 18, se muestra el diagrama de clases relacionado con lo descrito anteriormente.

En la implementación específica de la Heurística de Inserción de Kilby, estos métodos son sobrescritos para proporcionar un comportamiento específico, mientras siguen el marco general definido por el patrón *Template Method*. Esta estructura demuestra cómo este patrón permite definir un flujo de trabajo general, dejando a las subclases la responsabilidad de implementar las partes específicas, respetando así principios de diseño como el *Open/Close* [106], así como el fomento de la reutilización y la extensibilidad.

2.5 Conclusiones parciales

Tras el proceso de rediseño de la arquitectura de la biblioteca de clases y con la adaptación de nuevas heurísticas de construcción en diferentes variantes de VRP se obtienen las siguientes conclusiones:

- Se realizan las modificaciones en la arquitectura de BHCVRP para la incorporación de nuevas heurísticas de construcción y variantes VRP, cubriendo

las deficiencias planteadas en la versión anterior. Además, se tienen en cuenta la arquitectura n-capas con enfoque basado en reutilización.

- Se incorporan mejoras para el funcionamiento de la biblioteca como son el tratamiento de excepciones propias relacionadas con la distancia, capacidad de los vehículos, velocidad y tiempo de ejecución, así como la aplicación de principios de diseño como SOLID, Ley de Deméter, Encapsulación de la Variabilidad, *Hollywood* y *Don't Repeat Yourself*.
- Se incorpora la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) para la fase de asignación de clientes a depósitos en la variante MDVRP.
- Se incorporan servicios OSRM para el cálculo de distancias reales.
- Se propone la primera versión de la biblioteca de clases que cubre los lenguajes de programación Python y Java.
- Se incorporan dos nuevas adaptaciones de las heurísticas de construcción: el Algoritmo de Ahorro basado en *Matching* y la Heurística de Asignación Generalizada de Fisher & Jaikumar.
- Se incorporan tres nuevas variantes VRP: el Problema de Planificación de Rutas de Autobuses Escolares, el Problema de Planificación de Rutas de Vehículos Abiertos y el Problema de Planificación de Rutas de Vehículos con Ventanas de Tiempo.
- En el diseño de la biblioteca se incorpora el uso del patrón *Template Method* que promueve la flexibilidad de las heurísticas de construcción implementadas.
- Se incorpora el formato JSON para la carga de datos y los formatos JSON, XML y CSV para la exportación de resultados.

Capítulo 3: Análisis y validación de BHCVRP

En este capítulo se presentan los resultados obtenidos con las diez heurísticas implementadas en la nueva versión de BHCVRP para siete variantes de problemas de planificación de rutas de vehículos. Inicialmente, se explican los diferentes tipos de experimentos a realizar, las características de las instancias y las configuraciones necesarias para las heurísticas y variantes VRP. Por último, se analizan los resultados alcanzados en los diferentes experimentos.

3.1 Descripción de los experimentos

Para validar el funcionamiento de la nueva versión de BHCVRP, se decide realizar los siguientes experimentos:

1. Experimento 1: consiste en ejecutar las diez heurísticas implementadas en Python, para analizar su funcionamiento en las siete variantes VRP.
2. Experimento 2: se realiza una comparación de los resultados actuales obtenidos, tanto en Java como en Python, con los resultados de la versión anterior de la biblioteca en Java, teniendo en cuenta siete heurísticas y cuatro variantes VRP.
3. Experimento 3: se realiza una comparación de los mejores resultados obtenidos en BHCVRP con los resultados de OR-Tools para las variantes CVRP, HFVRP y VRPTW.

3.1.1 Descripción de las instancias

Para la ejecución de los experimentos se seleccionan para siete variantes de planificación de rutas de vehículos instancias de la literatura [86, 88, 111]. En las Tabla 34, Tabla 35, Tabla 36, Tabla 37, Tabla 38 y Tabla 39 del Anexo 1, se muestran las instancias empleadas en las variantes CVRP, OVRP, MDVRP, HFVRP, TTRP, VRPTW y SBRP, respectivamente. En el resto del capítulo, se muestran algunas otras tablas, que se complementan con los detalles que están en el Anexo 2.

Para la variante CVRP se seleccionan instancias del autor Cordeau disponibles en [111], con ligeras transformaciones. Para cada instancia se mantienen los clientes, cantidad de vehículos y sus capacidades con sus datos originales, sólo se adaptan para trabajar con un único depósito. Para la experimentación se utilizan un total de ocho instancias de

prueba. En la Tabla 34 del Anexo 1, se presenta para cada instancia del problema: el total de clientes, el total de vehículos y la capacidad de estos vehículos. Las cantidades de clientes de estas instancias oscilan entre 50 y 600. La cantidad de vehículos varía en la mayoría de las instancias y sus capacidades oscilan entre 200 y 500. Para la variante OVRP, no se disponen de instancias de pruebas en la literatura, por lo que se toman las mismas instancias seleccionadas para CVRP. Esto se debe a que la única diferencia entre estas variantes es el retorno o no al depósito.

En el caso particular de la variante HFVRP, no están disponibles instancias de pruebas en la literatura y se decide utilizar las instancias seleccionadas para la variante CVRP. Para cada instancia se mantienen los clientes y cantidad de vehículos con sus datos originales, mientras que sus capacidades se distribuyen de manera aleatoria entre los vehículos, siempre respetando la capacidad total original. Con este procedimiento se obtiene una flota de vehículos con varios tipos de vehículos diferentes según su capacidad. En la Tabla 36 del Anexo 1, se muestra en la segunda y tercera columna el total de clientes y vehículos respectivamente. Luego, en las restantes columnas se presentan los datos para cada tipo de vehículo definido (total y capacidad). Al igual que en la variante CVRP, se dispone de ocho instancias para realizar los experimentos. Estas instancias tienen las mismas características en cuanto a la cantidad de clientes, vehículos y la capacidad total.

Para la variante MDVRP se seleccionan instancias del autor Cordeau disponibles en [111]. Para los experimentos se utilizan un total de ocho instancias de prueba. En la Tabla 35 del Anexo 1, se presenta para cada instancia del problema: el total de clientes, el total de depósitos, las cantidades de vehículos por depósito y las capacidades de los vehículos. Las cantidades de clientes de estas instancias oscilan entre 50 y 360. Para el caso de los depósitos, la máxima cantidad es nueve y la mínima dos. Además, la cantidad de vehículos y sus capacidades varía en la mayoría de las instancias.

Para la variante TTRP se emplean ocho instancias de referencias de la literatura reportadas en [88] como se muestra en la Tabla 37 del Anexo 1. Estas instancias TTRP provienen de siete problemas VRP de prueba [86]. Las cantidades de clientes de estas

instancias oscilan entre 50 y 199. Las capacidades de los camiones toman valores entre 100 y 150. Para el caso de los remolques la capacidad siempre es 100.

Para la variante VRPTW se emplean ocho instancias disponibles en [112]. En la Tabla 38 del Anexo 1, se presenta para cada instancia del problema: el total de clientes, cantidad de vehículos y sus capacidades, así como el rango de intervalo en ventanas de tiempo. Las cantidades de clientes oscilan entre 200 y 600. Las cantidades de vehículos entre 50 y 150, y sus capacidades en todos los casos son de 200. El rango de intervalo en ventanas de tiempo oscila entre 1 y 1609. Además, se establecen como parámetros fijos la máxima velocidad de los vehículos a 83.33 km/h y el máximo tiempo en que un vehículo debe completar su ruta en 1500 minutos.

Por último, para la variante SBRP se seleccionan ocho instancias disponibles en [113], con una variación en la cantidad de pasajeros entre 50 y 1600. Se tienen entre 5 y 80 vehículos, con capacidades entre 25 y 50. Además, se tiene la cantidad de paradas que oscila entre las 10 y 480. En cuanto a las restricciones, se tiene que la capacidad máxima de las paradas es homogénea con un límite de 15 personas por parada y la máxima distancia que los pasajeros pueden caminar varía entre 5 y 244 unidades. En la Tabla 39 del Anexo 1 se presentan los detalles para cada instancia.

3.1.2 Configuración de los experimentos

Todas las heurísticas adaptadas a las distintas variantes fueron implementadas en el lenguaje de programación Python versión 3.12.0, utilizando el IDE de programación PyCharm 2024.1.3 (*Community Edition*) [114]. Para el caso de las versiones anterior [38] y actual en Java, se utilizó el IDE Apache NetBeans 16 [115] y JDK 11. Los experimentos se ejecutaron en un ordenador con un procesador Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz 2.11 GHz, 20 GB de RAM y sistema operativo de 64 bits, versión Windows 10. En todos los casos se realizan 20 ejecuciones de las heurísticas seleccionadas debido a que no son determinísticas y sus resultados varían, por lo que es necesario realizar pruebas estadísticas.

Heurísticas de construcción y parámetros

Se utilizan las diez heurísticas de construcción implementadas en BHCVRP. Algunas de estas heurísticas requieren de parámetros en su ejecución. A continuación, se presenta la configuración de estos parámetros para las heurísticas que lo requieren:

1. Algoritmo Aleatorio (*Random*): este algoritmo construye una solución realizando sucesivas inserciones de los clientes de forma aleatoria.
2. Heurística del Vecino más Cercano con Lista de Candidatos Restringidos (*NN*): propone la construcción de una solución insertando al cliente más cercano dentro de la lista de candidatos restringida al último cliente ya insertado. Posee como parámetro el tamaño de la lista de candidatos restringidos (*size_RLC*) con valor 3.
3. Algoritmo de Ahorros en su versión secuencial (*SaveSeq*): este algoritmo plantea que en una solución dos rutas diferentes pueden ser combinadas formando una nueva ruta. Esta versión del algoritmo propone la construcción de las rutas una a una. Utiliza el parámetro λ (*parameter_shape*) con valor 1 para la construcción de la matriz de ahorros.
4. Algoritmo de Ahorros en su versión paralela (*SaveParall*): esta otra versión del algoritmo tiene la misma filosofía, pero trabaja sobre todas las rutas de forma simultánea. Utiliza el parámetro λ (*parameter_shape*) con valor 1 para la construcción de la matriz de ahorros.
5. Algoritmo de Ahorros basado en *Matching* (*SaveMatch*): esta heurística constituye otra alternativa del algoritmo, pero resuelve un problema de *matching* de peso máximo teniendo en cuenta los pesos de las aristas de cada nodo perteneciente a la lista de clientes en las rutas. Por último, verifica si la unión de dichas rutas es factible, teniendo en cuenta la posible afectación en el futuro. Utiliza el parámetro λ (*parameter_shape*) con valor 1 para la construcción de la matriz de ahorros.
6. Heurística de Inserción Secuencial de Mole & Jameson (*MJ*): esta heurística de inserción utiliza dos medidas para decidir el próximo cliente a insertar en la solución. Utiliza los parámetros λ (*c1*) con valor 1 que representa el peso del costo

directo entre los clientes en la ruta, μ (c2) con valor 1 que representa el peso del costo desde el depósito hasta el cliente que se está considerando insertar y la forma de seleccionar el primer cliente a insertar en la ruta (*select_customer_MJ_type*) con valor 1 que significa el más cercano al depósito.

7. Heurística de Barrido (*Sweep*): esta heurística trabaja en dos fases, primero forma los grupos de clientes y luego construyen las rutas para cada grupo.
8. Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (*CMT*): esta heurística de inserción opera en dos fases. En la primera fase se determina la cantidad de rutas a utilizar, junto con un cliente para inicializar cada ruta. En la segunda fase se crean dichas rutas y se inserta el resto de los clientes en ellas según el costo de inserción. Utiliza el parámetro λ (*parameter_L*) con valor 1, que controla el peso del costo de viajar desde el cliente j hasta el cliente i en el cálculo del costo de inserción.
9. Algoritmo de Kilby (*Kilby*): esta heurística de inserción construye tantas rutas como vehículos y se añade el cliente que más cercano esté en cada momento según su costo de inserción.
10. Heurística de Asignación Generalizada de Fisher y Jaikumar (*FJ*): esta heurística trabaja en dos fases, primero asigna clientes a vehículos teniendo en cuenta el criterio de poseer mayor demanda y luego construye las rutas para cada grupo siguiendo los pasos de la heurística del Vecino más Cercano.

Variantes VRP

En los casos de las variantes existentes, se requiere de distintos tipos de configuración para realizar su resolución. En el caso de MDVRP es necesario realizar la asignación de los clientes a cada depósito a priori de la ejecución de la heurística. Para la variante HFVRP se debe organizar la lista de las capacidades para lograr obtener una solución que respete las capacidades de los vehículos de la flota y que sea comprensible para el usuario. BHCVRP contempla varias opciones para las configuraciones de cada una de estas variantes VRP.

A continuación, se muestran los métodos de asignación para MDVRP que utiliza la biblioteca BHAVRP [96]: *BestCyclicAssignment*, *BestNearest*, *CoefficientPropagation*, *K_Means*, *NearestByCustomer*, *NearestByDepot*, *PAM*, *Parallel*, *RandomByElement*, *RandomNearestByCustomer*, *RandomNearestByDepot*, *RandomSequentialCyclic*, *RandomSequentialNearestByDepot*, *SequentialCyclic*, *SequentialNearestByDepot*, *Simplified*, *Sweep*, *ThreeCriteriaClustering* y *UPGMC*. Por último, los métodos de ordenamiento para HFVRP son: *Ascending*, *Descending* y *Random*.

Pruebas estadísticas no paramétricas

Las técnicas estadísticas que involucran la estimación de parámetros, la construcción de intervalos de confianza y la realización de pruebas de hipótesis se agrupan bajo el término de estadística paramétrica. Estas técnicas requieren la definición previa de una forma específica para la distribución de la variable aleatoria y para los estadísticos derivados de los datos. Dentro de la estadística paramétrica, se presupone que la población de la que se extraen muestras es normal o presenta una similitud notable a la normalidad. Esta suposición es crucial para garantizar la validez de las pruebas de hipótesis. Sin embargo, en numerosos escenarios, no es posible identificar la distribución primigenia ni la distribución de los estadísticos, lo que significa que, en lugar de estimar parámetros, se buscan comparar distribuciones. Este enfoque se denomina estadística no paramétrica [116].

Para evaluar los resultados obtenidos, se diseña una prueba de hipótesis con el fin de determinar si existen diferencias en el comportamiento de los algoritmos en general:

H_0 : No existen diferencias significativas entre los algoritmos.

H_1 : Existen diferencias significativas entre los algoritmos.

Todas las pruebas de hipótesis se realizan con un nivel de significancia de $\alpha = 0.05$.

Para examinar el comportamiento general de los algoritmos y analizar el rendimiento de los algoritmos en la versión actual, tanto en Java como en Python, y la versión anterior en Java, se emplea la prueba estadística no paramétrica de Friedman [117]. La prueba de Friedman se utiliza para el análisis de varianza de dos clasificaciones por rango, en situaciones donde se cuentan con k muestras correlacionadas y k problemas distintos.

Esta prueba implica calcular un *ranking* para los resultados generados por cada uno de los k algoritmos para cada problema, asignando el *ranking* 1 al mejor resultado y el *ranking* k al peor. La hipótesis nula de esta prueba asume que los resultados producidos por los algoritmos son equivalentes, lo que implicaría que sus *rankings* son similares [117].

3.2 Análisis de los resultados

Esta sección está dividida en tres partes, una por cada tipo de experimento realizado. En la primera sección se analiza el comportamiento de cada una de las heurísticas de construcción para cada variante del problema de planificación de rutas de vehículos. Se muestran gráficas para un mejor análisis de los resultados obtenidos respecto al costo de las soluciones y el tiempo de ejecución en las siete variantes. Además, se demuestra con la prueba estadística no paramétrica de Friedman [117], la validez de los resultados obtenidos.

En la segunda sección se analizan los resultados obtenidos de comparar las siete heurísticas de construcción de la versión anterior en Java de la biblioteca con estas mismas heurísticas en la versión actual en Java y en Python para las cuatro variantes VRP. Se compara de forma detallada los resultados en cuanto a costo en distancia euclidiana y tiempo de ejecución en segundos. La prueba de Friedman [117] es utilizada para detectar diferencias significativas entre los algoritmos de cada versión de la biblioteca.

Por último, en la tercera parte se analizan los mejores resultados obtenidos en la versión actual desarrollada en Python con OR-Tools, para las variantes CVRP, HFVRP y VRPTW. Se tiene en cuenta el costo en distancia euclidiana y el tiempo de ejecución en segundos.

3.2.1 Resultados obtenidos con BHCVRP en Python

Después de realizados los experimentos se analizan los resultados y se muestra un resumen por cada variante VRP. En las Tabla 15, Tabla 16, Tabla 17, Tabla 18, Tabla 19, Tabla 20, Tabla 21, Tabla 22, Tabla 23, Tabla 24, Tabla 25, Tabla 26, Tabla 27 y Tabla 28, que muestran el resumen de los resultados por cada problema se resalta en color azul, la mejor solución obtenida y el menor tiempo de ejecución. Para los valores

de costo en distancia euclidiana, se consideran dos cifras significativas y para los tiempos de ejecución se admiten cuatro cifras significativas. Se coloca entre paréntesis la cantidad de rutas obtenidas para las mejores soluciones alcanzadas.

Análisis de los resultados para la variante CVRP

En las Tabla 15 y Tabla 16 se resumen los resultados obtenidos sobre los ocho problemas de la variante CVRP seleccionados previamente.

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron en la heurística *SaveParall*. Sin embargo, también se obtienen buenos resultados para las heurísticas *SaveSeq* y *MJ*.
- Las soluciones más costosas obtenidas con las heurísticas *Sweep* y *NN* son superadas por los resultados de la heurística *Random*, por lo que se puede apreciar que esta heurística obtiene los peores resultados.
- Las heurísticas *CMT*, *SaveMatch*, *FJ* y *Kilby* tienen un comportamiento regular, pues no quedan entre las heurísticas de mejores resultados, pero tampoco entre las de peores resultados.
- Las heurísticas *SaveParall*, *SaveMatch* y *Kilby* son deterministas, pero al utilizar el operador 3-opt como método de post-optimización dejan de serlo.
- La heurística *MJ* obtiene buenos resultados, pero también es el que mayor tiempo emplea en su ejecución. Aunque es importante destacar que, mientras aumenta la cantidad de clientes, las que obtienen mayores tiempos son *SaveParall*, *SaveMatch* y *Kilby*. Sin embargo, *Random* obtiene los peores resultados en las soluciones, pero el menor tiempo de ejecución en todos los casos.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución. Además, es necesario destacar que obtiene tiempos de ejecución pequeños y para las instancias que poseen pocos vehículos se comporta aceptable. Sin embargo, para grandes cantidades como 50, 100 y 150 vehículos resulta ineficiente y obtiene los peores resultados.

Tabla 15: Resumen de los resultados en las ocho instancias CVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1470.02 ₍₄₎	1537.38	0.0019	1326.55 ₍₄₎	1474.08	0.1186	558.89 ₍₄₎	576.60	0.5448	561.46 ₍₄₎	568.86	1.2490	953.83 ₍₅₎	1005.35	0.6707
2	2190.94 ₍₆₎	2392.85	0.0535	2195.57 ₍₆₎	2249.47	0.3844	787.61 ₍₆₎	815.77	1.2557	733.2 ₍₆₎	750.04	3.1785	1168.18 ₍₆₎	1260.79	2.1261
3	2199.51 ₍₃₎	2344.06	0.0456	2081.68 ₍₃₎	2214.83	0.4135	674.88 ₍₃₎	702.43	1.2211	669.99 ₍₃₎	673.47	2.9397	1133.91 ₍₃₎	1203.82	2.2798
4	3003.18 ₍₄₎	3051.01	0.0584	2577.62 ₍₄₎	2729.04	0.8748	855.49 ₍₄₎	878.09	2.7445	785.84 ₍₄₎	786.31	6.4984	1679.47 ₍₄₎	1748.28	4.2555
5	42857.41 ₍₃₎	45456.2	0.1443	20577.08 ₍₃₎	22842.9	16.024	6134.72 ₍₄₎	6215.2	94.805	5386.65 ₍₄₎	5386.7	309.6680	33783.06 ₍₅₎	35587.11	180.0369
6	14447.08 ₍₁₈₎	14776.5	0.1187	13590.27 ₍₁₈₎	13947.04	5.0068	2885.78 ₍₁₈₎	2981.08	16.2967	2689.54 ₍₁₉₎	2689.54	47.8148	6360.92 ₍₁₈₎	6442.67	33.7310
7	42733.71 ₍₃₇₎	43345.77	0.0544	40768.37 ₍₃₇₎	41755.41	3.4151	8129.33 ₍₃₆₎	8426.49	12.167	7261.48 ₍₃₈₎	7261.48	496.4777	14464.05 ₍₃₆₎	14597.07	236.7240
8	89891.21 ₍₅₆₎	89997.35	0.188	85460.28 ₍₅₆₎	86530.78	6.434	16998.52 ₍₅₆₎	17402.21	451.87	13678.65 ₍₅₆₎	13678.65	951.639	29423.53 ₍₅₆₎	29516.93	889.3156

Tabla 16: Resumen de los resultados en las ocho instancias CVRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	597.38 ₍₄₎	612.15	2.4681	598.96 ₍₄₎	640.49	0.5086	1044.96 ₍₅₎	1060.82	1.1911	1435.45 ₍₄₎	1514.74	0.2259	1287.21 ₍₅₎	1287.21	0.0522
2	801.6 ₍₆₎	816.92	8.7697	806.83 ₍₆₎	854.40	1.3093	1199.09 ₍₉₎	1255.48	5.9780	2293.26 ₍₆₎	2386.57	0.7546	2248.92 ₍₉₎	2248.92	0.0815
3	666.86 ₍₃₎	676.99	15.7553	800.91 ₍₃₎	881.26	1.7931	1824.51 ₍₃₎	1856.99	2.6769	2247.21 ₍₃₎	2354.26	0.8910	1701.09 ₍₃₎	1701.09	0.1379
4	814.49 ₍₄₎	835.15	36.1031	1005.45 ₍₄₎	1100.78	3.3230	1532.82 ₍₅₎	1570.03	7.3232	3043.83 ₍₄₎	3252.77	1.9394	2489.6 ₍₅₎	2489.6	0.2383
5	6499.42 ₍₄₎	6534.7	337.7738	16421.59 ₍₄₎	17693.7	31.3677	31942.41 ₍₅₎	32017.5	311.9699	40998.82 ₍₄₎	44109.3	87.2194	16614.79 ₍₅₎	16614.79	7.5981
6	2689.13 ₍₁₈₎	2875.5	125.1227	2391.66 ₍₁₈₎	2454.23	9.7152	6231.99 ₍₅₀₎	6316.00	445.3573	14552.85 ₍₁₈₎	14925.9	13.2280	16996.74 ₍₅₀₎	16996.74	0.3405
7	7256.86 ₍₃₆₎	7495.7	63.1086	6239.59 ₍₃₆₎	6467.23	136.5664	18854.77 ₍₃₈₎	18904.3	304.4157	42050.21 ₍₃₇₎	43031.5	10.8614	45671.81 ₍₁₀₀₎	45671.81	1.2048
8	15288.58 ₍₅₆₎	15604.8	214.7608	13386.91 ₍₅₆₎	13421.6	62.2868	40000.05 ₍₅₇₎	42390.56	500.9123	84491.08 ₍₅₆₎	85832.8	449.4740	94784.64 ₍₁₅₀₎	94784.64	2.7531

Análisis de los resultados para la variante HFVRP

En las Tabla 17 y Tabla 18 se resumen los resultados obtenidos sobre los ocho problemas de la variante HFVRP. Para esta variante se realizan experimentos para el método de ordenamiento *Descending* de la lista de capacidades, por obtener los mejores resultados en la versión anterior [38].

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron con las heurísticas *SaveSeq*, *SaveParall* y *MJ*, mientras que las soluciones de mayor costo fueron encontradas con los algoritmos *Random*, *Sweep* y *FJ*.
- Las heurísticas *CMT*, *SaveMatch*, *NN* y *Kilby* no obtienen los mejores resultados, pero tampoco los peores, por lo cual su comportamiento es regular.
- Se puede observar que *Random* obtiene los peores resultados en las soluciones, pero el menor tiempo de ejecución en todos los casos.
- La heurística *MJ* obtiene buenos resultados, pero también es el que mayor tiempo emplea en su ejecución. Aunque, a medida que aumenta la cantidad de clientes, las que obtienen mayores tiempos son *SaveMatch* y *Kilby*.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución. Además, obtiene tiempos de ejecución pequeños para todas las instancias probadas.

Tabla 17: Resumen de los resultados en las ocho instancias HFVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1058.99 ₍₃₎	1152.25	0.0351	806.62 ₍₃₎	917.40	0.1540	550.34 ₍₄₎	566.53	0.4820	474.33 ₍₂₎	497.99	1.0935	839.36 ₍₃₎	922.51	1.7534
2	1914.54 ₍₅₎	2061.52	0.0471	1604.56 ₍₅₎	1696.16	0.4395	802.7 ₍₆₎	835.85	1.4416	851.96 ₍₇₎	851.96	1.181	1133.9 ₍₃₎	1185.49	7.1024
3	1444.78 ₍₄₎	1594.74	0.0456	1130.49 ₍₄₎	1171.20	0.4109	666.31 ₍₃₎	698.18	1.3554	653.83 ₍₃₎	658.68	4.1937	1170.71 ₍₃₎	1213.99	3.7284
4	2983.77 ₍₃₎	3123.60	0.0721	1899.58 ₍₃₎	1960.17	0.8415	768.28 ₍₃₎	774.38	2.6611	982.69 ₍₄₎	990.28	13.602	1613.45 ₍₃₎	1527.92	9.8802
5	43889.95 ₍₃₎	44459.46	0.2249	21942.66 ₍₃₎	22355.98	33.670	6182.04 ₍₄₎	6255.36	24.611	7245.90 ₍₄₎	7277.43	41.33	35223.4 ₍₄₎	35622.27	425.6863
6	13510.74 ₍₁₄₎	13923.44	0.2639	12729.4 ₍₁₄₎	13096.29	6.1536	2597.15 ₍₁₅₎	2680.11	20.9691	2525.17 ₍₁₅₎	2620.46	102.1874	6320.81 ₍₁₅₎	6320.81	572.2572
7	39048.76 ₍₂₂₎	39743.58	0.2715	36842.51 ₍₂₂₎	38047.66	35.7714	5787.86 ₍₂₃₎	5934.57	126.7698	5955.38 ₍₅₎	5970.02	4.4937	13838.25 ₍₂₃₎	13838.25	8460.7132
8	85067.42 ₍₅₁₎	86007.76	0.5714	81370.56 ₍₅₁₎	83576.92	119.4991	15294.68 ₍₅₁₎	15882.32	220.9266	16967.84 ₍₈₎	18945.3	7.9899	31177.03 ₍₅₁₎	31177.03	40309.355

Tabla 18: Resumen de los resultados en las ocho instancias HFVRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	566.31 ₍₄₎	582.13	3.9872	587.91 ₍₄₎	625.13	0.6129	1043.87 ₍₄₎	1065.77	1.3930	1029.56 ₍₃₎	1121.32	0.2746	810.55 ₍₄₎	810.55	0.1514
2	799.87 ₍₆₎	816.30	9.3635	802.23 ₍₆₎	851.58	1.3590	1211.89 ₍₉₎	1255.69	5.7586	1883.38 ₍₅₎	2067.59	0.8829	1600.32 ₍₉₎	1600.32	0.3367
3	670.57 ₍₃₎	698.27	16.6713	848.13 ₍₃₎	913.49	1.7733	1826.66 ₍₃₎	1896.52	3.0008	1343.62 ₍₂₎	1578.88	0.8762	1400.01 ₍₃₎	1400.01	0.3044
4	782.9 ₍₃₎	784.41	32.1689	1180.74 ₍₃₎	1242.09	3.4130	1533.05 ₍₅₎	1556.23	7.3406	2121.06 ₍₃₎	2123.70	1.8406	1995.87 ₍₅₎	1995.87	0.8203
5	6266.98 ₍₄₎	6395.63	6855.8267	16782.7 ₍₄₎	18917.08	112.6921	31860.28 ₍₅₎	32125.54	85.1255	43438.99 ₍₃₎	44260.40	80.9879	38990.99 ₍₅₎	38990.99	0.3167
6	2340.17 ₍₁₅₎	2408.97	371.9073	2118.76 ₍₁₅₎	2163.73	19.9983	6148.38 ₍₅₀₎	6231.82	490.645	12729.4 ₍₁₄₎	13096.29	6.1536	14678.4 ₍₅₀₎	14678.4	3.9011
7	5442.21 ₍₂₃₎	5503.95	217.3151	4705.29 ₍₂₃₎	4905.91	121.9479	18757.2 ₍₁₀₀₎	18890.44	6833.156	39203.75 ₍₂₂₎	40799.19	213.4071	34894.6 ₍₁₀₀₎	34894.6	11.2789
8	13769.09 ₍₅₁₎	13940.0	183.9995	11564.1 ₍₅₁₎	12345.32	1003.677	36976.48 ₍₁₅₀₎	37395.33	1392.285	85733.38 ₍₅₁₎	86365.48	336.3865	8325.11 ₍₁₅₀₎	8325.11	20.0897

Análisis de los resultados para la variante MDVRP

En las Tabla 19 y Tabla 20 se resumen los resultados obtenidos sobre los ocho problemas de la variante MDVRP seleccionados de la literatura. Para esta variante se realizan experimentos para el método de asignación *BestNearest*, por obtener los mejores resultados en la versión anterior [38].

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron con la heurística *MJ*, mientras que las soluciones de mayor costo fueron encontradas con las heurísticas *FJ*, *Random*, *Sweep*, *NN* y *SaveMatch*.
- Las heurísticas *SaveParall* y *CMT* obtienen buenos resultados.
- Las heurísticas *SaveSeq* y *Kilby* tienen un comportamiento regular, pues no quedan entre las heurísticas de construcción con los mejores ni los peores resultados.
- En los casos donde coinciden los valores mínimos y el promedio, significa que la heurística tuvo un comportamiento determinista y que el operador *3opt* no obtuvo mejoras.
- Los mejores tiempos se obtienen con la heurística *Random*, *FJ*, *NN* y *Sweep*, mientras que los peores tiempos fueron con *MJ*, *Kilby* y *SaveMatch*.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución. Además, presenta tiempos de ejecución pequeños.

Tabla 19: Resumen de los resultados en las ocho instancias MDVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	954.14 ₍₁₂₎	1094.55	0.0900	963.18 ₍₁₂₎	1014.82	0.0828	1022.54 ₍₁₂₎	1062.98	0.2845	786.07 ₍₁₃₎	786.07	0.4551	1418.77 ₍₁₂₎	1418.77	0.2841
2	2407.18 ₍₉₎	2552.84	0.0788	2269.18 ₍₉₎	2359.61	0.5166	1099.85 ₍₉₎	1144.25	1.4595	974.44 ₍₉₎	974.44	3.3595	1293.48 ₍₁₀₎	1406.76	2.2098
3	8031.91 ₍₁₆₎	8215.06	0.1177	8200.01 ₍₁₆₎	8386.06	0.8904	5562.36 ₍₁₆₎	5698.81	2.7322	3318.6 ₍₁₆₎	3318.6	4.7684	8403.63 ₍₁₆₎	8403.63	4.2784
4	18571.65 ₍₃₇₎	18917.24	0.3887	18718.21 ₍₃₆₎	19071.19	3.3262	18709.53 ₍₃₆₎	18886.34	12.5386	7466.85 ₍₃₆₎	7466.85	16.923	20070.49 ₍₃₆₎	20070.49	20.0164
5	10754.42 ₍₂₃₎	11242.51	0.2773	8704.52 ₍₂₃₎	8965.72	3.4387	4930.64 ₍₂₃₎	5039.68	13.976	4024.24 ₍₂₃₎	4024.24	23.3302	6286.74 ₍₂₂₎	6659.06	16.0464
6	15116.31 ₍₂₇₎	15516.12	0.2783	13267.24 ₍₂₈₎	13761.88	2.5329	8102.38 ₍₂₇₎	8420.09	9.2313	6021.26 ₍₂₇₎	6021.26	15.7345	10028.45 ₍₂₆₎	10448.35	8.4764
7	853.62 ₍₇₎	936.43	0.0925	877.89 ₍₇₎	925.82	0.1336	735.19 ₍₇₎	755.19	0.2717	602.23 ₍₇₎	602.23	0.3698	792.08 ₍₇₎	872.21	2.1052
8	5677.25 ₍₁₂₎	6009.17	0.1951	4798.05 ₍₁₁₎	4966.42	0.6425	2686.37 ₍₁₁₎	2761.28	2.3026	2639.18 ₍₁₂₎	2639.18	3.6211	3726.6 ₍₁₂₎	3911.75	4.2291

Tabla 20: Resumen de los resultados en las ocho instancias MDVRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	645.99 ₍₈₎	645.99	0.3784	712.66 ₍₁₂₎	748.74	0.2281	1157.68 ₍₁₆₎	1157.68	0.2965	920.69 ₍₁₂₎	1027.11	0.1339	980.22 ₍₁₆₎	980.22	0.0899
2	790.09 ₍₆₎	803.09	9.9856	981.77 ₍₉₎	1030.56	1.8661	1406.47 ₍₁₀₎	1434.98	4.9874	2474.59 ₍₉₎	2572.76	0.5599	2456.35 ₍₁₀₎	2456.35	0.1124
3	2690.26 ₍₁₂₎	2790.24	18.0643	3603.04 ₍₁₆₎	3752.61	3.4766	4716.17 ₍₂₀₎	4756.19	8.8169	7963.48 ₍₁₆₎	8219.78	0.6854	8190.09 ₍₂₀₎	8190.09	0.2546
4	6339.63 ₍₂₇₎	6392.52	82.2219	8419.52 ₍₃₆₎	8677.32	16.2640	10813.19 ₍₄₅₎	10937.99	43.0887	18357.21 ₍₃₆₎	18875.29	1.5296	18900.67 ₍₄₅₎	18900.67	1.9780
5	3664.36 ₍₁₉₎	3736.03	107.2054	3936.36 ₍₂₃₎	4139.97	15.0633	6876.3 ₍₂₄₎	7028.82	65.3264	10795.79 ₍₂₃₎	11196.25	2.6812	9765.45 ₍₂₄₎	9765.45	1.9845
6	4952.36 ₍₂₂₎	5133.44	48.9223	5308.3 ₍₂₇₎	5427.71	9.4167	8611.26 ₍₃₂₎	8769.58	54.9366	14979.95 ₍₂₇₎	15591.27	2.0606	14678.66 ₍₃₂₎	14678.66	1.5677
7	351.46 ₍₃₎	367.82	0.7900	586.94 ₍₇₎	635.25	0.3275	839.98 ₍₈₎	862.37	0.4989	873.73 ₍₇₎	947.57	0.0618	867.21 ₍₈₎	867.21	0.2044
8	1834.98 ₍₇₎	1894.69	17.2066	2586.14 ₍₁₁₎	2761.5	3.4065	3784.55 ₍₁₂₎	3890.21	5.7184	5763.37 ₍₁₁₎	5966.67	0.4829	5900.02 ₍₁₂₎	5900.02	0.4453

Análisis de los resultados para la variante TTRP

En las Tabla 21 y Tabla 22 se resumen los resultados obtenidos sobre los ocho problemas de la variante TTRP seleccionados de la literatura.

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron con la heurística *SaveParall*, mientras que las soluciones de mayor costo fueron encontradas con las heurísticas *Random*, *Sweep*, *FJ* y *NN*.
- Las heurísticas *MJ* y *CMT* obtienen buenos resultados, similar a *SaveParall*.
- Las heurísticas *SaveSeq* y *Kilby* tienen un comportamiento regular, pues no obtienen los mejores ni los peores resultados.
- En los casos donde coinciden los valores mínimos y el promedio, por ejemplo, la heurística *Kilby*, significa que tuvo un comportamiento determinista y que el operador *3opt* no obtuvo mejoras.
- Los mejores tiempos se obtienen con la heurística *Random*, mientras que los peores tiempos fueron con *MJ*.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución. Además, obtiene tiempos de ejecución pequeños para todas las instancias probadas.

Tabla 21: Resumen de los resultados en las ocho instancias TTRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1449.5 ₍₆₎	1561.81	0.045	1393.7 ₍₆₎	1524.49	0.1817	605.8 ₍₁₎	610.36	0.417	570.14 ₍₄₎	580.79	0.5273	935.6 ₍₄₎	956.92	0.4926
2	2402.12 ₍₁₂₎	2545.77	0.0508	2337.93 ₍₁₂₎	2381.66	0.3972	1031.69 ₍₂₎	1038.77	1.273	809.01 ₍₈₎	820.58	1.4296	1300.66 ₍₈₎	1411.35	2.4013
3	3136.67 ₍₁₀₎	3327.62	0.0630	2971.46 ₍₉₎	3075.71	0.7898	1085.56 ₍₂₎	1106.50	2.391	869.89 ₍₆₎	879.66	2.701	1456.12 ₍₆₎	1678.23	1.9087
4	6054.28 ₍₉₎	6376.07	0.0799	2973.18 ₍₈₎	3217.73	1.3613	998.48 ₍₁₎	998.48	4.411	1024.31 ₍₅₎	1070.66	5.8528	1478.90 ₍₅₎	1590.34	5.8976
5	3752.04 ₍₁₁₎	3886.42	0.0825	2174.34 ₍₁₁₎	2308.14	0.9214	1014.53 ₍₃₎	1101.46	3.323	740.11 ₍₆₎	747.81	3.8310	1200.45 ₍₇₎	1316.76	4.9598
6	6236.11 ₍₂₃₎	6569.20	0.1228	5970.13 ₍₂₂₎	6191.25	5.0142	1834.74 ₍₂₎	1888.82	11.138	1290.53 ₍₁₃₎	1308.28	17.616	1889.98 ₍₁₄₎	1993.44	20.5612
7	1475.73 ₍₇₎	1583.77	0.0405	1466.58 ₍₈₎	1563.25	0.1554	730.51 ₍₂₎	786.99	0.3428	595.38 ₍₅₎	597.02	0.4937	845.33 ₍₅₎	901.23	0.3565
8	4612.43 ₍₁₂₎	4918.28	0.0837	4310.24 ₍₁₂₎	4535.62	2.4486	1197.32 ₍₁₎	1189.75	5.613	967.84 ₍₈₎	977.43	7.9899	1709.87 ₍₈₎	1876.91	6.4523

Tabla 22: Resumen de los resultados en las ocho instancias TTRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	603.98 ₍₅₎	616.11	1.8323	822.49 ₍₉₎	881.23	0.3237	1208.06 ₍₁₀₎	1208.06	1.1457	1419.09 ₍₆₎	1535.37	0.2584	1583.34 ₍₈₎	1583.34	0.0498
2	966.86 ₍₁₀₎	967.12	6.4166	1344.54 ₍₁₅₎	1401.16	1.3781	1712.69 ₍₁₈₎	1712.69	4.9588	2396.65 ₍₁₂₎	2529.72	0.7926	2617.0 ₍₁₄₎	2617	0.0665
3	1125.96 ₍₈₎	1150.70	14.5486	1267.31 ₍₁₀₎	1334.63	1.8367	1953.44 ₍₁₀₎	2124.76	5.0726	3126.42 ₍₉₎	3323.73	1.8182	2902.9 ₍₁₂₎	2902.9	0.1496
4	1216.31 ₍₈₎	1230.55	56.5215	1580.5 ₍₁₀₎	1686.62	2.4692	3385.09 ₍₁₄₎	3385.09	14.4167	5918.16 ₍₉₎	6381.68	3.0313	2572.43 ₍₁₁₎	2572.4	0.3034
5	983.85 ₍₁₀₎	1016.63	22.0210	1208.92 ₍₁₃₎	1288.28	1.6385	2682.64 ₍₂₀₎	2682.64	10.1838	3677.54 ₍₁₂₎	3873.64	1.8079	2159.16 ₍₁₅₎	2159.2	0.1506
6	1876.13 ₍₁₉₎	1904.92	105.1122	2084.52 ₍₂₂₎	2125.39	13.2150	3858.2 ₍₃₄₎	3858.2	135.5542	6521.21 ₍₂₂₎	6634.31	12.2426	5928.13 ₍₂₆₎	5928.1	0.4862
7	631.11 ₍₅₎	645.83	1.8917	813.44 ₍₈₎	891.55	0.2515	1199.14 ₍₁₀₎	1199.14	1.0958	1515.52 ₍₇₎	1601.22	0.2534	1648.9 ₍₈₎	1648.9	0.0449
8	1264.43 ₍₉₎	1284.69	71.0268	1634.92 ₍₁₆₎	1716.66	4.8927	2733.57 ₍₂₄₎	2733.57	45.2076	4848.46 ₍₁₃₎	4944.19	5.7698	4448.21 ₍₁₈₎	4448.2	0.3022

Análisis de los resultados para la variante OVRP

En las Tabla 23 y Tabla 24 se resumen los resultados obtenidos sobre los ocho problemas de la variante OVRP seleccionados previamente.

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron en la heurística *SaveParall*. Sin embargo, también se obtienen buenos resultados para las heurísticas *SaveSeq* y *MJ*.
- Las soluciones más costosas obtenidas con las heurísticas *Random* y *NN* son superadas por los resultados de la heurística *Sweep*, por lo que se puede apreciar que esta heurística obtiene los peores resultados.
- Las heurísticas *CMT*, *SaveMatch*, *FJ* y *Kilby* tienen un comportamiento regular, pues no quedan sus resultados entre los mejores, pero tampoco entre los peores.
- La heurística *MJ* obtiene buenos resultados, pero también es el que mayor tiempo emplea en su ejecución. Aunque es importante destacar que, mientras aumenta la cantidad de clientes, las que obtienen mayores tiempos son *SaveParall*, *SaveMatch* y *Kilby*. Sin embargo, *Random* obtiene los resultados desfavorables en las soluciones, pero el menor tiempo de ejecución en todos los casos.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución.

Tabla 23: Resumen de los resultados en las ocho instancias OVRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1472.51 ₍₄₎	1670.25	0.0218	1378.18 ₍₄₎	1570.50	0.1506	493.19 ₍₄₎	650.50	0.5345	501.63 ₍₄₎	651.86	1.120	804.37 ₍₅₎	995.12	0.7269
2	2199.21 ₍₆₎	2390.50	0.0309	2121.62 ₍₆₎	2320.75	0.4229	747.62 ₍₆₎	900.75	1.3653	672.1 ₍₆₎	722.55	3.1446	1070.67 ₍₆₎	1381.07	2.1462
3	2115.43 ₍₃₎	2300.75	0.0299	2190.84 ₍₃₎	2390.25	0.4159	662.84 ₍₃₎	820.25	1.2935	636.66 ₍₃₎	736.78	3.4317	1020.79 ₍₃₎	1271.04	2.2729
4	3038.36 ₍₄₎	3230.60	0.0429	2475.26 ₍₄₎	2670.40	0.8218	827.02 ₍₄₎	990.40	2.7057	753.05 ₍₄₎	828.72	7.3473	1629.23 ₍₄₎	1805.03	4.7184
5	57157.67 ₍₄₎	57350.11	0.2394	37155.27 ₍₄₎	37350.27	33.901	5839.91 ₍₄₎	6012.07	99.2656	5220.21 ₍₄₎	5340.55	296.70	36288.36 ₍₄₎	36638.36	164.1424
6	13876.33 ₍₁₈₎	14100.50	0.099	13133.47 ₍₁₈₎	13350.15	5.1991	2098.8 ₍₁₈₎	2300.92	18.6371	1758.9 ₍₁₉₎	1909.80	49.0948	5118.39 ₍₁₈₎	5343.99	30.5363
7	38937.32 ₍₃₇₎	39150.02	0.2822	36743.25 ₍₃₇₎	36950.38	36.028	5098.23 ₍₃₆₎	5300.63	132.922	4818.34 ₍₃₈₎	5018.45	441.3207	12020.32 ₍₃₆₎	12296.22	238.3054
8	82145.17 ₍₅₇₎	82300.43	0.5644	77879.86 ₍₅₇₎	78050.04	130.57	10167.89 ₍₅₆₎	10350.5	473.27	8718.58 ₍₅₇₎	9118.58	1772.99	27124.56 ₍₅₆₎	27224.56	850.3247

Tabla 24: Resumen de los resultados en las ocho instancias OVRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	512.65 ₍₄₎	657.95	3.1268	750.74 ₍₄₎	850.99	0.1616	997.03 ₍₅₎	1247.63	0.1209	1475.36 ₍₄₎	1750.36	0.2473	1161.69 ₍₅₎	1161.69	0.0539
2	751.8 ₍₆₎	851.95	10.0072	1041.63 ₍₆₎	1317.33	0.4976	1118.11 ₍₉₎	1418.51	0.3328	2260.33 ₍₆₎	2410.33	0.7939	2032.11 ₍₉₎	2032.11	0.0908
3	671.73 ₍₃₎	772.18	18.7827	1213.42 ₍₃₎	1364.22	0.2374	1887.6 ₍₃₎	2063.30	0.1596	2348.76 ₍₃₎	2548.76	0.7889	1614.96 ₍₃₎	1614.96	0.1476
4	755.24 ₍₄₎	845.84	42.4126	1541.67 ₍₄₎	1767.12	0.4977	1509.78 ₍₅₎	1909.78	0.4179	3000.95 ₍₄₎	3400.95	1.9747	2358.1 ₍₅₎	2358.1	0.2453
5	6153.64 ₍₄₎	6553.64	1266.1553	24322.45 ₍₄₎	24672.45	5.7888	32068.13 ₍₅₎	35568.01	14.2324	59327.58 ₍₄₎	59502.58	86.0853	15789.98 ₍₅₎	15789.98	8.2460
6	1777.16 ₍₁₈₎	1927.66	130.0405	2002.11 ₍₁₈₎	2402.11	18.2387	6126.85 ₍₅₀₎	6426.85	412.3347	13559.23 ₍₁₉₎	13859.23	14.7176	14520.93 ₍₅₀₎	14520.93	0.3665
7	4584.89 ₍₃₆₎	4884.89	904.9372	5175.23 ₍₃₆₎	5351.13	226.2404	12319.26 ₍₁₀₀₎	12569.26	277.5988	38631.6 ₍₃₈₎	38781.86	109.5669	37767.34 ₍₍₁₀₀₎₎	37767.34	1.3045
8	9261.71 ₍₅₆₎	9500.69	1142.9552	9875.07 ₍₅₆₎	10875.07	1174.4112	24969.23 ₍₁₅₀₎	25119.23	1385.956	80931.77 ₍₅₇₎	81931.77	358.3202	80166.98 ₍₁₅₀₎	80166.98	2.8888

Análisis de los resultados para la variante VRPTW

En las Tabla 25 y Tabla 26 se resumen los resultados obtenidos sobre los ocho problemas de la variante VRPTW seleccionados previamente.

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron en la heurística *SaveParall*. Sin embargo, también se obtienen buenos resultados para las heurísticas *SaveSeq* y *MJ*.
- Las soluciones más costosas obtenidas con las heurísticas *Sweep* y *NN* son superadas por los resultados de la heurística *Random*, por lo que se puede apreciar que esta heurística obtiene los peores resultados.
- Las heurísticas *CMT*, *SaveMatch* y *FJ* tienen un comportamiento regular, pues no quedan entre las heurísticas de construcción de los mejores resultados, pero tampoco entre las heurísticas de construcción con los peores resultados.
- La heurística *Kilby* obtiene resultados aceptables, pero también es el que mayor tiempo emplea en su ejecución. Aunque es importante destacar que también obtienen tiempos elevados *SaveParall* y *SaveMatch*. Sin embargo, *Random* obtiene los resultados desfavorables en las soluciones, pero el menor tiempo de ejecución en todos los casos.
- La heurística *FJ* es determinista, por eso sus valores promedios coinciden con la mejor solución.

Tabla 25: Resumen de los resultados en las ocho instancias VRPTW (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	14913.15 ₍₅₀₎	15163.90	0.4259	16243.1 ₍₄₈₎	16393.60	2.7763	6473.03 ₍₃₅₎	6748.63	51.5756	3076.18 ₍₂₄₎	3201.68	61.2799	11721.84 ₍₃₄₎	12021.84	63.5195
2	14837.82 ₍₁₉₎	15013.42	0.6109	14505.93 ₍₁₈₎	14806.18	6.9752	3258.83 ₍₁₉₎	3359.23	19.3678	2665.15 ₍₁₉₎	2865.15	63.3887	5470.18 ₍₁₈₎	5645.93	79.5684
3	15223.11 ₍₂₄₎	15523.23	0.7313	15226.9 ₍₂₂₎	15402.80	3.2311	4559.73 ₍₂₂₎	4859.73	79.8968	2889.12 ₍₂₁₎	3189.22	65.3902	8043.0 ₍₂₂₎	8293.30	72.3473
4	14840.41 ₍₁₉₎	15065.86	0.3799	14458.1 ₍₁₈₎	14683.55	2.9989	3625.79 ₍₁₉₎	3801.69	19.9722	2803.32 ₍₂₀₎	2979.07	50.1480	6818.1 ₍₁₉₎	6968.10	45.1075
5	43007.55 ₍₁₀₀₎	43357.55	1.5738	44981.6 ₍₈₀₎	45331.60	18.1005	19640.87 ₍₇₃₎	19891.1	388.23	9990.51 ₍₅₃₎	10340.53	453.6232	33894.53 ₍₇₃₎	34294.53	588.6174
6	42566.39 ₍₃₆₎	42757.19	1.5039	39352.55 ₍₃₆₎	39552.65	33.3926	8257.94 ₍₃₆₎	8407.94	132.567	7273.54 ₍₃₈₎	7523.79	438.2462	15637.12 ₍₃₆₎	15862.72	301.7654
7	91548.39 ₍₁₅₀₎	91948.39	4.4416	92066.4 ₍₉₉₎	92441.60	60.5418	35755.67 ₍₉₅₎	36155.61	1205.43	18568.01 ₍₇₆₎	18743.93	1743.664	67853.82 ₍₁₀₉₎	71603.82	1835.479
8	88569.86 ₍₆₂₎	88845.16	4.2431	83037.42 ₍₅₆₎	83188.17	117.523	17479.54 ₍₅₆₎	17679.72	557.059	14279.23 ₍₅₇₎	14479.24	1703.078	42088.19 ₍₅₉₎	42288.19	1850.917

Tabla 26: Resumen de los resultados en las ocho instancias VRPTW (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	8753.17 ₍₄₇₎	8903.17	15.4804	8482.62 ₍₅₆₎	8682.62	41.6234	6150.62 ₍₅₀₎	6150.62	424.2524	14746.52 ₍₅₀₎	14946.52	5.4562	8081.16 ₍₅₀₎	8081.16	1.2247
2	2799.72 ₍₁₈₎	2974.72	133.3087	3339.49 ₍₂₀₎	3489.49	33.8390	6150.62 ₍₅₀₎	6150.62	435.5513	14523.11 ₍₂₀₎	14798.11	18.6112	18063.25 ₍₅₀₎	18063.25	1.5241
3	4797.05 ₍₂₄₎	5047.05	46.2125	7153.24 ₍₃₅₎	7453.24	46.0562	6150.62 ₍₅₀₎	6150.62	421.5786	14696.52 ₍₂₄₎	14996.52	4.1638	9844.91 ₍₅₀₎	9844.91	1.0762
4	3171.68 ₍₁₉₎	3471.68	63.4315	5108.64 ₍₂₆₎	5208.64	21.1883	6150.62 ₍₅₀₎	6150.62	495.8248	14794.05 ₍₁₉₎	15044.05	4.5757	11247.7 ₍₅₀₎	11247.7	1.1339
5	22541.96 ₍₉₂₎	22941.96	120.1218	23507.48 ₍₁₁₇₎	23857.48	494.8934	18245.27 ₍₁₀₀₎	18425.62	357.7228	42078.06 ₍₁₀₀₎	42455,48	42.0278	23246.62 ₍₁₀₀₎	23246.62	4.4349
6	6953.46 ₍₃₆₎	7128.46	995.3597	8859.31 ₍₃₇₎	9034.31	250.4818	18425.27 ₍₁₀₀₎	18425.62	350.9431	42243.48 ₍₃₆₎	42428.06	97.7299	50039.94 ₍₁₀₀₎	50039.94	4.1958
7	43295.21 ₍₁₃₁₎	43645.21	393.7526	50071.73 ₍₁₈₉₎	50471.73	232.6407	35433.01 ₍₁₅₀₎	35433.01	1822.7999	90350.78 ₍₁₅₀₎	90750,78	140.8676	49041.56 ₍₁₅₀₎	49041.56	27.8548
8	14678.38 ₍₅₆₎	14778.38	5575.9093	24964.37 ₍₆₉₎	25214.37	64.4284	35433.01 ₍₁₅₀₎	35433.01	1725.7743	88030.48 ₍₆₁₎	88330,48	479.3974	96022.87 ₍₁₅₀₎	96022.87	24.2157

Análisis de los resultados para la variante SBRP

En las Tabla 27 y Tabla 28 se resumen los resultados obtenidos sobre los ocho problemas de la variante SBRP seleccionados previamente.

A partir de los resultados alcanzados se puede concluir que:

- Las mejores soluciones se encontraron en la heurística *SaveParall*. Sin embargo, también se obtienen buenos resultados para las heurísticas *SaveSeq* y *MJ*.
- Las soluciones más costosas obtenidas con las heurísticas *SaveMatch*, *FJ* y *CMT*.
- Las heurísticas *NN*, *Sweep*, *Kilby* y *Random* tienen un comportamiento regular, pues no obtienen los mejores resultados, pero tampoco los peores.
- Los menores tiempos de ejecución los obtienen las heurísticas *Random* y *FJ*.
- De manera general, se obtienen tiempos de ejecución pequeños en casi todas las heurísticas. Esto se debe a que, aunque existan instancias con cantidad de clientes elevados, las rutas se conforman en base a las paradas, las cuales son menos de 200.
- Las heurísticas *FJ* y *SaveMatch* tienen un comportamiento determinista en todas las instancias probadas.

Tabla 27: Resumen de los resultados en las ocho instancias SBRP (I).

Problema	Random			NN			SaveSeq			SaveParall			SaveMatch		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	225.07 ₍₅₎	375.39	0.0119	242.7 ₍₅₎	342.70	0.0197	275.77 ₍₉₎	425.92	0.0247	153.75 ₍₅₎	253.75	0.0329	318.37 ₍₁₀₎	318.37	0.0233
2	3095.04 ₍₄₇₎	3370.84	0.0458	3232.22 ₍₄₇₎	3352.72	0.6337	2987.09 ₍₆₇₎	3087.34	1.0612	1987.57 ₍₄₅₎	2137.87	3.2188	3340.1 ₍₇₂₎	3340.1	2.0086
3	1380.17 ₍₂₎	1580.62	0.0179	904.67 ₍₂₎	1029.97	0.1027	203.53 ₍₅₎	378.98	0.2812	211.11 ₍₃₎	386.71	0.7390	3212.82 ₍₄₀₎	3212.82	0.4190
4	11722.37 ₍₄₁₎	12072.97	0.1359	7224.91 ₍₄₆₎	7574.91	10.0920	13919.22 ₍₂₁₆₎	14319.22	28.6674	2209.03 ₍₄₄₎	2509.18	136.9790	15919.44 ₍₂₄₀₎	15919.44	57.7646
5	678.1 ₍₅₎	803.85	0.0139	616.17 ₍₅₎	792.07	0.0299	450.23 ₍₁₂₎	600.23	0.0562	187.16 ₍₅₎	287.16	0.1257	841.32 ₍₁₉₎	841.32	0.0688
6	866.51 ₍₁₎	1041.71	0.0139	820.7 ₍₁₎	1020.70	0.0379	29.18 ₍₁₎	54.93	0.0967	68.51 ₍₁₎	218.51	0.1895	1423.88 ₍₂₄₎	1423.88	0.1177
7	4423.15 ₍₁₀₎	4723.25	0.0508	2850.35 ₍₉₎	3150.35	1.2905	4919.35 ₍₉₉₎	5269.95	4.1618	360.57 ₍₉₎	560.67	12.2547	6337.41 ₍₁₂₀₎	6337.41	8.1166
8	19165.74 ₍₈₃₎	22431.78	0.4049	11421.38 ₍₉₅₎	13876.22	64.5174	25253.9 ₍₄₇₀₎	25553.90	23.7713	3656.58 ₍₈₁₎	4031.78	1914.981	25787.78 ₍₄₈₀₎	25787.78	456.7195

Tabla 28: Resumen de los resultados en las ocho instancias SBRP (II).

Problema	MJ			CMT			Kilby			Sweep			FJ		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	276.45 ₍₈₎	522.12	0.1149	375.39	591.06	0.0753	318.37 ₍₁₀₎	318.37	0.0149	244.01 ₍₅₎	419.01	0.0209	318.37 ₍₁₀₎	318.37	0.0111
2	3150.87 ₍₆₅₎	3509.79	0.1481	3370.84	3729.76	0.0635	3340.1 ₍₇₂₎	3340.1	0.0481	2929.14 ₍₄₅₎	3179.14	1.1429	3340.1 ₍₇₂₎	3340.1	0.0279
3	1781.56 ₍₃₎	2094.01	1.2527	1580.62	1893.07	1.5515	202.81 ₍₅₎	202.81	1.2927	1360.01 ₍₃₎	1660.01	0.1495	3212.82 ₍₄₀₎	3212.82	0.0254
4	13950.22 ₍₃₇₎	14135.56	1121.9451	12072.97	12258.31	117.7874	2036.99 ₍₄₀₎	2036.99	1139.9311	11630.59 ₍₄₁₎	11780.59	25.9841	15919.44 ₍₂₄₀₎	15919.44	0.1375
5	712.67 ₍₁₇₎	988.48	0.2139	803.85	1079.66	0.5843	841.32 ₍₁₉₎	841.32	0.0139	596.14 ₍₅₎	721.14	0.0329	841.32 ₍₁₉₎	841.32	0.0166
6	758.45 ₍₆₎	1156.66	0.1415	1041.71	1439.92	0.1034	593.91 ₍₁₀₎	593.91	0.3415	924.08 ₍₁₎	1124.08	0.0445	1423.88 ₍₂₄₎	1423.88	0.0125
7	2950.12 ₍₁₈₎	3095.90	89.8207	4723.25	4869.03	72.3313	1012.03 ₍₂₀₎	1012.03	89.6007	4274.07 ₍₉₎	4624.07	3.2325	6337.41 ₍₁₂₀₎	6337.41	0.0536
8	10745.78 ₍₃₅₎	11113.68	955.7232	22431.78	22899.68	874.3215	3869.26 ₍₄₀₎	3869.26	957.7792	18879.46 ₍₈₂₎	21004.54	205.4101	25787.78 ₍₄₈₀₎	25787.78	0.5197

Análisis estadístico

Para mayor certeza se decide utilizar la prueba estadística no paramétrica de Friedman [117]. Esta permite comprobar si los resultados obtenidos con las heurísticas de construcción presentan diferencias significativas. Se utiliza el costo promedio de los problemas como medida para comparar el rendimiento de los diez algoritmos heurísticos.

Tabla 29: Rankings de las heurísticas para las siete variantes VRP en cuanto a calidad de soluciones en distancia euclidiana.

Algoritmos	Rankings						
	CVRP	HFVRP	MDVRP	TTRP	SBRP	VRPTW	OVRP
<i>Random</i>	8.875	9.375	7.5	8.625	5.6667	9.2	9.25
<i>NN</i>	7.5	6.5625	7.625	7.25	4.8333	9	7.75
<i>SaveSeq</i>	2.875	2.625	5	2.5	3.1667	2.8	2.625
<i>SaveParall</i>	1.875	2.75	2.625	1.125	1.5	1	1.25
<i>SaveMatch</i>	5.625	6.5	7	4.875	9.25	6.2	5.25
<i>MJ</i>	2.375	2.375	1	2.375	4.6667	3.4	2.375
<i>CMT</i>	2.875	2.75	2.375	4.125	6.1667	4.8	4.125
<i>Kilby</i>	6	6.625	5.75	6.5	5.6667	3.8	5.875
<i>Sweep</i>	9	8.5625	7.75	9.625	4.8333	8.2	9.375
<i>FJ</i>	8	6.875	8.375	8	9.25	6.6	7.125

En la Tabla 29, se resalta en negrita que para las variantes CVRP, TTRP, SBRP, VRPTW y OVRP la heurística *SaveParall* se posiciona como el mejor algoritmo. Sin embargo, para HFVRP y MDVRP se destaca la heurística *MJ*. Como los valores de p-valor de las mejores heurísticas tienen valor 0, sugieren la existencia de diferencias significativas entre los restantes algoritmos heurísticos considerados. Por tanto, se decide aplicar los métodos *post-hoc* Li, Finner y Holm como se aprecia en las Tabla 40, Tabla 41, Tabla 42, Tabla 43, Tabla 44, Tabla 45 y Tabla 46 del Anexo 2. Estos resultados implican lo siguiente:

- Para la variante CVRP, no existen diferencias significativas en las heurísticas *MJ*, *SaveSeq* y *CMT*, con respecto a *SaveParall*.

- Para la variante HFVRP, no existen diferencias significativas en las heurísticas *SaveSeq*, *CMT* y *SaveParall* con respecto a *MJ*.
- Para la variante MDVRP, no existen diferencias significativas en las heurísticas *CMT*, *SaveParall* y *SaveSeq*, con respecto a *MJ*.
- Para la variante TTRP, no existen diferencias significativas en las heurísticas *MJ*, *SaveSeq* y *CMT*, con respecto a *SaveParall*.
- Para la variante SBRP, no existen diferencias significativas en las heurísticas *SaveSeq*, *MJ*, *Sweep* y *NN*, con respecto a *SaveParall*.
- Para la variante VRPTW, no existen diferencias significativas en las heurísticas *SaveSeq*, *MJ* y *Kilby*, con respecto a *SaveParall*.
- Para la variante OVRP, no existen diferencias significativas en las heurísticas *MJ*, *SaveSeq* y *CMT*, con respecto a *SaveParall*.
- Para el resto de las heurísticas que no se mencionan, en las siete variantes VRPs presentan un p-valor menor que 0.05, por lo que sí presentan diferencias significativas con respecto a la mejor heurística.

Respecto al tiempo de ejecución, queda *Random* como la heurística que menos tiempo demora en su ejecución para las siete variantes de problemas de planificación de rutas de vehículos.

3.2.2 Comparación de los resultados obtenidos por las diferentes versiones de BHCVRP

En las Tabla 47, Tabla 48, Tabla 49, Tabla 50, Tabla 51, Tabla 52, Tabla 53 y Tabla 54 del Anexo 2, se resumen los resultados obtenidos de aplicar las siete heurísticas implementadas en la versión anterior de BHCVRP sobre las ocho instancias de cada variante VRP. En cada problema se resalta en color azul la mejor solución obtenida por las siete heurísticas. En los valores de costo en distancia euclidiana y tiempo de ejecución se consideran dos cifras significativas.

A partir de los resultados alcanzados en [38], se obtiene que la heurística *SaveParall* es la que da mejores resultados para las variantes CVRP, MDVRP y TTRP, mientras que

para HFVRP es la heurística *SaveSeq*. Sin embargo, al ejecutar esta versión anterior de BHCVRP para otras instancias, se obtienen mejores resultados para la variante TTRP con la heurística *SaveSeq*. Por otra parte, las heurísticas *Random* y *Sweep* obtienen las soluciones más costosas.

Con el objetivo de lograr un mayor grado de certeza en la comparación de los algoritmos ya existentes en BHCVRP, se decide aplicar la prueba estadística no paramétrica de Friedman [117]. Esta prueba se utiliza para determinar si hay diferencias significativas entre los resultados promedio alcanzados en cada heurística por cada versión de BHCVRP para las siete heurísticas en cuestión. En las Tabla 55, Tabla 56, Tabla 57 y Tabla 58 del Anexo 2 se resumen dichos resultados. Además, se confecciona la Tabla 30, para obtener un *ranking* general de las diferentes versiones de BHCVRP. Para ello se tuvo en cuenta la unión de los *rankings* de las siete heurísticas para las cuatro variantes VRPs.

Tabla 30: Ranking general de las versiones de BHCVRP.

Versión de BHCVRP	<i>Ranking</i>
<i>Java_anterior</i>	2
<i>Java_actual</i>	1.875
<i>Python_actual</i>	2.125

Para alcanzar mayor precisión, se aplican los métodos *post-hoc* Li, Finner y Holm, como se aprecia en la Tabla 59. Al obtener un p-valor mayor que 0.05, se demuestra que no existen diferencias significativas entre las diferentes versiones de BHCVRP, a pesar de que para las instancias probadas la versión actual desarrollada en Java obtuvo los mejores resultados.

3.2.3 Comparación de los resultados con OR-Tools

Debido a la relevancia de la herramienta OR-Tools, sobre todo para la comunidad en Python, se decide realizar un experimento para determinar la calidad de las soluciones en cuanto a costos y tiempo de ejecución. En las Tabla 31, Tabla 32 y Tabla 33, se resumen los mejores resultados en la actual versión de BHCVRP en Python y OR-Tools, para las variantes CVRP, HFVRP y VRPTW. En cada problema se destaca en negrita color azul la mejor solución obtenida y el menor tiempo de ejecución. En los valores de costo en distancia euclidiana y tiempo de ejecución se consideran dos cifras significativas.

Tabla 31: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias CVRP.

Problema	BHCVRP			OR-Tools		
	Costo en distancia	Total de rutas	Tiempo de ejecución (s)	Costo en distancia	Total de rutas	Tiempo de ejecución (s)
1	558.89	4	0.54	2152.00	4	0.13
2	787.61	6	1.25	3916.00	6	0.33
3	666.86	3	15.75	4133.00	3	0.27
4	814.49	4	36.10	4534.00	4	0.57
5	6134.72	4	94.80	9298.00	4	15.00
6	2391.66	18	9.71	2597.00	19	1.61
7	6239.59	36	136.57	6970.00	37	9.12
8	13386.91	56	62.29	13881.00	59	15.00

Los resultados obtenidos para la variante CVRP, resumidos en la Tabla 31, muestran que BHCVRP presenta las mejores soluciones en cuanto a costo en distancia euclidiana para todas las instancias. Sin embargo, OR-Tools obtiene en todos los casos los mejores tiempos de ejecución.

Tabla 32: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias HFVRP.

Problema	BHCVRP			OR-Tools		
	Costo en distancia	Total de rutas	Tiempo de ejecución (s)	Costo en distancia	Total de rutas	Tiempo de ejecución (s)
1	550.34	4	0.48	3225.00	4	0.14
2	799.87	6	9.36	3730.00	4	0.30
3	666.31	3	1.35	4030.00	3	0.40
4	768.28	3	2.66	1512.00	3	0.71
5	6125.32	4	24.61	9327.00	4	12.03
6	2118.76	15	19.99	2597.00	19	1.61
7	4705.29	23	121.95	6970.00	37	9.12
8	11564.10	51	1003.68	13881.00	59	15.00

A partir de los resultados alcanzados para la variante HFVRP en la Tabla 32, se puede apreciar que BHCVRP obtiene las mejores soluciones en cuanto a costo en distancia euclidiana para todas las instancias. Sin embargo, OR-Tools obtiene en todos los casos los mejores tiempos de ejecución.

Tabla 33: Resumen de los resultados obtenidos por BHCVRP y OR-Tools en las ocho instancias VRPTW.

Problema	BHCVRP			OR-Tools		
	Costo en distancia	Total de rutas	Tiempo de ejecución (s)	Costo en distancia	Total de rutas	Tiempo de ejecución (s)
1	3076.18	24	61.2799	2990.0	21	4.97
2	2665.15	19	63.3887	2815.0	20	10.95
3	2889.12	21	65.3902	2859.0	21	7.27
4	2803.32	20	50.1480	2708.0	20	7.65
5	9990.51	53	453.6232	7242.0	40	39.42
6	7273.54	38	438.2462	7491.0	38	54.87
7	18568.01	76	1743.664	15364.0	64	60.00
8	14279.23	57	1703.078	15778.0	59	60.00

Por último, para la variante VRPTW se obtienen los resultados resumidos en la Tabla 33. Estos demuestran que OR-Tools obtiene los mejores resultados en cuanto a costo en distancia euclidiana para la mayoría de las instancias, así como los mejores tiempos de ejecución para todos los casos.

3.3 Conclusiones parciales

Después de la realización de los experimentos definidos previamente con el objetivo de analizar las heurísticas de construcción y su comportamiento en cuatro variantes de VRP, se obtienen las siguientes conclusiones:

- Los resultados obtenidos para las variantes CVRP, TTRP, OVRP, SBRP y VRPTW sugieren que las heurísticas que alcanzan mejores soluciones son *SaveParall*, *MJ* y *SaveSeq*. Sin embargo, para la variante HFVRP los mejores resultados son con las heurísticas *MJ* y *SaveSeq*, y para la variante MDVRP son *MJ* y *CMT*.
- Las pruebas estadísticas realizadas confirman que para las variantes CVRP, TTRP, OVRP, SBRP y VRPTW, la heurística *SaveParall* es la de mejor rendimiento y para las variantes HFVRP y MDVRP es recomendable *MJ*.
- En cuanto al parámetro tiempo, se evidencia que las heurísticas *Kilby*, *SaveMatch* y *MJ* son las que mayor cantidad de tiempo consumen, solo esta última obteniendo buenos resultados. Sin embargo, la heurística *Random* obtiene tiempos pequeños de ejecución alcanzando los peores resultados en todos los casos.
- Las pruebas estadísticas realizadas para la comparación entre las diferentes versiones de BHCVRP, demuestran que no existen diferencias significativas entre las implementaciones de las mismas.
- En la comparación realizada con la herramienta OR-Tools, se evidencia que BHCVRP obtuvo mejores resultados en cuanto a costo en distancia euclidiana para las variantes CVRP y HFVRP, aunque OR-Tools presentó menores tiempos de ejecución para todos los casos. Sin embargo, OR-Tools obtuvo los mejores resultados globales para la variante VRPTW.

Conclusiones

Después de dar cumplimiento a los objetivos trazados en este trabajo y a partir de los resultados alcanzados se puede concluir que los Problemas de Planificación de Rutas de Vehículos son problemas de optimización combinatoria y una buena alternativa para solucionarlos son los algoritmos heurísticos y metaheurísticos. Las heurísticas de construcción presentan buenos resultados para estos problemas y son aplicables como punto de partida en el contexto de las metaheurísticas. Actualmente, existen escasos componentes de software que implementen métodos heurísticos para resolver variantes de VRP. De los componentes de software estudiados solo BICIAM, VRPH, BHCVRP, JSprit, VROOM y OR-Tools resuelven problemas de planificación de rutas de vehículos. Sin embargo, solo BHCVRP implementa heurísticas de construcción exclusivamente. Es importante destacar que este último resuelve solamente las variantes CVRP, MDVRP, HFVRP y TTRP.

La nueva versión de BHCVRP resuelve las deficiencias encontradas en la versión anterior. Se incorporan dos nuevas heurísticas de construcción: el Algoritmo de Ahorro basado en *Matching* y la Heurística de Inserción de Kilby. Además, se plasman un conjunto de mejoras en la arquitectura y funcionamiento de la biblioteca como son: la creación de la primera versión que cubre los lenguajes de programación Python y Java, el tratamiento de excepciones propias relacionadas con la distancia, capacidad de los vehículos, velocidad y tiempo de ejecución, así como la aplicación de principios de diseño como encapsulación de la variabilidad, *Hollywood*, cohesión y acoplamiento, *Open/Close* y *Don't Repeat Yourself*, y el patrón de diseño *Template*.

Los resultados alcanzados en los experimentos haciendo uso de la prueba estadística de Friedman permiten afirmar que en las variantes CVRP, TTRP, OVRP, SBRP y VRPTW, la heurística *SaveParall* es la de mejor rendimiento. Por otra parte, para las variantes HFVRP y MDVRP, la heurística más apropiada es *MJ*. En cuanto al tiempo de ejecución, se evidencia que la heurística *MJ* es la que emplea mayor cantidad para la mayoría de las variantes VRPs, pero alcanza buenos resultados. Sin embargo, el Método Aleatorio obtiene un tiempo de ejecución pequeño con los peores resultados para la mayoría de los casos.

A partir de la comparación realizada para las diferentes versiones de BHCVRP, se aprecia que no existen diferencias significativas. Por último, en el experimento comparativo realizado con la herramienta OR-Tools, BHCVRP obtiene mejores resultados en cuanto a costo en distancia euclidiana para las variantes CVRP y HFVRP, a pesar de tomar mayor cantidad de tiempo. Sin embargo, OR-Tools obtiene los mejores resultados globales para la variante VRPTW.

Recomendaciones

A partir del desarrollo de este trabajo se identifican un conjunto de aspectos a ser considerados en futuras iteraciones de este proceso de investigación. A continuación, se presentan las principales recomendaciones que se derivan de este trabajo:

- Incorporar otras variantes de VRP que reflejen nuevas características de la vida real. Por ejemplo, *Green VRP*, *Electric VRP*, *Dynamic VRP*, entre otras.
- Extender la biblioteca a partir de la incorporación de nuevas heurísticas de construcción, como puede ser la Heurística de Localización de Bramel y Simchi-Levi, el Algoritmo de Pétalos, entre otras.
- Incorporar nuevas excepciones propias, por ejemplo, para las heurísticas que poseen parámetros como las versiones del Algoritmo de Ahorros y las heurísticas de Inserción *Mole & Jameson*, *CMT* y *Kilby*.
- Incorporar experimentos con el resto de las configuraciones que brindan las variante HFVRP (orden *Ascending* y *Random*) y MDVRP (todas las asignaciones que brinda BHAVRP, excepto *BestNearest*).
- Realizar nuevos experimentos con diferentes tipos de distancias y compararlos con los resultados publicados en la literatura.
- Aplicar técnicas de minería de datos como árboles de decisión para determinar para qué instancias es adecuado utilizar determinada heurística.

Referencias bibliográficas

- [1] C. H. Papadimitriou and K. Steiglitz, *Combinatorial optimization: algorithms and complexity*. Courier Corporation, 1998.
- [2] P. Toth and D. Vigo, *The vehicle routing problem*. SIAM, 2002.
- [3] B. Eksioglu, A. V. Vural, and A. Reisman, "The vehicle routing problem: A taxonomic review," *Computers & Industrial Engineering*, vol. 57, no. 4, pp. 1472-1483, 2009.
- [4] S. N. Kumar and R. Panneerselvam, "A survey on the vehicle routing problem and its variants," *Intelligent Information Management*, vol. 04, no. 03, 2012.
- [5] G. B. Dantzig and J. H. Ramser, "The truck dispatching problem," *Management science*, vol. 6, no. 1, pp. 80-91, 1959.
- [6] D. L. Applegate, *The traveling salesman problem: a computational study*. Princeton university press, 2006.
- [7] G. Clarke and J. W. Wright, "Scheduling of vehicles from a central depot to a number of delivery points," *Operations research*, vol. 12, no. 4, pp. 568-581, 1964.
- [8] M. Zirour, "Vehicle routing problem: models and solutions," *Journal of Quality Measurement and Analysis JQMA*, vol. 4, no. 1, pp. 205-218, 2008.
- [9] G. Laporte, "The vehicle routing problem: An overview of exact and approximate algorithms," *European journal of operational research*, vol. 59, no. 3, pp. 345-358, 1992.
- [10] G. D. Konstantakopoulos, S. P. Gayialis, and E. P. Kechagias, "Vehicle routing problem and related algorithms for logistics distribution: A literature review and classification," *Operational research*, vol. 22, no. 3, pp. 2033-2062, 2022.
- [11] E. Arifta and F. Rakhmawati, "Analysis of Book Distribution Routes Using the Capacity Vehicle Routing Problem (CVRP) Method Using the Sweep Algorithm," *Sinkron: jurnal dan penelitian teknik informatika*, vol. 8, no. 1, pp. 360-367, 2023.

- [12] F. Morsidi and I. Y. Panessai, "Overview of the Integral Impact of MDVRP Routing Variables on Routing Heuristics," *Applied Information Technology And Computer Science*, vol. 4, no. 1, pp. 1723-1738, 2023.
- [13] V. R. Máximo, J.-F. Cordeau, and M. C. Nascimento, "An adaptive iterated local search heuristic for the Heterogeneous Fleet Vehicle Routing Problem," *Computers & Operations Research*, vol. 148, p. 105954, 2022.
- [14] L. Accorsi and D. Vigo, "A hybrid metaheuristic for single truck and trailer routing problems," *Transportation Science*, vol. 54, no. 5, pp. 1351-1371, 2020.
- [15] Y. Conde, "Biblioteca de heurísticas de construcción para el problema de ruteo de camiones y remolques," Universidad Tecnológica de La Habana José Antonio Echeverría CUJAE., La Habana, 2014.
- [16] Rita M. Newton and W. Thomas, "Design of School Bus Routes by Computer," *Socio-Economic Planning Sciences*, vol. 3, pp. 75-85, 1969.
- [17] V. Y. Piqueras, "Optimización heurística económica aplicada a las redes de transporte del tipo VRPTW," Phd, Departamento de Ingeniería de la Construcción y Proyectos de Ingeniería Civil, Universidad Politécnica de Valencia, Valencia, España, 2002.
- [18] M. A.-e. Asghari and S. M. J. Mirzapour, "Green vehicle routing problem: A state-of-the-art review," *International Journal of Production Economics*, vol. 231, p. 107899, 2021.
- [19] B. H. O. Rios, "Heuristics for vehicle routing problems with uncertainty= Heurísticas para problemas de roteamento de veículos com incertezas," [sn], 2023.
- [20] E. Ruiz y Ruiz, I. García-Calvillo, and S. Nucamendi-Guillén, "Open vehicle routing problem with split deliveries: mathematical formulations and a cutting-plane method," *Operational Research*, vol. 22, no. 2, pp. 1017-1037, 2022.
- [21] H. Qin, X. Su, T. Ren, and Z. Luo, "A review on the electric vehicle routing problems: Variants and algorithms," *Frontiers of Engineering Management*, vol. 8, pp. 370-389, 2021.

- [22] D. S. Hochba, "Approximation algorithms for NP-hard problems," *ACM Sigact News*, vol. 28, no. 2, pp. 40-52, 1997.
- [23] E.-G. Talbi, *Metaheuristics: from design to implementation*. Hoboken, New Jersey: John Wiley & Sons, Inc., 2009.
- [24] M. L. Fisher and R. Jaikumar, "A generalized assignment heuristic for vehicle routing," *Networks*, vol. 11, no. 2, pp. 109-124, 1981.
- [25] M. H. Romanycia and F. J. Pelletier, "What is a heuristic?," *Computational intelligence*, vol. 1, no. 1, pp. 47-58, 1985.
- [26] R. Martí, "Procedimientos metaheurísticos en optimización combinatoria," *Matemáticas, Universidad de Valencia*, vol. 1, no. 1, pp. 3-62, 2003.
- [27] W. J. Clancey, "Heuristic classification," *Artificial intelligence*, vol. 27, no. 3, pp. 289-350, 1985.
- [28] G. W. Nurcahyo, R. A. Alias, S. M. Shamsuddin, and M. N. M. Sap, "Sweep algorithm in vehicle routing problem for public transport," *Jurnal Antarabangsa Teknologi Maklumat*, vol. 2, pp. 51-64, 2002.
- [29] L. B. R. Medina, E. C. G. La Rota, and J. A. O. Castro, "Una revisión al estado del arte del problema de ruteo de vehículos: Evolución histórica y métodos de solución," *Ingeniería*, vol. 16, no. 2, pp. 35-55, 2011.
- [30] R. Mole and S. Jameson, "A sequential route-building algorithm employing a generalised savings criterion," *Journal of the Operational Research Society*, vol. 27, no. 2, pp. 503-511, 1976.
- [31] A. Olivera, "Heurísticas para problemas de ruteo de vehículos," *Reportes Técnicos 04-08*, 2004.
- [32] D. A. González Restrepo and D. Y. Gómez Veloza, "Solución al problema de ruteo de vehículos con entregas y recogidas aplicando el algoritmo de pétalos y la heurística del vecino más cercano," 2019.

- [33] A. C. P. Pérez, E. S. Ansola, and A. Rosete, "A metaheuristic solution for the school bus routing problem with homogeneous fleet and bus stop selection," *Ingeniería*, vol. 26, no. 2, pp. 233-253, 2021.
- [34] A. L. Infante Abreu, R. Díaz Hernández, M. André Ampuero, A. Rosete Suárez, J. Fajardo Calderín, and K. Escalera Fariñas, "Solución al problema de conformación de equipos de proyectos de software utilizando la biblioteca de clases BICIAM," *Revista Cubana de Ciencias Informáticas*, vol. 9, pp. 126-140, 2015.
- [35] M. Maischberger, "COIN-OR METSlib a Metaheuristics Framework in Modern C++," *Firenze University*. Recuperado de <https://projects.coin-or.org/metslib>, 2011.
- [36] C. Groër, B. Golden, and E. Wasil, "A library of local search heuristics for the vehicle routing problem," *Mathematical Programming Computation*, vol. 2, pp. 79-101, 2010.
- [37] K. Florios and G. Mavrotas, "Generation of the exact pareto set in multi-objective traveling salesman and set covering problems," *Applied Mathematics and Computation*, vol. 237, pp. 1-19, 2014.
- [38] L. Díaz, "Nueva versión de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos," Tesis de Diploma, Instituto Superior Politécnico José Antonio Echeverría, 2016.
- [39] I. Torres, "Componente de software: BHCVRP," *Transferencia Tecnológica CITI*, 2016.
- [40] F. Didier, L. Perron, S. Mohajeri, S. A. Gay, T. Cuvelier, and V. Furnon, "OR-Tools' vehicle routing solver: a generic constraint-programming solver with heuristic search for routing problems," 2023.
- [41] E. Lima, "CommuteVRP: otimização de serviços privados de transporte contínuo," in *Anais do XIII Simpósio Brasileiro de Sistemas de Informação*, 2017, pp. 547-554: SBC.
- [42] A. Mahmoud, T. Chouaki, S. Hörl, and J. Puchinger, "Extending JSprit to solve electric vehicle routing problems with recharging," *Procedia Computer Science*, vol. 201, pp. 289-295, March 22-25 2022.

- [43] M. Karkula, J. Duda, and I. Skalna, "Comparison of capabilities of recent open-source tools for solving Capacitated Vehicle Routing Problems with Time Windows," *Carpathian Logistics Congress*, Accessed on: December 2-4, 2019
- [44] M. PAREJO *et al.*, "Desarrollo de un algoritmo para seleccionar la ubicación de puntos de abastecimiento minimizando los tiempos y costos de transporte," *Revista Espacios*, vol. 41, no. 17, pp. 1-15, 2020.
- [45] O. E. Calderón Calderón, "Diseño e implementación de modelo matemático para la optimización del recorrido de camiones recolectores de residuos sólidos en el municipio de León," 2020.
- [46] S. C. Gupta, "Euclidean Distance in Optimization Problems," *Optimization Techniques*, vol. 3, no. 2, pp. 100-110, 2017.
- [47] J. Han, M. Kamber, and J. Pei, "Data Mining: Concepts and," *Techniques*, Waltham: Morgan Kaufmann Publishers, 2012.
- [48] W. Haversine, "The Haversine Formula: A Mathematical Approach to Calculate the Distance Between Two Points on the Earth," *Journal of Geodesy*, vol. 11, pp. 107-120, 2002.
- [49] B. K. Hachem, "Applications of Chebyshev Distance in Route Optimization Problems," *Computational Geometry*, vol. 25, pp. 85-92, 2016.
- [50] G. Developers. (29 de enero). *Distance Matrix API*. Available: <https://developers.google.com/maps/documentation/distance-matrix>.
- [51] GraphHopper. (29 de enero). *GraphHopper Directions API*. Available: <https://www.graphhopper.com/products/>
- [52] Mapbox. (29 de enero). *Navigation APIs*. Available: <https://docs.mapbox.com/api/navigation/>
- [53] M. Minghini, L. Juhász, P. Mooney, and G. Yeboah, "OpenStreetMap as a multidisciplinary nexus: perspectives, practices and procedures," *ISPRS International Journal of Geo-Information*, vol. 11, no. 4, p. 230, 2022.

- [54] Project_OSRM. (2024, 15 de febrero). *Open Source Routing Machine (OSRM)*. Available: <https://project-osrm.org/docs/v5.24.0/api/#>
- [55] P. Offermann, S. Blom, M. Schönherr, and U. Bub, "Artifact types in information systems design science—a literature review," in *Global Perspectives on Design Science Research: 5th International Conference, DESRIST 2010, St. Gallen, Switzerland, June 4-5, 2010. Proceedings*. 5, 2010, pp. 77-92: Springer.
- [56] Ananda de la Caridad Morales Morales, Dayana Vázquez Rodríguez, Isis Torres Pérez, and A. Rosete, "Comparison between BHCVRP and OR-Tools," in *VI Congreso Internacional de Ingeniería Informática y Sistemas de Información, XXI Convención Científica de Ingeniería y Arquitectura*, 2024.
- [57] A. Morales, "Nueva versión de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos," in *Jornada Científica Estudiantil a nivel de universidad*, CITI, CUJAE, 2024.
- [58] A. Morales, "Nueva versión de la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos," in *Jornada Científica Estudiantil a nivel de facultad*, Facultad de Ingeniería Informática, 2024.
- [59] M. Cuadrado and V. Griffin, "Modelos Matemáticos para la Optimización de la Distribución de Vehículos Nuevos en Venezuela. Caso: Clover International CA," *Ingeniería Industrial. Actualidad y Nuevas Tendencias*, vol. 1, no. 1, pp. 53-65, 2009.
- [60] H. S. Salazar, "Optimizacion Multiobjetivo Aplicado a un Problema de Ruta Corta Estocastico," 2004.
- [61] R. Matai, S. P. Singh, and M. L. Mittal, "Traveling salesman problem: an overview of applications, formulations, and solution approaches," *Traveling salesman problem, theory and applications*, vol. 1, no. 1, pp. 1-25, 2010.
- [62] K. L. Hoffman, M. Padberg, and G. Rinaldi, "Traveling salesman problem," *Encyclopedia of operations research and management science*, vol. 1, pp. 1573-1578, 2013.

- [63] M. R. Garey and D. S. Johnson, "A Guide to the Theory of NP-Completeness," *Computers and intractability*, pp. 37-79, 1990.
- [64] F. Liu, C. Lu, L. Gui, Q. Zhang, X. Tong, and M. Yuan, "Heuristics for vehicle routing problem: A survey and recent advances," *arXiv preprint arXiv:2303.04147*, 2023.
- [65] M. Salehi Sarbijan and J. Behnamian, "Emerging research fields in vehicle routing problem: a short review," *Archives of Computational Methods in Engineering*, vol. 30, no. 4, pp. 2473-2491, 2023.
- [66] J.-F. Cordeau and G. d. é. e. d. r. e. a. d. décisions, *The VRP with time windows*. Citeseer, 2000.
- [67] R. Baldacci, P. Toth, and D. Vigo, "Exact algorithms for routing problems under vehicle capacity constraints," *Annals of Operations Research*, vol. 175, pp. 213-245, 2010.
- [68] J. L. Franco and S. N. Isaza, "Heurística para la Generación de un Conjunto de Referencia de Soluciones que Resuelvan el Problema de Ruteo de Vehículos con Múltiples Depósitos MDVRP," 2012.
- [69] D. I. G. MATEOS, A. G. Gómez, D. D. A. MARTINO, J. P. GARCÍA, and N. G. Fernández, "Desarrollo de un método híbrido para la resolución del MDVRP," *Revista de la Escuela Jacobea de Posgrado* <http://revista.jacobe.edu.mx>, no. 5, pp. 45-64, 2013.
- [70] B. Golden, A. Assad, L. Levy, and F. Gheysens, "The fleet size and mix vehicle routing problem," *Computers & Operations Research*, vol. 11, no. 1, pp. 49-66, 1984.
- [71] S.-W. Lin, F. Y. Vincent, and S.-Y. Chou, "Solving the truck and trailer routing problem based on a simulated annealing heuristic," *Computers & Operations Research*, vol. 36, no. 5, pp. 1683-1692, 2009.
- [72] E. E. Zachariadis and C. T. Kiranoudis, "An open vehicle routing problem metaheuristic for examining wide solution neighborhoods," *Computers & Operations Research*, vol. 37, no. 4, pp. 712-723, 2010.

- [73] A. F. Tangarife Álvarez, "Revisión del estado del arte del problema de ruteo abierto (OVRP)," 2017.
- [74] *DVRP-OCR: Un Método Cooperativo Basado en Heurísticas Simples para el DVRP*, 2012.
- [75] M. Dror, G. Laporte, and P. Trudeau, "Vehicle routing with split deliveries," *Discrete Applied Mathematics*, vol. 50, no. 3, pp. 239-254, 1994.
- [76] S. Baptista, R. C. Oliveira, and E. Zúquete, "A period vehicle routing case study," *European Journal of Operational Research*, vol. 139, no. 2, pp. 220-229, 2002.
- [77] G. Laporte and F. V. Louveaux, *Solving stochastic routing problems with the integer L-shaped method*. Springer, 1998.
- [78] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuyse, "The vehicle routing problem: State of the art classification and review," *Computers & industrial engineering*, vol. 99, pp. 300-313, 2016.
- [79] J. F. Bard, L. Huang, M. Dror, and P. Jaillet, "A branch and cut algorithm for the VRP with satellite facilities," *IIE transactions*, vol. 30, no. 9, pp. 821-834, 1998.
- [80] J. Puchinger and G. R. Raidl, "Combining metaheuristics and exact algorithms in combinatorial optimization: A survey and classification," in *International work-conference on the interplay between natural and artificial computation*, 2005, pp. 41-53: Springer.
- [81] F. Narducci, "Programación de talleres intermitentes flexibles, por medio de la heurística del margen de tolerancia. 117 p," Tesis de maestría (Ingeniería Industrial). Universidad del Norte. Barranquilla, 2005.
- [82] V. J. Rayward-Smith, I. H. Osman, C. R. Reeves, and G. D. Smith, *Modern heuristic search methods*. Wiley, 1996.
- [83] J. M. Daza, J. R. Montoya, and F. Narducci, "Resolución del problema de enrutamiento de vehículos con limitaciones de capacidad utilizando un procedimiento metaheurístico de dos fases," *Revista EIA*, no. 12, pp. 23-38, 2009.

- [84] B. E. Gillett and L. R. Miller, "A heuristic algorithm for the vehicle-dispatch problem," *Operations research*, vol. 22, no. 2, pp. 340-349, 1974.
- [85] H. Gabow, "An Efficient Implementation of Edmonds' Maximum-Matching Algorithm," Stanford University, 1972 (Technical Report No. 31). <ftp://reports.stanford.edu/pubs/edmonds/>...1972.
- [86] N. Christofides, "The vehicle routing problem," *Combinatorial optimization*, 1979.
- [87] J. Bramel and D. Simchi-Levi, "A location based heuristic for general routing problems," *Operations research*, vol. 43, no. 4, pp. 649-660, 1995.
- [88] I. M. Chao, "A tabu search method for the truck and trailer routing problem," *Computers & Operations Research*, vol. 29, pp. 33-51, 2002.
- [89] V. R. d. Souza, "Uma análise do framework OptaPlanner aplicado ao problema de empacotamento unidimensional," *Research Gate*, 2 de febrero de 2020.
- [90] L. d. I. C. M. Lores, "Operadores de mutación basados en heurísticas de construcción para Problemas de Planificación de Rutas de Vehículos," Trabajo de diploma para optar por el título de Ingeniería en Informática, Facultad de Ingeniería Informática, Instituto Superior Politécnico "José Antonio Echeverría", La Habana, Cuba, 2017.
- [91] E. H. Fernández, "Nueva versión de la Capa de servicio para soluciones a Problemas de Planificación de Rutas de Vehículos," Trabajo de diploma para optar por el título de Ingeniería en Informática, Facultad de Ingeniería Informática, Instituto Superior Politécnico "José Antonio Echeverría", La Habana, Cuba, 2016.
- [92] SciPy. (2025, 29 de enero). *SciPy documentation*. Available: <https://docs.scipy.org/doc/scipy/>
- [93] Geopy. (29 de enero). *Geopy: Geocoding, Reverse Geocoding, Distance Calculations*. Available: <https://pypi.org/project/geopy/>
- [94] F. Pedregosa, "Scikit-learn: Machine learning in python Fabian," *Journal of machine learning research*, vol. 12, p. 2825, 2011.

- [95] R. Nerey, "Herramienta para la generación de rutas en el problema del transporte obrero: Trans-o," Tesis de Diploma, Facultad de Ingeniería Informática, Universidad Tecnológica de La Habana "José Antonio Echeverría" (CUJAE), La Habana, 2017.
- [96] N. Pérez, "Incorporación de algoritmos de agrupamiento en la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos," Tesis de diploma, Facultad de Ingeniería Informática, Universidad Tecnológica de La Habana José Antonio Echeverría (CUJAE), La Habana, 2022.
- [97] I. Sommerville and A. Wesley, "Introducción a la Ingeniería de Software," *I. Sommerville, Ingeniería de Software. Pearson Educación*, 2011.
- [98] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design patterns: elements of reusable object-oriented software*. Pearson Deutschland GmbH, 1995.
- [99] B. Eckel, *Thinking in Patterns with Java*. President, MindView, Inc., 2003.
- [100] F. Martínez and J. Cueva, "Guía de Construcción de Software en Java con Patrones de Diseño, 2000," *Universidad de Oviedo: Escuela Universitaria de Ingeniería Técnica Informática*.
- [101] M. Desrochers and T. Verhoog, "A new heuristic for the fleet size and mix vehicle routing problem," *Computers & Operations Research*, vol. 18, no. 3, pp. 263-274, 1991.
- [102] K. Altinkemer and B. Gavish, "Parallel savings based heuristics for the delivery problem," *Operations research*, vol. 39, no. 3, pp. 456-469, 1991.
- [103] P. Wark and J. Holt, "A repeated matching heuristic for the vehicle routing problem," *Journal of the Operational Research Society*, vol. 45, no. 10, pp. 1156-1167, 1994.
- [104] N. Tok and Ş. Özkar, "Solution Of Capacitated Vehicle Routing Problem For A Food Delivery Company With Heuristic Methods," *International Review of Economics and Management*, vol. 11, no. 1, pp. 1-16, 2023.
- [105] C. B. Reynoso, "Introducción a la Arquitectura de Software," *Universidad de Buenos Aires*, vol. 33, 2004.

- [106] V. K. Madasu, T. Venna, T. Eltaeib, M. Moalla, N. Almuslet, and A. Badaoui, "Solid principles in software architecture and introduction to resm concept in oop," *Studies*, vol. 35, no. 38, p. 8, 2015.
- [107] C. Larman, "Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development," *Pearson Education*, 2002.
- [108] R. Capilla Sevilla, "Temas Diseño y Arquitectura del Software," 2022.
- [109] P. Lalanda, "Style-specific techniques to design product-line architectures," ed: Thomson-CSF Corporate Research Laboratory, Domaine de Corbeville, Orsay, France, 1999.
- [110] D. Thomas and A. Hunt, *The Pragmatic Programmer: your journey to mastery*. Addison-Wesley Professional, 2019.
- [111] E. Alba, "Networking and emerging optimization," Available: <http://neo.lcc.uma.es/vrp/vrp-instances/>
- [112] J. Homberger and H. Gehring, "Two evolutionary metaheuristics for the vehicle routing problem with time windows," *INFOR: Information Systems and Operational Research*, vol. 37, no. 3, pp. 297-318, 1999.
- [113] P. Schittekat, J. Kinable, K. Sörensen, M. Sevaux, F. Spieksma, and J. Springael, "A metaheuristic for the school bus routing problem with bus stop selection," *European Journal of Operational Research*, vol. 229, no. 2, pp. 518-528, 2013.
- [114] PyCharm. (2024, 25 de agosto). *Install PyCharm | PyCharm Documentation*. Available: <https://www.jetbrains.com/help/pycharm/installation-guide.html>
- [115] ApacheNetBeans. (2024, 25 de agosto). *Welcome to Apache NetBeans*. Available: <https://netbeans.apache.org/front/main/index.html>
- [116] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical test as methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 3-18, 2011.

[117] M. Friedman, "The use of ranks to avoid the assumption of normality implicit in the analysis of variance," *Journal of the American Statistical Association*, vol. 32, no. 200, pp. 674-701, 1937.

Anexo 1

Descripción de las instancias:

Tabla 34: Descripción de las instancias CVRP y OVRP.

Problema	Total de clientes	Total de vehículos	Capacidad de los vehículos
1	50	5	200
2	75	9	250
3	75	3	480
4	100	5	400
5	360	5	500
6	200	50	200
7	400	100	200
8	600	150	200

Tabla 35: Descripción de las instancias MDVRP.

Problema	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
1	50	4	4	80
2	100	2	5	200
3	160	4	5	60
4	360	9	5	60
5	288	4	6	175
6	249	4	8	500
7	50	4	2	160
8	144	4	3	190

Tabla 36: Descripción de las instancias HFVRP.

Datos originales			Flota de Vehículos	
Problema	Total de clientes	Total de vehículos	Total	Capacidad
1	50	5	1	200
			1	400
			1	100
			1	250
			1	50
2	75	9	2	250
			2	100
			2	455
			1	90
			1	50
			1	500
3	75	3	2	470
			1	500
4	100	5	1	400
			1	100
			1	200
			2	650
5	360	5	3	500
			1	400
			1	600
6	200	50	25	240
			25	160
7	400	100	50	320
			50	80
8	600	150	75	220
			75	180

Tabla 37: Descripción de las instancias TTRP.

Problema	Clientes			Camiones		Remolques	
	Total	VC	TC	Total	Capacidad	Total	Capacidad
1	50	38	12	5	100	3	100
2	75	38	37	9	100	5	100
3	100	50	50	8	150	4	100
4	120	60	60	7	150	4	100
5	100	50	50	10	150	5	100
6	199	100	99	17	150	9	100
7	50	13	37	5	100	3	100
8	150	38	112	12	150	6	100

Tabla 38: Descripción de las instancias VRPTW.

Problema	Total de clientes	Total de vehículos	Capacidad de los vehículos	Rango de intervalos en ventanas de tiempo
1	200	50	200	[1, 1100]
2	200	50	200	[32, 1158]
3	200	50	200	[53, 1153]
4	200	50	200	[71, 1183]
5	400	100	200	[30, 1192]
6	400	100	200	[1, 10]
7	600	150	200	[1, 1150]
8	600	150	200	[10, 1403]

Tabla 39: Descripción de las instancias SBRP.

Problema	Total de pasajeros	Total de vehículos	Capacidad de los vehículos	Máxima distancia a caminar	Total de paradas	Máxima capacidad de las paradas
1	100	10	25	40	10	15
2	800	80	25	10	72	15
3	100	5	50	177	40	15
4	800	40	25	210	240	15
5	100	20	25	153	19	15
6	50	10	50	222	24	15
7	400	20	50	202	120	15
8	1600	80	25	162	480	15

Anexo 2

Resultados obtenidos en las versiones de BHCVRP (*Java_anterior*, *Java_actual* y *Python_actual*):

Tabla 40: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante CVRP en Python.

<i>i</i>	algorithm	$z = (R_0 - R_i)/SE$	<i>p</i>	Holm	Finner	Li
9	Sweep	4.70662	0.000003	0.005556	0.005683	0.013622
8	Random	4.624048	0.000004	0.00625	0.011334	0.013622
7	FJ	4.046042	0.000052	0.007143	0.016952	0.013622
6	NN	3.715753	0.000203	0.008333	0.022539	0.013622
5	Kilby	2.724885	0.006432	0.01	0.028094	0.013622
4	SaveMatch	2.477168	0.013243	0.0125	0.033617	0.013622
3	SaveSeq	0.660578	0.508883	0.016667	0.039109	0.013622
2	CMT	0.660578	0.508883	0.025	0.04457	0.013622
1	MJ	0.330289	0.741182	0.05	0.05	0.05

Tabla 41: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante HFVRP en Python.

<i>i</i>	algorithm	$z = (R_0 - R_i)/SE$	<i>p</i>	Holm	Finner	Li
9	Random	4.624048	0.000004	0.005556	0.005683	0.006904
8	Sweep	4.087328	0.000044	0.00625	0.011334	0.006904
7	FJ	2.972602	0.002953	0.007143	0.016952	0.006904
6	Kilby	2.807458	0.004993	0.008333	0.022539	0.006904
5	NN	2.766171	0.005672	0.01	0.028094	0.006904
4	SaveMatch	2.724885	0.006432	0.0125	0.033617	0.006904
3	SaveParall	0.247717	0.804353	0.016667	0.039109	0.006904
2	CMT	0.247717	0.804353	0.025	0.04457	0.006904
1	SaveSeq	0.165145	0.86883	0.05	0.05	0.05

Tabla 42: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante MDVRP en Python.

<i>i</i>	algorithm	$z = (R_0 - R_i)/SE$	<i>p</i>	Holm	Finner	Li
9	FJ	4.871765	0.000001	0.005556	0.005683	0.033488
8	Sweep	4.458903	0.000008	0.00625	0.011334	0.033488
7	NN	4.376331	0.000012	0.007143	0.016952	0.033488
6	Random	4.293759	0.000018	0.008333	0.022539	0.033488
5	SaveMatch	3.96347	0.000074	0.01	0.028094	0.033488
4	Kilby	3.137747	0.001703	0.0125	0.033617	0.033488
3	SaveSeq	2.642313	0.008234	0.016667	0.039109	0.033488
2	SaveParall	1.07344	0.283074	0.025	0.04457	0.033488
1	CMT	0.908295	0.363722	0.05	0.05	0.05

Tabla 43: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante TTRP en Python.

i	algorithm	$z = (R_0 - R_i)/SE$	p	Holm	Finner	Li
9	Sweep	5.614915	0	0.005556	0.005683	0.031107
8	Random	4.954337	0.000001	0.00625	0.011334	0.031107
7	FJ	4.541476	0.000006	0.007143	0.016952	0.031107
6	NN	4.046042	0.000052	0.008333	0.022539	0.031107
5	Kilby	3.550608	0.000384	0.01	0.028094	0.031107
4	SaveMatch	2.477168	0.013243	0.0125	0.033617	0.031107
3	CMT	1.981735	0.047509	0.016667	0.039109	0.031107
2	SaveSeq	0.908295	0.363722	0.025	0.04457	0.031107
1	MJ	0.825723	0.408961	0.05	0.05	0.05

Tabla 44: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante SBRP en Python.

i	algorithm	$z = (R_0 - R_i)/SE$	p	Holm	Finner	Li
9	SaveMatch	4.433601	0.000009	0.005556	0.005683	0.034718
8	FJ	4.433601	0.000009	0.00625	0.011334	0.034718
7	CMT	2.669695	0.007592	0.007143	0.016952	0.034718
6	Random	2.383656	0.017142	0.008333	0.022539	0.034718
5	Kilby	2.383656	0.017142	0.01	0.028094	0.034718
4	NN	1.906925	0.05653	0.0125	0.033617	0.034718
3	Sweep	1.906925	0.05653	0.016667	0.039109	0.034718
2	MJ	1.811579	0.070051	0.025	0.04457	0.034718
1	SaveSeq	0.953463	0.340356	0.05	0.05	0.05

Tabla 45: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante VRPTW en Python.

i	algorithm	$z = (R_0 - R_i)/SE$	p	Holm	Finner	Li
9	Random	4.28231	0.000018	0.005556	0.005683	0.034357
8	NN	4.177864	0.000029	0.00625	0.011334	0.034357
7	Sweep	3.760077	0.00017	0.007143	0.016952	0.034357
6	FJ	2.924505	0.00345	0.008333	0.022539	0.034357
5	SaveMatch	2.715611	0.006615	0.01	0.028094	0.034357
4	CMT	1.984485	0.047202	0.0125	0.033617	0.034357
3	Kilby	1.462252	0.143672	0.016667	0.039109	0.034357
2	MJ	1.253359	0.210075	0.025	0.04457	0.034357
1	SaveSeq	0.940019	0.347208	0.05	0.05	0.05

Tabla 46: Resultados de los métodos *post-hoc* Li, Finner y Holm para la variante OVRP en Python.

i	algorithm	$z = (R_0 - R_i)/SE$	p	Holm	Finner	Li
9	Sweep	5.367198	0	0.005556	0.005683	0.028558
8	Random	5.284626	0	0.00625	0.011334	0.028558
7	NN	4.293759	0.000018	0.007143	0.016952	0.028558
6	FJ	3.880897	0.000104	0.008333	0.022539	0.028558
5	Kilby	3.055174	0.002249	0.01	0.028094	0.028558
4	SaveMatch	2.642313	0.008234	0.0125	0.033617	0.028558
3	CMT	1.899162	0.057543	0.016667	0.039109	0.028558
2	SaveSeq	0.908295	0.363722	0.025	0.04457	0.028558
1	MJ	0.743151	0.457391	0.05	0.05	0.05

Tabla 47: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias CVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1487.59 ₍₄₎	1702.14	0.034	1092.39 ₍₅₎	1233.61	0.105	567.63 ₍₄₎	575.52	0.087	561.64 ₍₄₎	561.64	0.196
2	2190.94 ₍₆₎	2392.85	0.047	1579.44 ₍₆₎	1749.63	0.087	759.68 ₍₆₎	786.85	0.149	723.2 ₍₆₎	724.73	0.692
3	2126.94 ₍₃₎	2310.81	0.032	630.41 ₍₃₎	632.28	0.123	659.32 ₍₃₎	671.75	0.137	631.85 ₍₃₎	631.85	0.690
4	3082.93 ₍₄₎	3256.04	0.039	1932.01 ₍₄₎	2049.58	0.133	841.33 ₍₄₎	859.02	0.175	762.48 ₍₄₎	762.48	1.644
5	57163.43 ₍₄₎	58923.64	0.049	19443.38 ₍₄₎	20534.06	1.400	6145.42 ₍₄₎	6258.79	1.435	5374.23 ₍₄₎	5374.23	233.309
6	14512.57 ₍₁₈₎	14832.38	0.039	9070.42 ₍₁₉₎	9341.01	0.361	2861.28 ₍₁₈₎	2936.79	0.376	2628.41 ₍₁₉₎	2629.57	22.909
7	41197.21 ₍₃₇₎	42051.26	0.077	27385.16 ₍₃₇₎	27979.17	1.437	7516.1 ₍₃₆₎	7648.00	2.016	6871.46 ₍₃₈₎	6871.46	377.605
8	86706.52 ₍₅₈₎	87593.49	0.088	57024.2 ₍₅₇₎	58727.53	6.434	16998.52 ₍₅₇₎	17235.64	451.87	13678.65 ₍₅₇₎	13678.65	1951.639

Tabla 48: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias CVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	610.01 ₍₄₎	611.59	0.244	741.3 ₍₅₎	854.59	0.114	1298.37 ₍₄₎	1367.20	0.045
2	843.93 ₍₆₎	848.90	0.366	919.7 ₍₇₎	978.99	0.132	1772.65 ₍₆₎	1851.70	0.079
3	659.28 ₍₃₎	664.63	0.482	954.37 ₍₄₎	1004.02	0.201	1728.84 ₍₃₎	1740.60	0.057
4	827.68 ₍₄₎	835.15	0.867	1172.05 ₍₅₎	1286.96	0.168	2134.69 ₍₄₎	2158.63	0.068
5	6181.64 ₍₄₎	6266.93	61.637	16650.47 ₍₅₎	17953.30	1.182	70053.21 ₍₄₎	70053.21	0.157
6	2848.17 ₍₁₈₎	2947.41	1.732	3451.65 ₍₂₁₎	3582.78	0.506	7701.55 ₍₁₉₎	7745.33	0.081
7	7353.73 ₍₃₆₎	7420.82	9.114	9495.97 ₍₄₄₎	9773.22	3.424	22059.61 ₍₃₈₎	22205.85	0.150
8	14919.46 ₍₅₆₎	14948.77	41.346	18770.32 ₍₆₇₎	19176.97	18.213	45755.14 ₍₅₇₎	46104.44	0.361

Tabla 49: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias HFVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1313.9 ₍₃₎	1457.71	0.024	957.99 ₍₃₎	1001.56	0.077	524.66 ₍₃₎	527.10	0.059	536.53 ₍₃₎	536.53	0.270
2	2205.73 ₍₃₎	2275.20	0.025	1273.59 ₍₃₎	1411.39	0.080	645.89 ₍₃₎	652.98	0.093	851.96 ₍₇₎	851.96	1.181
3	2124.18 ₍₃₎	2264.74	0.026	1340.46 ₍₃₎	1429.92	0.106	653.88 ₍₃₎	662.28	0.094	869.89 ₍₆₎	879.66	2.701
4	2983.77 ₍₃₎	3123.60	0.036	1899.58 ₍₃₎	1962.89	0.266	768.28 ₍₃₎	774.38	0.409	1024.31 ₍₅₎	1070.66	5.8528
5	58491.9 ₍₄₎	61785.35	0.037	20348.32 ₍₄₎	22736.22	1.033	6255.43 ₍₄₎	6397.67	1.092	7400.11 ₍₆₎	7470.81	3.8310
6	14529.19 ₍₁₆₎	14755.53	0.091	8904.08 ₍₁₅₎	9131.46	0.547	2534.17 ₍₁₅₎	2588.6	0.673	1290.53 ₍₁₎	1308.28	17.616
7	40180.34 ₍₂₃₎	41402.19	0.123	25685.67 ₍₂₃₎	26688.21	2.158	5511.25 ₍₂₃₎	5597.72	2.527	5955.38 ₍₅₎	5970.02	4.4937
8	85814.92 ₍₅₂₎	87507.12	0.070	56058.93 ₍₅₂₎	57878.02	7.989	15294.68 ₍₅₁₎	15672.3	20.9242	16967.84 ₍₈₎	977.43	7.9899

Tabla 50: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias HFVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	536.15 ₍₃₎	538.77	0.186	652.83 ₍₃₎	690.15	0.076	1280.30 ₍₃₎	1280.31	0.019
2	681.73 ₍₃₎	686.02	0.358	911.82 ₍₄₎	972.90	0.102	1720.60 ₍₃₎	1723.95	0.059
3	662.41 ₍₃₎	665.11	0.372	922.66 ₍₃₎	993.36	0.098	1725.50 ₍₃₎	1731.74	0.037
4	782.9 ₍₃₎	784.41	4.810	1180.74 ₍₃₎	1242.09	0.867	2122.11 ₍₃₎	2123.70	0.087
5	6453.51 ₍₄₎	6537.06	51.711	16345.57 ₍₅₎	20324.56	1.245	70056.72 ₍₄₎	70279.88	0.138
6	2711.9 ₍₁₅₎	2714.86	2.892	3189.95 ₍₁₇₎	3454.43	0.698	7890.30 ₍₁₅₎	7904.14	0.221
7	5426.43 ₍₂₃₎	5545.46	15.457	7739.26 ₍₂₅₎	8646.55	2.866	20987.24 ₍₂₄₎	21087.88	0.271
8	13949.38 ₍₅₁₎	13952.11	50.595	20424.46 ₍₅₄₎	22242.66	17.180	45301.08 ₍₅₂₎	45417.50	0.448

Tabla 51: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias MDVRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	900.42 ₍₁₂₎	957.80	0.032	802.75 ₍₁₂₎	826.44	0.037	636.64 ₍₁₂₎	646.41	0.048	626.2 ₍₁₃₎	626.2	0.058
2	2361.66 ₍₉₎	2563.63	0.040	1373.25 ₍₉₎	1501.49	0.082	878.92 ₍₉₎	901.40	0.115	833.09 ₍₉₎	833.09	0.324
3	7956.15 ₍₁₆₎	8193.67	0.043	4820.68 ₍₁₆₎	5110.09	0.132	3273.09 ₍₁₆₎	3312.51	0.133	3302.04 ₍₁₆₎	3302.04	0.280
4	18286.31 ₍₃₆₎	18825.05	0.064	11459.03 ₍₃₇₎	11864.58	0.211	7456.18 ₍₃₆₎	7543.33	0.262	7429.57 ₍₃₆₎	7429.57	0.583
5	10140.46 ₍₂₃₎	10529.67	0.065	6203.79 ₍₂₃₎	6387.67	0.189	3344.44 ₍₂₂₎	3437.04	0.335	3016.17 ₍₂₃₎	3016.17	1.888
6	13518.87 ₍₂₇₎	14134.88	0.062	7890.4 ₍₂₇₎	8066.29	0.160	4586.49 ₍₂₇₎	4727.56	0.261	3993.25 ₍₂₇₎	3993.25	1.241
7	856.98 ₍₇₎	902.52	0.029	704.23 ₍₇₎	740.36	0.032	525.66 ₍₇₎	529.71	0.048	528.52 ₍₇₎	528.52	0.059
8	5309.65 ₍₁₁₎	5609.60	0.040	3408.17 ₍₁₁₎	3621.49	0.081	2078.62 ₍₁₁₎	2123.68	0.115	1951.45 ₍₁₂₎	1951.45	0.226

Tabla 52: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias MDVRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	639.01 ₍₁₂₎	639.01	0.062	733.42 ₍₁₄₎	759.23	0.044	823.08 ₍₁₂₎	850.86	0.030
2	884.87 ₍₉₎	894.12	0.216	1114.1 ₍₁₁₎	1155.09	0.106	1660.64 ₍₉₎	1691.91	0.052
3	3237.39 ₍₁₆₎	3237.39	0.303	3943.05 ₍₂₁₎	4288.36	0.141	6067.77 ₍₁₆₎	6106.68	0.059
4	7284.11 ₍₃₆₎	7284.11	0.995	9646.23 ₍₄₅₎	10024.89	0.308	12779.03 ₍₃₆₎	12862.69	0.077
5	3277.74 ₍₂₃₎	3287.15	1.320	4421.83 ₍₂₆₎	4602.53	0.374	7486.01 ₍₂₃₎	7632.53	0.078
6	4341.31 ₍₂₇₎	4343.88	0.701	5489.8 ₍₃₀₎	5815.21	0.240	9081.2 ₍₂₈₎	9156.59	0.101
7	526.21 ₍₇₎	526.21	0.065	611.04 ₍₇₎	643.19	0.057	756.73 ₍₇₎	773.11	0.033
8	2041.57 ₍₁₁₎	2089.05	0.348	2744.63 ₍₁₃₎	2879.68	0.143	4042.76 ₍₁₁₎	4110.23	0.051

Tabla 53: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias TTRP (I).

Problema	Random			NN			SaveSeq			SaveParall		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	1553.98 ₍₆₎	1627.30	0.052	1167.8 ₍₅₎	1259.35	0.078	602.8 ₍₁₎	602.8	0.117	765.25 ₍₅₎	765.25	0.196
2	2528.93 ₍₁₃₎	2672.96	0.127	1798.35 ₍₁₁₎	1877.47	0.256	1020.68 ₍₂₎	1033.49	0.273	1809.01 ₍₈₎	1820.58	1.4296
3	3229.75 ₍₉₎	3423.11	0.051	2226.32 ₍₈₎	2321.50	0.231	1085.56 ₍₂₎	1106.50	0.391	1869.89 ₍₆₎	1879.66	2.701
4	6122.35 ₍₇₎	6371.67	0.072	2059.33 ₍₈₎	2152.60	0.368	998.48 ₍₁₎	998.48	0.411	1024.31 ₍₅₎	1070.66	5.8528
5	3831.82 ₍₁₁₎	4085.60	0.086	1382.79 ₍₉₎	1733.76	0.316	1012.99 ₍₃₎	1101.46	0.323	1740.11 ₍₆₎	1747.81	3.8310
6	6491.96 ₍₂₂₎	6679.67	0.066	4416.71 ₍₂₀₎	4553.98	1.191	1834.74 ₍₂₎	1888.82	1.138	1990.53 ₍₁₃₎	2308.28	17.616
7	1519.75 ₍₆₎	1632.47	0.040	1158.41 ₍₆₎	1242.58	0.104	729.48 ₍₂₎	782.74	0.128	1595.38 ₍₅₎	1597.02	0.4937
8	5020.74 ₍₁₀₎	5140.10	0.056	3299.86 ₍₁₃₎	3440.13	0.618	1157.44 ₍₁₎	1166.62	0.613	1967.84 ₍₈₎	1977.43	7.9899

Tabla 54: Resumen de los resultados obtenidos por la versión anterior de BHCVRP [38] en las ocho instancias TTRP (II).

Problema	MJ			CMT			Sweep		
	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)	Min	Prom	Tiempo (s)
1	793.95 ₍₅₎	793.95	0.236	860.06 ₍₉₎	934.13	0.217	1419.09 ₍₆₎	1535.37	0.2584
2	1265.01 ₍₁₀₎	1269.96	0.455	1352.59 ₍₁₅₎	1417.89	0.421	2396.65 ₍₁₂₎	2529.72	0.7926
3	1180.2 ₍₉₎	1185.81	1.049	1336.94 ₍₁₁₎	1423.57	0.326	3126.42 ₍₉₎	3323.73	1.8182
4	1646.93 ₍₈₎	1646.97	2.480	1761.11 ₍₁₁₎	1835.43	0.524	5918.16 ₍₉₎	6381.68	3.0313
5	1202.0 ₍₁₀₎	1202.0	1.071	1340.68 ₍₁₃₎	1442.07	0.439	3677.54 ₍₁₂₎	3873.64	1.8079
6	1942.7 ₍₁₉₎	1945.89	5.621	2278.04 ₍₂₅₎	2353.39	1.770	6521.21 ₍₂₂₎	6634.31	12.2426
7	871.83 ₍₅₎	874.40	0.255	843.97 ₍₉₎	917.22	0.150	1515.52 ₍₇₎	1601.22	0.2534
8	1578.05 ₍₁₁₎	1603.21	3.825	1883.48 ₍₁₆₎	1986.04	0.741	4848.46 ₍₁₃₎	4944.19	5.7698

Tabla 55: Rankings de las heurísticas para la variante CVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.

Algoritmos	Rankings		
	BHCVRP_anterior_Java	BHCVRP_actual_Java	BHCVRP_actual_Python
Random	6.875	6.75	6.375
NN	5	5.375	5.25
SaveSeq	2.625	2.5	2.875
SaveParall	1	1.375	1.875
MJ	2.625	2.5	2.375
CMT	4.125	3.625	2.875
Sweep	5.75	5.875	6.375

Tabla 56: Rankings de las heurísticas para la variante HFVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.

Algoritmos	Rankings		
	BHCVRP_anterior_Java	BHCVRP_actual_Java	BHCVRP_actual_Python
Random	6.875	6.875	6.75
NN	5.375	5.3125	5.0625
SaveSeq	1.5	1.75	2.5
SaveParall	2.375	2.375	2.625
MJ	2.125	2	2.25
CMT	4	3.875	2.625
Sweep	5.75	5.8125	6.1875

Tabla 57: Rankings de las heurísticas para la variante MDVRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.

Algoritmos	Rankings		
	<i>BHCVRP_anterior_Java</i>	<i>BHCVRP_actual_Java</i>	<i>BHCVRP_actual_Python</i>
<i>Random</i>	7	6.875	5.75
<i>NN</i>	5	5.25	5.875
<i>SaveSeq</i>	3	3.25	4.625
<i>SaveParall</i>	1.375	1.75	2.625
<i>MJ</i>	1.625	1.375	1
<i>CMT</i>	4	3.75	2.375
<i>Sweep</i>	6	5.75	5.75

Tabla 58: Rankings de las heurísticas para la variante TTRP en cuanto a calidad de soluciones en distancia euclidiana para las tres versiones de BHCVRP.

Algoritmos	Rankings		
	<i>BHCVRP_anterior_Java</i>	<i>BHCVRP_actual_Java</i>	<i>BHCVRP_actual_Python</i>
<i>Random</i>	6.875	6.625	6
<i>NN</i>	4.75	4.75	5
<i>SaveSeq</i>	1	1.25	2.5
<i>SaveParall</i>	3.5	3.5	1.125
<i>MJ</i>	2.25	2.125	2.375
<i>CMT</i>	3.5	3.375	4
<i>Sweep</i>	6.125	6.375	7

Tabla 59: Resultados de los métodos *post-hoc* Li, Finner y Holm para el *ranking* general de Friedman de las diferentes versiones de BHCVRP.

Versión de BHCVRP	p-valor sin ajustar	p-valor de Li	p-valor de Finner	p-valor de Holm
<i>Python_actual</i>	0.349575	0.49265	0.576947	0.69915
<i>Java_anterior</i>	0.639994	0.639994	0.639994	0.69915